

Chapter 4

GDMA Controller (GDMA-AHB, GDMA-AXI)

4.1 Overview

General Direct Memory Access (GDMA) is a feature that allows peripheral-to-memory, memory-to-peripheral, and memory-to-memory data transfer at high speed. The CPU is not involved in the GDMA transfer and therefore is more efficient with less workload.

ESP32-P4 has two types of general-purpose DMA controllers, namely GDMA-AHB and GDMA-AXI, to directly access the AHB bus or the AXI bus respectively. Both GDMA-AHB and GDMA-AXI have six independent channels, i.e., three transmit channels and three receive channels.

The GDMA-AHB channels are shared and can be assigned to I3C, UHCI, I2S, ADC, or RMT to access internal and external memory.

The GDMA-AXI channels are also shared and can be assigned to LCD, CAM, two general-purpose SPIs, PARLIO, AES, or SHA to access internal and external memory.

GDMA-AHB and GDMA-AXI use configurable priority and weight arbitration schemes to manage peripherals' needs for bandwidth.

Unless otherwise specified, the term "GDMA controller" refers to both GDMA-AHB and GDMA-AXI in the following text.

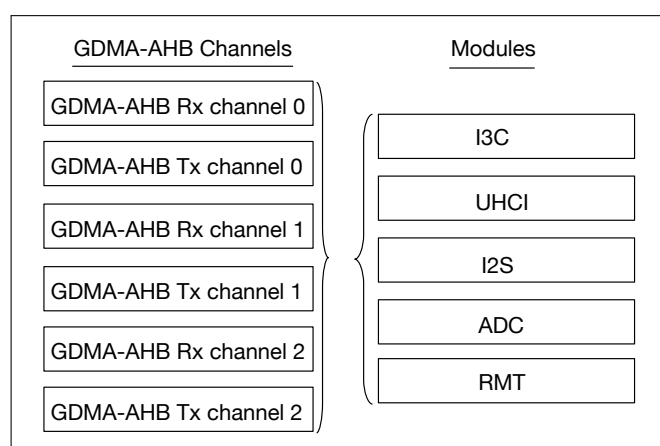


Figure 4.1-1. Modules that Share GDMA-AHB Channels

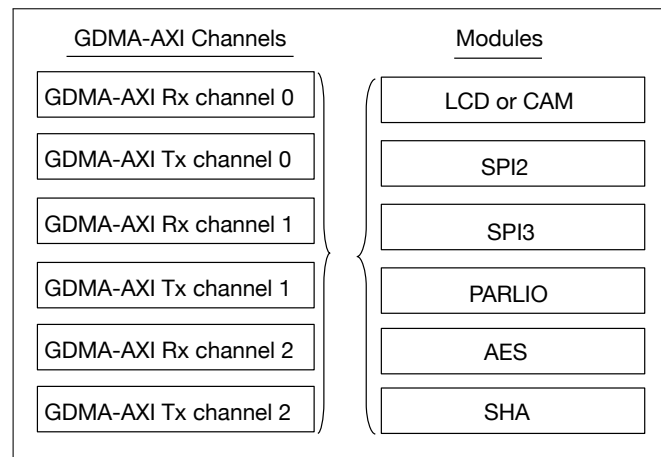


Figure 4.1-2. Modules that Share GDMA-AXI Channels

4.2 Features

GDMA-AHB and GDMA-AXI have the following features:

- Architecture:
 - GDMA-AHB: AHB bus architecture
 - GDMA-AXI: AXI bus architecture, which gives the possibility to complete up to 8 transactions out of order and up to 8 outstanding transactions
- Programmable length of data to be transferred in bytes
- Access via any address and size
- Alignment:
 - GDMA-AHB:
 - * Descriptor address: 1-word aligned
 - * Data address and length:
 - Internal memory and non-encrypted external memory address space: no requirements
 - Encrypted external memory address space: 16-byte aligned
 - GDMA-AXI:
 - * Descriptor address: 2-word aligned
 - * Data address and length:
 - Internal memory and non-encrypted external memory address space: no requirements
 - Encrypted external memory address space: 16-byte aligned
- Linked list of descriptors
- When accessing memory, GDMA-AHB supports INCR4, INCR8, or INCR16 burst transfer, and GDMA-AXI supports INCR burst transfer
- Three transmit channels and three receive channels for each controller

- Software-configurable selection of peripheral requesting its service
- Configurable channel priority and weight arbitration
- Support for memory transfer
- CRC calculation of data
- Interrupt response mechanism for GDMA-AXI linked list switching

4.3 Architecture

In ESP32-P4, all modules that need high-speed data transfer support GDMA. The GDMA controller and CPU data bus have access to the same address space in memory. Figure 4.3-1 shows the basic architecture of the GDMA controller.

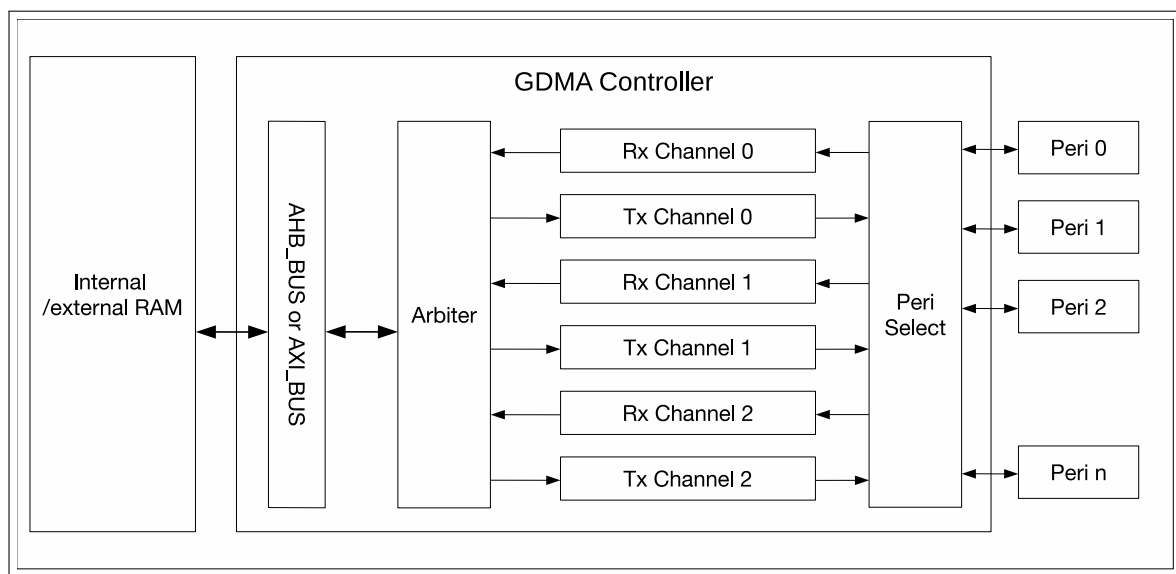


Figure 4.3-1. GDMA controller Architecture

GDMA-AHB and GDMA-AXI have six independent channels respectively, i.e., three transmit channels and three receive channels. Every channel can be connected to different peripherals. In other words, channels are general-purpose, and shared by peripherals.

GDMA-AHB and GDMA-AXI read data from or write data to internal and external memory via AHB_BUS or AXI_BUS respectively. Before this, GDMA uses configurable arbitration schemes for channels requesting read or write access. For the available address range of internal and external memory, please see Chapter 7 [System and Memory](#).

Software can use the GDMA through linked lists. GDMA-AHB linked lists are stored in internal memory, while GDMA-AXI linked lists are stored in internal or external memory. These linked lists consist of `outlinkn` and `inlinkn`, where `n` indicates the channel number (ranging from 0 to 2). The GDMA reads an `outlinkn` (i.e., a linked list of transmit descriptors) from memory and transmit data in corresponding memory according to the `outlinkn`, or read an `inlinkn` (i.e., a linked list of receive descriptors) and store received data into specific address space in memory according to the `inlinkn`.

4.4 Functional Description

4.4.1 Linked List

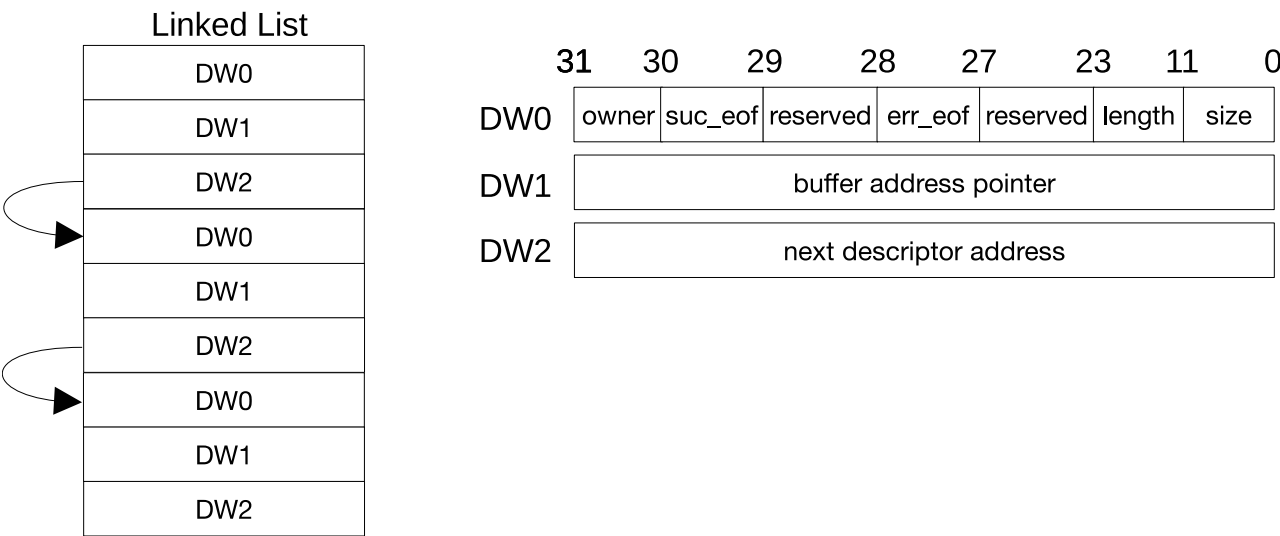


Figure 4.4-1. Structure of a Linked List

Figure 4.4-1 shows the structure of a linked list. An outlink and an inlink have the same structure. A linked list is formed by one or more descriptors, and each descriptor consists of three words. Linked lists should be stored in the memory for the GDMA to be able to use them. The meanings of a descriptor's fields are as follows:

- owner (DW0) [31]: Specifies who is allowed to access the buffer that this descriptor points to.
 - 0: CPU can access the buffer.
 - 1: The GDMA controller can access the buffer.When the GDMA controller stops using the buffer, this bit in a receive descriptor is automatically cleared by hardware, while this bit in a transmit descriptor can only be automatically cleared by hardware if [AHB/AXI_DMA_OUT_AUTO_WRBACK_CHn](#) is set to 1. Software can disable automatic clearing by hardware by setting the [AHB/AXI_DMA_OUT_LOOP_TEST_CHn](#) or [AHB/AXI_DMA_IN_LOOP_TEST_CHn](#) bit. When software loads a linked list, this bit should be set to 1.
 - suc_eof (DW0) [30]: Specifies whether the [AHB/AXI_DMA_IN_SUC_EOF_CHn_INT](#) or [AHB/AXI_DMA_OUT_EOF_CHn_INT](#) interrupt will be triggered when the data corresponding to this descriptor has been received or transmitted.
 - 0: No interrupt will be triggered after the current descriptor's successful transfer;
 - 1: An interrupt will be triggered after the current descriptor's successful transfer.For receive descriptors, software needs to clear this bit to 0, and hardware will set it to 1 after receiving data containing the EOF flag.
- For transmit descriptors, software needs to set this bit to 1 as needed.
- If software configures this bit to 1 in a descriptor, the GDMA will include the EOF flag in the data sent to the corresponding peripheral, indicating to the peripheral that this data segment marks the end of one

transfer phase.

- reserved (DWO) [29]: Reserved. The value of this bit does not matter.
- err_eof (DWO) [28]: Specifies whether the received data has errors.
0: The received data does not have errors.
1: The received data has errors.
This bit is used only when UHCI or PARLIO uses the GDMA to receive data. When an error is detected in the received data segment corresponding to a descriptor, this bit in the receive descriptor is set to 1 by hardware.
- reserved (DWO) [27:24]: Reserved.
- length (DWO) [23:12]: Specifies the number of valid bytes in the buffer that this descriptor points to. This field in a transmit descriptor is written by software and indicates how many bytes can be read from the buffer; this field in a receive descriptor is written by hardware automatically and indicates how many valid bytes have been stored in the buffer.
- size (DWO) [11:0]: Specifies the size of the buffer that this descriptor points to.
- buffer address pointer (DW1): Address of the buffer.
- next descriptor address (DW2): Address of the next descriptor. If the current descriptor is the last one, this value is 0. For GDMA-AHB, this field can only point to the internal memory space; for GDMA-AXI, this field can point to the internal or external memory space.

Note:

Please note that addresses of GDMA-AHB descriptors should be 4-byte aligned, and addresses of GDMA-AXI descriptors should be 8-byte aligned.

If the length of data received is smaller than the size of the buffer, the GDMA controller will not use the available space of the buffer in the next transaction.

4.4.2 Peripheral-to-Memory and Memory-to-Peripheral Data Transfer

The GDMA controller can transfer data from memory to peripheral (transmit) and from peripheral to memory (receive). A transmit channel transfers data in the specified memory location to a peripheral's transmitter via an outlink_{*n*}, whereas a receive channel transfers data received by a peripheral to the specified memory location via an inlink_{*n*}.

Every transmit and receive channel can be connected to any peripheral with the GDMA feature. Table 4.4-1 and Table 4.4-2 illustrate how to select the peripheral to be connected via registers. “Dummy-*n*” corresponds to register values for memory-to-memory data transfer. When a channel is connected to a peripheral, the rest of the channels cannot be connected to that peripheral.

Table 4.4-1. GDMA-AHB Selecting Peripherals via Register Configuration

AHB_DMA_PERI_IN_SEL_CH _{<i>n</i>} AHB_DMA_PERI_OUT_SEL_CH _{<i>n</i>}	Peripheral
0	I3C
1	Dummy-1

2	UHCI
3	I2S0
4	I2S1
5	I2S2
6 ~ 7	Dummy-6 ~ 7
8	ADC
9	Dummy-9
10	RMT
11 ~ 15	Dummy-11 ~ 15
16 ~ 63	Invalid

Table 4.4-2. GDMA-AXI Selecting Peripherals via Register Configuration

AXI_DMA_PERI_IN_SEL_CH n AXI_DMA_PERI_OUT_SEL_CH n	Peripheral
0	LCD or CAM
1	SPI2
2	SPI3
3	PARLIO
4	AES
5	SHA
6 ~ 15	Dummy-6 ~ 15
16 ~ 63	Invalid

4.4.3 Memory-to-Memory Data Transfer

The GDMA controller also allows memory-to-memory data transfer. Such data transfer can be enabled by setting `AHB/AXI_DMA_MEM_TRANS_EN_CH $n$` , which connects the output of transmit channel n to the input of receive channel n . Note that a transmit channel is only connected to the receive channel with the same number (n), and `AHB/AXI_DMA_PERI_IN_SEL_CH $n$` and `AHB/AXI_DMA_PERI_OUT_SEL_CH $n$` should be configured to the same value corresponding to “Dummy”.

GDMA-AHB only supports memory-to-memory data transfer on channels 1 and 2, and the `AXI_DMA_PERI_IN_SEL_CH $n$` and `AXI_DMA_PERI_OUT_SEL_CH $n$` ($n = 1 \sim 2$) fields need to be configured to fixed values. When using channel 1, configure both fields to 11; when using channel 2, configure both fields to 12.

4.4.4 Enabling GDMA

The software uses the GDMA controller through linked lists. When the GDMA controller receives data, software loads an inlink, configures the `AHB/AXI_DMA_INLINK_ADDR_CH $n$` field with the address of the first receive descriptor, and sets the `AHB/AXI_DMA_INLINK_START_CH $n$` bit to enable GDMA. When the GDMA controller transmits data, software loads an outlink, prepares data to be transmitted, configures the `AHB/AXI_DMA_OUTLINK_ADDR_CH $n$` field with address of the first transmit descriptor, and sets the `AHB/AXI_DMA_OUTLINK_START_CH $n$` bit to enable GDMA. The `AHB/AXI_DMA_INLINK_START_CH $n$` bit and `AHB/AXI_DMA_OUTLINK_START_CH $n$` bit are cleared automatically by hardware.

In some cases, you may want to append more descriptors to a DMA transfer that is already started. Naively, it would seem to be possible to do this by clearing the EOF bit of the final descriptor in the existing list and setting its next descriptor address pointer field (DW2) to the first descriptor of the to-be-added list. However, this strategy fails if the existing DMA transfer is almost or entirely finished. Instead, the GDMA controller has specialized logic to make sure a DMA transfer can be continued or restarted: if the transfer is ongoing, the controller will make sure to take the appended descriptors into account; if the transfer has already finished, the controller will restart with the new descriptors. This is implemented by the Restart function.

When using the Restart function, software needs to rewrite the address of the first descriptor in the new list to DW2 of the last descriptor in the loaded list, and set [AHB/AXI_DMA_INLINK_RESTART_CHn](#) bit or [AHB/AXI_DMA_OUTLINK_RESTART_CHn](#) bit (these two bits are cleared automatically by hardware). As shown in Figure 4.4-2, by doing so hardware can obtain the address of the first descriptor in the new list when reading the last descriptor in the loaded list, and then read the new list.

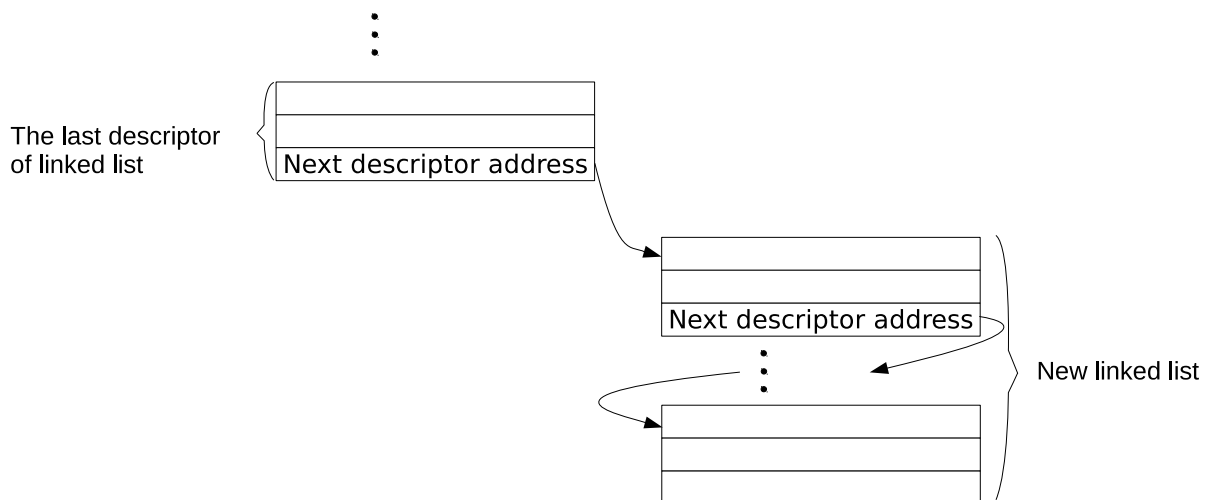


Figure 4.4-2. Relationship among Linked Lists

4.4.5 Linked List Reading Process

Once configured and enabled by software, the GDMA controller starts to read the linked list from memory. The GDMA performs checks on descriptors in the linked list. Only if descriptors pass the checks, the corresponding GDMA channel will start data transfer. If the descriptors fail any of the checks, hardware will trigger descriptor error interrupt (either [AHB/AXI_DMA_IN_DSCR_ERR_CHn_INT](#) or [AHB/AXI_DMA_OUT_DSCR_ERR_CHn_INT](#)), and the channel will halt.

The checks performed on descriptors are:

- Owner bit check when [AHB/AXI_DMA_IN_CHECK_OWNER_CHn](#) or [AHB/AXI_DMA_OUT_CHECK_OWNER_CHn](#) is set to 1. If the owner bit is 0, the buffer is accessed by the CPU. In this case, the owner bit fails the check. The owner bit will not be checked if [AHB/AXI_DMA_IN_CHECK_OWNER_CHn](#) or [AHB/AXI_DMA_OUT_CHECK_OWNER_CHn](#) is 0.
- Descriptor address check, which checks if the descriptor address is located in configured memory space. If a GDMA-AHB descriptor points to [AHB_DMA_ACCESS_INTR_MEM_START_ADDR](#) ~ [AHB_DMA_ACCESS_INTR_MEM_END_ADDR](#), it passes the check. If a GDMA-AXI descriptor points to [AXI_DMA_ACCESS_INTR_MEM_START_ADDR](#) ~ [AXI_DMA_ACCESS_INTR_MEM_END_ADDR](#), or [AXI_DMA_ACCESS_EXTR_MEM_START_ADDR](#) ~ [AXI_DMA_ACCESS_EXTR_MEM_END_ADDR](#), it passes

the check. For details, please refer to Section 4.4.8.

After the software detects a descriptor error interrupt, it must reset the corresponding channel, and enable GDMA by setting [AHB/AXI_DMA_OUTLINK_START_CHn](#) or [AHB/AXI_DMA_INLINK_START_CHn](#) bit.

4.4.6 Interrupt Response Mechanism for GDMA-AXI Linked List Switching

GDMA-AXI supports generating an interrupt when a linked list switching is completed. Software can set the [AXI_DMA_LINK_SWITCH_STATE_CHn](#) bit to 1 to enable the linked list switching. Once the linked list switching is completed, if the [AXI_DMA_OUT_LINK_SWITCH_CHn_INT](#) interrupt is enabled, GDMA-AXI will generate this interrupt, and clear the [AXI_DMA_LINK_SWITCH_STATE_CHn](#) bit, indicating that the linked list has been switched.

This feature serves only as a status indication and does not affect normal GDMA-AXI data transfers.

4.4.7 EOF

Note: In this chapter, EOF of transmit descriptors refers to `suc_eof` (i.e., bit 30 of DWO), while EOF of receive descriptors refers to both `suc_eof` and `err_eof` (i.e., bit 28 of DWO).

The GDMA controller uses EOF (end of frame) flags to indicate the end of data segment transfer corresponding to a specific descriptor.

For data transmission, the GDMA generates two types of EOF interrupts:

- [AHB/AXI_DMA_OUT_EOF_CHn_INT](#), generated when the `suc_eof` bit of any descriptor in the linked list is set, and the data corresponding to this descriptor has been transmitted. This interrupt is enabled by setting the [AHB/AXI_DMA_OUT_EOF_CHn_INT_ENA](#) bit.
- [AHB/AXI_DMA_OUT_TOTAL_EOF_CHn_INT](#), generated when the `suc_eof` bit of the last descriptor in the linked list is set, and the data corresponding to the last descriptor has been transmitted. This interrupt is enabled by setting the [AHB/AXI_DMA_OUT_TOTAL_EOF_CHn_INT_ENA](#) bit.

For data reception, the GDMA also generates two types of EOF interrupts:

- [AHB/AXI_DMA_IN_SUC_EOF_CHn_INT](#), generated when a data segment with an EOF flag has been received. This interrupt is enabled by setting the [AHB/AXI_DMA_IN_SUC_EOF_CHn_INT_ENA](#) bit.
- (UHCI and PARLIO only) [AHB/AXI_DMA_IN_ERR_EOF_CHn_EOF_INT](#), generated when a data segment corresponding to a descriptor has been received with errors. This interrupt is enabled by setting the [AHB/AXI_DMA_IN_ERR_EOF_CHn_INT_ENA](#) bit, and is valid only when the channel is connected to UHCI or PARLIO.

When detecting an [AHB/AXI_DMA_OUT_TOTAL_EOF_CHn_INT](#) or [AHB/AXI_DMA_IN_SUC_EOF_CHn_INT](#) interrupt, software can read the value of the [AHB/AXI_DMA_OUT_EOF_DES_ADDR_CHn](#) or

[AHB/AXI_DMA_IN_SUC_EOF_DES_ADDR_CHn](#) field, which stores the address of the finished descriptor.

Therefore, the software can tell which descriptors have been used and reclaim them as needed. For RX, the address of the descriptor can also be stored to the [AHB/AXI_DMA_IN_ERR_EOF_DES_ADDR_CHn](#) field when an [AHB/AXI_DMA_IN_ERR_EOF_CHn_INT](#) interrupt is triggered.

4.4.8 Accessing Memory

The transmit and receive channels of GDMA-AHB can access the internal and external memory space configured by [AHB_DMA_ACCESS_INTR_MEM_START_ADDR](#) and [AHB_DMA_ACCESS_INTR_MEM_END_ADDR](#).

The transmit and receive channels of GDMA-AXI can access:

- Internal memory space configured by [AXI_DMA_ACCESS_INTR_MEM_START_ADDR](#) and [AXI_DMA_ACCESS_INTR_MEM_END_ADDR](#).
- External memory space configured by [AXI_DMA_ACCESS_EXTR_MEM_START_ADDR](#) and [AXI_DMA_ACCESS_EXTR_MEM_END_ADDR](#).

To improve data transfer efficiency, GDMA can send data in burst mode. For GDMA-AHB, this mode is disabled by default, and can be enabled for receive channels by setting [AHB_DMA_IN_DATA_BURST_EN_CH \$n\$](#) , and enabled for transmit channels by setting [AHB_DMA_OUT_DATA_BURST_EN_CH \$n\$](#) . For GDMA-AXI, this mode is enabled by default, and cannot be disabled. Users can configure the number of data bytes in a single AXI burst (burst length) to 8, 16, 32, 64, or 128 bytes via [AXI_DMA_IN_BURST_SIZE_SEL_CH \$n\$](#) and [AXI_DMA_OUT_BURST_SIZE_SEL_CH \$n\$](#) .

When GDMA-AHB and GDMA-AXI access the configured internal memory and non-encrypted external memory space, there are no requirements for descriptor field alignment; when they access the encrypted external memory space (see Chapter [32 External Memory Encryption and Decryption \(XTS_AES\)](#)), buffer address pointer and data should be 16-byte aligned, and other fields are not required to be aligned.

4.4.9 Arbitration

To ensure timely response to peripherals running at a high speed with low latency (such as SPI), the GDMA controller implements two arbitration schemes, based on channel priority and channel weight respectively.

- Priority arbitration: Each channel can be assigned a priority from 0 ~ 5 (in total 6 levels). The larger the number, the higher the priority. When several channels perform data transfers at the same time, the GDMA would respond to the transfer requests according to priority levels, and the channel with higher priority would get a response more timely.
- Weight arbitration:
 - Each channel is assigned a weight (i.e., the number of tokens) from 0 ~ 15. The GDMA divides the AHB or AXI bus clock period into multiple time slots, and in each time slot, the number of transfers performed by a channel is determined by the number of its tokens. Every time a channel performs a data transfer, one of its tokens is spent. If all of its tokens have been spent in a time slot, then this channel's transfer request will no longer be responded to until the time slot expires, or until tokens of all channels have been spent and the new time slot starts.
 - If the number of tokens assigned to a channel is not zero, the GDMA waits for this channel's tokens to be spent even if it does not have data transfer requests, and will not exit from this time slot ahead of expiration. This leads to a waste of bandwidth. Therefore, the GDMA provides weight arbitration optimization. When this feature is enabled, a channel without data transfer requests will not be involved in the arbitration and its tokens will be ignored. When all channels no longer have transfer

requests, the GDMA arbiter exits from the current time slot before expiration. This way, the arbitration response is accelerated and the transfer efficiency is improved.

Note:

- If channels have the same weight but different priorities, on condition that the number of bytes to be transferred is the same and not large, channels with higher priorities would finish data transfers first.
- If channels have different priorities and weights, on condition that the number of bytes to be transferred is the same, channels with higher weights would be allocated more bandwidth and finish data transfers first.

4.4.10 Bus Error Response Information Logging

During GDMA-AHB data transfers, if a slave device on the bus returns an error response, the data transfer will not be interrupted; GDMA-AHB will continue to transfer data according to the existing linked list. As a result, the actual data received after the transfer may not match the expected data. To facilitate troubleshooting, if an error response is received during GDMA-AHB data transfer, the following internal registers can be used to store relevant transfer information:

- [AHB_DMA_AHBINF_RESP_ERR_ADDR](#): Records the 32-bit address corresponding to the current bus transfer.
- [AHB_DMA_AHBINF_RESP_ERR_CH_ID](#): Records the GDMA-AHB channel used for the current bus transfer.
- [AHB_DMA_AHBINF_RESP_ERR_ID](#): Records the transfer ID corresponding to the current the peripheral-to-memory, memory-to-peripheral, or memory-to-memory transfer on the bus.
- [AHB_DMA_AHBINF_RESP_ERR_WR](#): Records the type of transfer for the current bus transfer.

When GDMA-AHB initiates an AHB bus transfer as the master and receives an error response, it will also generate an error response interrupt to notify the CPU. After the interrupt is generated, GDMA-AHB will not interrupt the data transfer. In addition, GDMA-AHB does not latch data into registers for the first AHB bus transfer that triggers an error response. If GDMA-AHB receives multiple consecutive error responses, the error response logging registers always reflect the most recent error; only the last error response prior to reading the registers is retained.

4.4.11 Automatic Clock Gating

When GDMA-AHB is operating, not all channels are needed in certain scenarios; in this case, the clocks for unused channels can be disabled to reduce power consumption. Similarly, the internal arbitration and bus interface modules only need to operate during data transfer, and their clocks can also be disabled during idle periods to save power. Currently, GDMA-AHB has the following internal modules:

- Bus interface processing module
- Arbitration module
- RX data processing module
- RX descriptor processing module
- TX data processing module

- TX descriptor processing module
- Register configuration synchronization module

Clocks of above modules are automatically disabled when idle to further minimize power consumption. These clocks can also be forced to enable by setting corresponding register bits. For example, setting [AHB_DMA_AHBINF_CLK_EN](#) will keep the bus interface processing module clock always on; the configurations for keeping other module clocks always on are similar.

4.4.12 CRC Calculation

The GDMA has a CRC calculation module for both TX and RX. This module supports arbitrary polynomials and 8-bit parallel computation, namely 8 bits at a time. The CRC result is up to 32-bit wide for GDMA-AHB, and up to 16-bit for GDMA-AXI. It is calculated for each linked list descriptor by descriptor and becomes invalid when the Restart function is triggered during transfer.

For easier understanding of register configuration rules, assume:

- The 8-bit data to calculate each time is data[7:0].
- The required CRC result is 5-bit wide, and the polynomial used is $crc[4 : 0] = x \oplus 5 + x \oplus 2 + 1$;
- The initial CRC value is 0x1F, and the intermediate result calculated in each clock cycle is crc_tmp[4:0].

Taking as an example the GDMA reading data from memory and transmitting the data to peripherals, the following fields are provided by hardware for CRC computation:

- [AHB/AXI_DMA_OUT_CRC_INIT_DATA_CHn](#): Configures the initial CRC value, i.e., 0x1F in the above example. [AHB_DMA_OUT_CRC_INIT_DATA_CHn](#) can be configured with a 32-bit value at the maximum, while [AXI_DMA_OUT_CRC_INIT_DATA_CHn](#) can be configured with a 16-bit value at the maximum, meaning that the CRC result could be 32 bits or 16 bits at most.
- [AHB/AXI_DMA_OUT_CRC_FINAL_RESULT_CHn](#): Stores the computation result of all data in a DMA transfer (until data pointed by the last descriptor in the linked list has been transferred). The computation result could be 32 bits at most.
- [AHB/AXI_DMA_TX_CRC_WIDTH_CHn](#): Configures the CRC result length.
- [AHB/AXI_DMA_TX_CRC_LAUTCH_FLGA_CHn](#), which latches the CRC result.
- [AHB/AXI_DMA_TX_CRC_EN_WR_DATA_CHn](#): Indicates which bits in crc_tmp[4:0] are used to compute crc[x] (x is configured via [AHB/AXI_DMA_TX_CRC_EN_ADDR_CHn](#)). If [AHB/AXI_DMA_TX_CRC_EN_WR_DATA_CHn](#) is configured as 0xF, it means that only crc_tmp[3], crc_tmp[2], crc_tmp[1], and crc_tmp[0] are used for computation, and other bits are not used (in this case crc_tmp[4]).
- [AHB/AXI_DMA_TX_CRC_DATA_EN_WR_DATA_CHn](#): Indicates which bits in data[7:0] are used to compute crc[x] (x is configured via [AHB/AXI_DMA_TX_CRC_DATA_EN_ADDR_CHn](#)). GDMA calculates the CRC in the unit of byte, so only the lower 8 bits of this field needs to be configured. If [AHB/AXI_DMA_TX_CRC_DATA_EN_WR_DATA_CHn](#) is configured as 0xF, it means that only data[3], data[2], data[1], and data[0] are used for computation.

CRC calculation is achieved in the GDMA by configuring a parallel calculation matrix derived from the polynomial. If the expected CRC result is n bits, then the matrix has n rows and n + 8 columns.

Based on the assumptions at the beginning of this section, the calculation matrix would be 5 rows and 13 columns, and the actual hardware CRC calculation formulas would be as follows ($\oplus$ represents the XOR operation):

$$\begin{aligned} crc[0] &= crc_tmp[0] \oplus crc_tmp[2] \oplus crc_tmp[3] \oplus data[0] \oplus data[3] \oplus data[5] \oplus data[6] \\ crc[1] &= crc_tmp[1] \oplus crc_tmp[3] \oplus crc_tmp[4] \oplus data[1] \oplus data[4] \oplus data[6] \oplus data[7] \\ crc[2] &= crc_tmp[0] \oplus crc_tmp[3] \oplus crc_tmp[4] \oplus data[0] \oplus data[2] \oplus data[3] \oplus data[6] \oplus data[7] \\ crc[3] &= crc_tmp[0] \oplus crc_tmp[1] \oplus crc_tmp[4] \oplus data[1] \oplus data[4] \oplus data[3] \oplus data[7] \\ crc[4] &= crc_tmp[1] \oplus crc_tmp[2] \oplus data[2] \oplus data[4] \oplus data[5] \end{aligned}$$

The above calculation formula can be abstracted as a switch matrix shown in Figure 4.4-3:

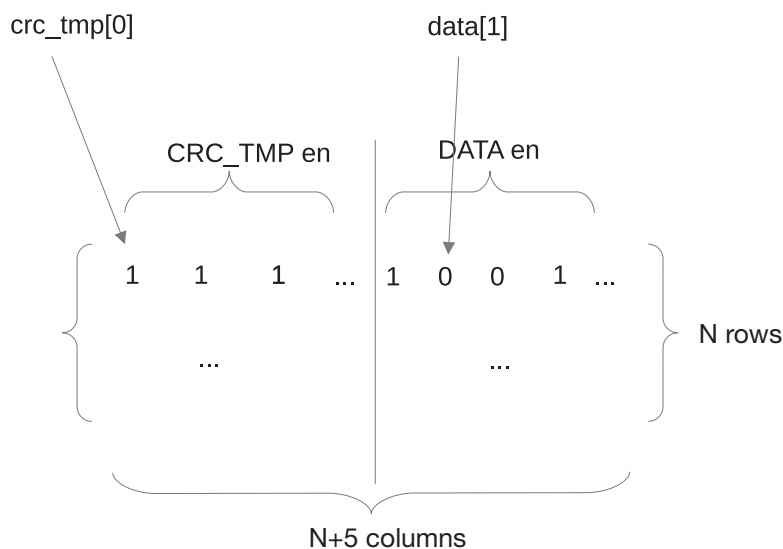


Figure 4.4-3. CRC Calculation Matrix

Note:

The CRC calculation described above is a Linear Feedback Shift Register (LFSR) implementation of the bitwise CRC algorithm. The formulas can be generated with some online tools according to the polynomial and the data width for parallel computation (always 8 bits for ESP32-P4).

4.5 Event Task Matrix Feature

The GDMA controller on ESP32-P4 supports the Event Task Matrix (ETM) function, which allows GDMA's ETM tasks to be triggered by any peripherals' ETM events, or GDMA's ETM events to trigger any peripherals' ETM tasks. This section introduces the ETM tasks and events related to GDMA. For more information, please refer to Chapter 13 [Event Task Matrix \(ETM\)](#).

GDMA can receive the following ETM tasks:

- GDMA_AHB/AXI_TASK_IN_START_CH n : Enables the corresponding RX channel n for data transfer.
- GDMA_AHB/AXI_TASK_OUT_START_CH n : Enables the corresponding TX channel n for data transfer.

Note:

Above ETM tasks can achieve the same functions as CPU configuring [AHB/AXI_DMA_INLINK_START_CH \$n\$](#) and [AHB/AXI_DMA_OUTLINK_START_CH \$n\$](#) . When [AHB/AXI_DMA_IN_ETM_EN_CH \$n\$](#) or [AHB/AXI_DMA_OUT_ETM_EN_CH \$n\$](#) is 1, only ETM tasks can be used to configure the transfer direction and enable the corresponding GDMA channel. When [AHB/AXI_DMA_IN_ETM_EN_CH \$n\$](#) or [AHB_DMA_OUT_ETM_EN_CH \$n\$](#) is 0, only CPU can be used to enable the corresponding GDMA channel.

GDMA can generate the following ETM events:

- [GDMA_AHB/AXI_EVT_IN_DONE_CH \$n\$](#) : Indicates that the data has been received according to the receive descriptor via channel n .
- [GDMA_AHB/AXI_EVT_IN_SUC_EOF_CH \$n\$](#) : Indicates that the data corresponding to a receive descriptor has been received via channel n and the EOF bit of this descriptor is 1.
- [GDMA_AHB/AXI_EVT_IN_FIFO_EMPTY_CH \$n\$](#) : Indicates that the RX FIFO has become empty.
- [GDMA_AHB/AXI_EVT_IN_FIFO_FULL_CH \$n\$](#) : Indicates that the RX FIFO has become full.
- [GDMA_AHB/AXI_EVT_OUT_DONE_CH \$n\$](#) : Indicates that the data has been transmitted according to the transmit descriptor via channel n .
- [GDMA_AHB/AXI_EVT_OUT_SUC_EOF_CH \$n\$](#) : Indicates that the data corresponding to a transmit descriptor has been transmitted or received via channel n and the EOF bit of this descriptor is 1.
- [GDMA_AHB/AXI_EVT_OUT_TOTAL_EOF_CH \$n\$](#) : Indicates that the data corresponding to the last transmit descriptors has been sent via transmit channel n and the EOF bit of this descriptor is 1.
- [GDMA_AHB/AXI_EVT_OUT_FIFO_EMPTY_CH \$n\$](#) : Indicates that the TX FIFO has become empty.
- [GDMA_AHB/AXI_EVT_OUT_FIFO_FULL_CH \$n\$](#) : Indicates that the TX FIFO has become full.

In practical applications, GDMA's ETM events can trigger its own ETM tasks. For example, the [GDMA_AHB/AXI_EVT_OUT_TOTAL_EOF_CH0](#) event can trigger the [GDMA_AHB/AXI_TASK_IN_START_CH1](#) task, and in this way trigger a new round of GDMA operations.

4.6 Interrupts

ESP32-P4's GDMA module can generate the following interrupt signals that will be sent to the [Interrupt Matrix](#).

- [AHB/AXI_PDMA_IN_CHO_INTR](#)
- [AHB/AXI_PDMA_IN_CH1_INTR](#)
- [AHB/AXI_PDMA_IN_CH2_INTR](#)
- [AHB/AXI_PDMA_OUT_CHO_INTR](#)
- [AHB/AXI_PDMA_OUT_CH1_INTR](#)
- [AHB/AXI_PDMA_OUT_CH2_INTR](#)

There are several internal interrupt sources from GDMA that can generate the above interrupt signals.

The [AHB/AXI_PDMA_IN_CH \$n\$ _INTR](#) (n is 0 ~ 2) signals are generated by the following interrupt sources:

- AHB/AXI_DMA_IN_DONE_CH n _INT: Triggered when all data corresponding to a receive descriptor has been received via receive channel n .
- AHB/AXI_DMA_IN_SUC_EOF_CH n _INT: Triggered when the suc_eof bit in a receive descriptor is 1 and the data corresponding to this receive descriptor has been received via receive channel n .
- AHB/AXI_DMA_IN_ERR_EOF_CH n _INT: Triggered when an error is detected in the data segment corresponding to a descriptor received via receive channel n . This interrupt is used only for UHCI (UART0 or UART1) or PARLIO.
- AHB/AXI_DMA_IN_DSCR_EMPTY_CH n _INT: Triggered when the size of the buffer pointed by receive descriptors is smaller than the length of data to be received via receive channel n .
- AHB/AXI_DMA_IN_DSCR_ERR_CH n _INT: Triggered when a receive descriptor on receive channel n fails any of the two descriptor checks.
- AHB_DMA_INFIFO_OVF_CH n _INT or AXI_DMA_INFIFO_L1/L2/L3_OVF_CH n _INT: Triggered when the RX FIFO of GDMA-AHB or the L1/L2/L3 RX FIFO of GDMA-AXI overflows.
- AHB_DMA_INFIFO_UDF_CH n _INT or AXI_DMA_INFIFO_L1/L2/L3_UDF_CH n _INT: Triggered when the RX FIFO of GDMA-AHB or the L1/L2/L3 RX FIFO of GDMA-AXI underflows.
- AHB_DMA_IN_AHBNF_RESP_ERR_CH n _INT: Triggered when GDMA-AHB receive channel n performs a data transfer on the AHB bus and an error response is received during the transfer.

The AHB/AXI_PDMA_OUT_CH n _INTR (n is 0 ~ 2) signals are generated by the following interrupt sources:

- AHB/AXI_DMA_OUT_DONE_CH n _INT: Triggered when all data corresponding to a transmit descriptor has been sent via transmit channel n .
- AHB/AXI_DMA_OUT_EOF_CH n _INT: Triggered when the suc_eof bit in a transmit descriptor is 1 and data corresponding to this descriptor has been sent via transmit channel n . If [AHB/AXI_DMA_OUT_EOF_MODE_CH \$n\$](#) is 0, this interrupt will be triggered when the last byte of data corresponding to this descriptor enters GDMA's transmit channel; if [AHB/AXI_DMA_OUT_EOF_MODE_CH \$n\$](#) is 1, this interrupt is triggered when the last byte of data is taken from GDMA's transmit channel.
- AHB/AXI_DMA_OUT_DSCR_ERR_CH n _INT: Triggered when a transmit descriptor on transmit channel n fails any of the two descriptor checks.
- AHB/AXI_DMA_OUT_TOTAL_EOF_CH n _INT: Triggered when all data corresponding to a linked list (including multiple descriptors) has been sent via transmit channel n .
- AHB_DMA_OUTFIFO_OVF_CH n _INT or AXI_DMA_OUTFIFO_L1/L2/L3_OVF_CH n _INT: Triggered when the TX FIFO of GDMA-AHB or the L1/L2/L3 TX FIFO of GDMA-AXI overflows.
- AHB_DMA_OUTFIFO_UDF_CH n _INT or AXI_DMA_OUTFIFO_L1/L2/L3_UDF_CH n _INT: Triggered when the TX FIFO of GDMA-AHB or the L1/L2/L3 TX FIFO of GDMA-AXI underflows.
- AXI_DMA_OUT_LINK_SWITCH_CH n _INT: Triggered when GDMA-AXI has switched to a new linked list on transmit channel n .
- AHB_DMA_OUT_AHBNF_RESP_ERR_CH n _INT: Triggered when GDMA-AHB transmit channel n performs a data transfer on the AHB bus and an error response is received during the transfer.

4.7 Programming Procedures

The clock gating for GDMA can be configured via [HP_SYS_CLKRST_AHB/AXI_PDMA_SYS_CLK_EN](#), and is enabled by default. GDMA can be reset by configuring [HP_SYS_CLKRST_RST_EN_AHB/AXI_PDMA](#).

4.7.1 Programming Procedures for GDMA's Transmit Channel

To transmit data, GDMA's transmit channel should be configured by software as follows:

1. Set [AHB/AXI_DMA_OUT_RST_CH \$n\$](#) first to 1 and then to 0, to reset the state machine of GDMA's transmit channel and FIFO pointer.
2. Load an outlink, and configure [AHB/AXI_DMA_OUTLINK_ADDR_CH \$n\$](#) with address of the first transmit descriptor.
3. Configure [AHB/AXI_DMA_PERI_OUT_SEL_CH \$n\$](#) with the value corresponding to the peripheral to be connected, as shown in Table 4.4-1 and Table 4.4-2.
4. Set [AHB/AXI_DMA_OUTLINK_START_CH \$n\$](#) to enable GDMA's transmit channel for data transfer.
5. Configure and enable the corresponding peripheral (SPI2, UHCI (UART0 or UART1), I2S, AES, SHA, and ADC). See details in individual chapters of these peripherals.
6. Wait for the [AHB/AXI_DMA_OUT_TOTAL_EOF_CH \$n\$ _INT](#) interrupt, which indicates the completion of data transfer.

4.7.2 Programming Procedures for GDMA's Receive Channel

To receive data, GDMA's receive channel should be configured by software as follows:

1. Set [AHB/AXI_DMA_IN_RST_CH \$n\$](#) first to 1 and then to 0, to reset the state machine of GDMA's receive channel and FIFO pointer.
2. Load an inlink, and configure [AHB/AXI_DMA_INLINK_ADDR_CH \$n\$](#) with address of the first receive descriptor.
3. Configure [AHB/AXI_DMA_PERI_IN_SEL_CH \$n\$](#) with the value corresponding to the peripheral to be connected, as shown in Table 4.4-1 and Table 4.4-2.
4. Set [AHB/AXI_DMA_INLINK_START_CH \$n\$](#) to enable GDMA's receive channel for data transfer.
5. Configure and enable the corresponding peripheral (SPI2, UHCI (UART0 or UART1), I2S, AES, SHA, and ADC). See details in individual chapters of these peripherals.

4.7.3 Programming Procedures for Memory-to-Memory Transfer

To transfer data from one memory location to another, GDMA should be configured by software as follows:

1. Set [AHB/AXI_DMA_OUT_RST_CH \$n\$](#) first to 1 and then to 0, to reset the state machine of GDMA's transmit channel and FIFO pointer.
2. Set [AHB/AXI_DMA_IN_RST_CH \$n\$](#) first to 1 and then to 0, to reset the state machine of GDMA's receive channel and FIFO pointer.

3. Load an outlink, and configure [AHB/AXI_DMA_OUTLINK_ADDR_CH \$n\$](#) with address of the first transmit descriptor.
4. Load an inlink, and configure [AHB/AXI_DMA_INLINK_ADDR_CH \$n\$](#) with address of the first receive descriptor.
5. Set [AHB/AXI_DMA_MEM_TRANS_EN_CH \$n\$](#) to enable memory-to-memory transfer.
6. Set [AHB/AXI_DMA_OUTLINK_START_CH \$n\$](#) to enable GDMA's transmit channel for data transfer.
7. Set [AHB/AXI_DMA_INLINK_START_CH \$n\$](#) to enable GDMA's receive channel for data transfer.
8. If the suc_eof bit is set in a transmit descriptor, an [AHB/AXI_DMA_IN_SUC_EOF_CH \$n\$ _INT](#) interrupt will be triggered when the data segment corresponding to this descriptor has been transmitted.

4.7.4 Programming Procedures for Channel Priority and Weight

The priority arbitration can be configured as follows:

1. Configure the channel priority for TX and RX respectively via [AHB/AXI_DMA_TX_PRI_CH \$n\$](#) and [AHB/AXI_DMA_RX_PRI_CH \$n\$](#) .

The weight arbitration can be configured as follows:

1. Configure the time slot
 - GDMA-AHB: TX and RX share the same time slot configured by [AHB/AHB_DMA_ARB_TIMEOUT](#).
 - GDMA-AXI: TX and RX use independent time slots configured by [AXI_DMA_ARB_TIMEOUT_TX](#) and [AXI_DMA_ARB_TIMEOUT_RX](#).
2. Configure the number of tokens for TX and RX respectively via [AHB/AXI_DMA_TX_CH_ARB_WEIGH_CH \$n\$](#) and [AHB/AXI_DMA_RX_CH_ARB_WEIGH_CH \$n\$](#) .
3. Enable weight arbitration optimization for TX and RX respectively by clearing [AHB/AXI_DMA_TX_ARB_WEIGH_OPT_DIR_CH \$n\$](#) and [AHB/AXI_DMA_RX_ARB_WEIGH_OPT_DIR_CH \$n\$](#) .
4. Enable the arbitration:
 - GDMA-AHB: Enable the arbitration for both TX and RX by setting [AHB_DMA_WEIGHT_EN](#).
 - GDMA-AXI: Enable the arbitration for TX and RX respectively by setting [AXI_DMA_WEIGHT_EN_TX](#) and [AXI_DMA_WEIGHT_EN_RX](#).

4.7.5 Programming Procedures for CRC Calculation

The CRC calculation registers can be configured as follows:

1. Configure which bits in `crc_tmp` and data are used in the matrix via [AHB/AXI_DMA_TX_CRC_EN_ADDR_CH \$n\$](#) , [AHB/AXI_DMA_TX_CRC_DATA_EN_ADDR_CH \$n\$](#) , [AHB/AXI_DMA_TX_CRC_EN_WR_DATA_CH \$n\$](#) , and [AHB/AXI_DMA_TX_CRC_DATA_EN_WR_DATA_CH \$n\$](#) .
2. Latch the values of the above four fields by first writing 1 and then 0 to [AHB/AXI_DMA_TX_CRC_LAUTCH_FLGA_CH \$n\$](#) . Then the configuration for a row is complete.
3. Repeat Step 1 and Step 2 until all rows in the matrix are set up, and then start DMA transfer.

4.8 Register Summary

4.8.1 GDMA-AHB Register Summary

The addresses in this section are relative to GDMA-AHB base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
Interrupt Registers			
AHB_DMA_IN_INT_RAW_CHO_REG	RX channel 0 raw interrupt status register	0x0000	R/WTC/SS
AHB_DMA_IN_INT_ST_CHO_REG	RX channel 0 masked interrupt status register	0x0004	RO
AHB_DMA_IN_INT_ENA_CHO_REG	RX channel 0 interrupt enable register	0x0008	R/W
AHB_DMA_IN_INT_CLR_CHO_REG	RX channel 0 interrupt clear register	0x000C	WT
AHB_DMA_IN_INT_RAW_CH1_REG	RX channel 1 raw interrupt status register	0x0010	R/WTC/SS
AHB_DMA_IN_INT_ST_CH1_REG	RX channel 1 masked interrupt status register	0x0014	RO
AHB_DMA_IN_INT_ENA_CH1_REG	RX channel 1 interrupt enable register	0x0018	R/W
AHB_DMA_IN_INT_CLR_CH1_REG	RX channel 1 interrupt clear register	0x001C	WT
AHB_DMA_IN_INT_RAW_CH2_REG	RX channel 2 raw interrupt status register	0x0020	R/WTC/SS
AHB_DMA_IN_INT_ST_CH2_REG	RX channel 2 masked interrupt status register	0x0024	RO
AHB_DMA_IN_INT_ENA_CH2_REG	RX channel 2 interrupt enable register	0x0028	R/W
AHB_DMA_IN_INT_CLR_CH2_REG	RX channel 2 interrupt clear register	0x002C	WT
AHB_DMA_OUT_INT_RAW_CHO_REG	TX channel 0 raw interrupt status register	0x0030	R/WTC/SS
AHB_DMA_OUT_INT_ST_CHO_REG	TX channel 0 masked interrupt status register	0x0034	RO
AHB_DMA_OUT_INT_ENA_CHO_REG	TX channel 0 interrupt enable register	0x0038	R/W
AHB_DMA_OUT_INT_CLR_CHO_REG	TX channel 0 interrupt clear register	0x003C	WT
AHB_DMA_OUT_INT_RAW_CH1_REG	TX channel 1 raw interrupt status register	0x0040	R/WTC/SS
AHB_DMA_OUT_INT_ST_CH1_REG	TX channel 1 masked interrupt status register	0x0044	RO
AHB_DMA_OUT_INT_ENA_CH1_REG	TX channel 1 interrupt enable register	0x0048	R/W
AHB_DMA_OUT_INT_CLR_CH1_REG	TX channel 1 interrupt clear register	0x004C	WT
AHB_DMA_OUT_INT_RAW_CH2_REG	TX channel 2 raw interrupt status register	0x0050	R/WTC/SS

Name	Description	Address	Access
AHB_DMA_OUT_INT_ST_CH2_REG	TX channel 2 masked interrupt status register	0x0054	RO
AHB_DMA_OUT_INT_ENA_CH2_REG	TX channel 2 interrupt enable register	0x0058	R/W
AHB_DMA_OUT_INT_CLR_CH2_REG	TX channel 2 interrupt clear register	0x005C	WT
Debug Registers			
AHB_DMA_AHB_TEST_REG	Reserved	0x0060	R/W
Configuration Registers			
AHB_DMA_MISC_CONF_REG	Miscellaneous register	0x0064	R/W
AHB_DMA_IN_CONFO_CHO_REG	Configuration register 0 of RX channel 0	0x0070	R/W
AHB_DMA_IN_CONF1_CHO_REG	Configuration register 1 of Rx channel 0	0x0074	R/W
AHB_DMA_IN_POP_CHO_REG	Pop control register of RX channel 0	0x007C	varies
AHB_DMA_IN_LINK_CHO_REG	Linked list descriptor configuration and control register of RX channel 0	0x0080	varies
AHB_DMA_OUT_CONFO_CHO_REG	Configuration register 0 of TX channel 0	0x00D0	R/W
AHB_DMA_OUT_CONF1_CHO_REG	Configuration register 1 of TX channel 0	0x00D4	R/W
AHB_DMA_OUT_PUSH_CHO_REG	Push control register of TX channel 0	0x00DC	varies
AHB_DMA_OUT_LINK_CHO_REG	Linked list descriptor configuration and control register of TX channel 0	0x00E0	varies
AHB_DMA_IN_CONFO_CH1_REG	Configuration register 0 of RX channel 1	0x0130	R/W
AHB_DMA_IN_CONF1_CH1_REG	Configuration register 1 of RX channel 1	0x0134	R/W
AHB_DMA_IN_POP_CH1_REG	Pop control register of RX channel 1	0x013C	varies
AHB_DMA_IN_LINK_CH1_REG	Linked list descriptor configuration and control register of RX channel 1	0x0140	varies
AHB_DMA_OUT_CONFO_CH1_REG	Configuration register 0 of TX channel 1	0x0190	R/W
AHB_DMA_OUT_CONF1_CH1_REG	Configuration register 1 of TX channel 1	0x0194	R/W
AHB_DMA_OUT_PUSH_CH1_REG	Push control register of TX channel 1	0x019C	varies
AHB_DMA_OUT_LINK_CH1_REG	Linked list descriptor configuration and control register of TX channel 1	0x01A0	varies
AHB_DMA_IN_CONFO_CH2_REG	Configuration register 0 of RX channel 2	0x01F0	R/W
AHB_DMA_IN_CONF1_CH2_REG	Configuration register 1 of RX channel 2	0x01F4	R/W

Name	Description	Address	Access
AHB_DMA_IN_POP_CH2_REG	Pop control register of RX channel 2	0x01FC	varies
AHB_DMA_IN_LINK_CH2_REG	Linked list descriptor configuration and control register of RX channel 2	0x0200	varies
AHB_DMA_OUT_CONF0_CH2_REG	Configuration register 0 of TX channel 2	0x0250	R/W
AHB_DMA_OUT_CONF1_CH2_REG	Configuration register 1 of TX channel 2	0x0254	R/W
AHB_DMA_OUT_PUSH_CH2_REG	Push control register of TX channel 2	0x025C	varies
AHB_DMA_OUT_LINK_CH2_REG	Linked list descriptor configuration and control register of TX channel 2	0x0260	varies
AHB_DMA_OUT_CRC_INIT_DATA_CHO_REG	TX channel 0 CRC initial value configuration register	0x02BC	R/W
AHB_DMA_TX_CRC_WIDTH_CHO_REG	TX channel 0 CRC result width configuration register	0x02C0	R/W
AHB_DMA_OUT_CRC_CLEAR_CHO_REG	TX channel 0 CRC result clear register	0x02C4	R/W
AHB_DMA_OUT_CRC_FINAL_RESULT_CHO_REG	TX channel 0 CRC result register	0x02C8	RO
AHB_DMA_TX_CRC_EN_WR_DATA_CHO_REG	CRC TX channel 0 CRC intermediate result mask register	0x02CC	R/W
AHB_DMA_TX_CRC_EN_ADDR_CHO_REG	TX channel 0 CRC intermediate result mask target register	0x02D0	R/W
AHB_DMA_TX_CRC_DATA_EN_WR_DATA_CHO_REG	TX channel 0 CRC data input mask register	0x02D4	R/W
AHB_DMA_TX_CRC_DATA_EN_ADDR_CHO_REG	TX channel 0 CRC data input mask target register	0x02D8	R/W
AHB_DMA_TX_CH_ARB_WEIGHT_CHO_REG	TX channel 0 arbitration weight configuration register	0x02DC	R/W
AHB_DMA_TX_ARB_WEIGHT_OPT_DIR_CHO_REG	TX channel 0 weight arbitration optimization enable register	0x02E0	R/W
AHB_DMA_OUT_CRC_INIT_DATA_CH1_REG	TX channel 1 CRC initial value configuration register	0x02E4	R/W
AHB_DMA_TX_CRC_WIDTH_CH1_REG	TX channel 1 CRC result width configuration register	0x02E8	R/W
AHB_DMA_OUT_CRC_CLEAR_CH1_REG	TX channel 1 CRC result clear register	0x02EC	R/W
AHB_DMA_OUT_CRC_FINAL_RESULT_CH1_REG	TX channel 1 CRC result register	0x02F0	RO
AHB_DMA_TX_CRC_EN_WR_DATA_CH1_REG	TX channel 1 CRC intermediate result mask register	0x02F4	R/W
AHB_DMA_TX_CRC_EN_ADDR_CH1_REG	TX channel 1 CRC intermediate result mask target register	0x02F8	R/W
AHB_DMA_TX_CRC_DATA_EN_WR_DATA_CH1_REG	TX channel 1 CRC data input mask register	0x02FC	R/W
AHB_DMA_TX_CRC_DATA_EN_ADDR_CH1_REG	TX channel 1 CRC data input mask target register	0x0300	R/W
AHB_DMA_TX_CH_ARB_WEIGHT_CH1_REG	TX channel 1 arbitration weight configuration register	0x0304	R/W
AHB_DMA_TX_ARB_WEIGHT_OPT_DIR_CH1_REG	TX channel 1 weight arbitration optimization enable register	0x0308	R/W
AHB_DMA_OUT_CRC_INIT_DATA_CH2_REG	TX channel 2 CRC initial value configuration register	0x030C	R/W

Name	Description	Address	Access
AHB_DMA_TX_CRC_WIDTH_CH2_REG	TX channel 2 CRC result width configuration register	0x0310	R/W
AHB_DMA_OUT_CRC_CLEAR_CH2_REG	TX channel 2 CRC result clear register	0x0314	R/W
AHB_DMA_OUT_CRC_FINAL_RESULT_CH2_REG	TX channel 2 CRC result register	0x0318	RO
AHB_DMA_TX_CRC_EN_WR_DATA_CH2_REG	TX channel 2 CRC intermediate result mask register	0x031C	R/W
AHB_DMA_TX_CRC_EN_ADDR_CH2_REG	TX channel 2 CRC intermediate result mask target register	0x0320	R/W
AHB_DMA_TX_CRC_DATA_EN_WR_DATA_CH2_REG	TX channel 2 CRC data input mask register	0x0324	R/W
AHB_DMA_TX_CRC_DATA_EN_ADDR_CH2_REG	TX channel 2 CRC data input mask target register	0x0328	R/W
AHB_DMA_TX_CH_ARB_WEIGHT_CH2_REG	TX channel 2 arbitration weight configuration register	0x032C	R/W
AHB_DMA_TX_ARB_WEIGHT_OPT_DIR_CH2_REG	TX channel 2 weight arbitration optimization enable register	0x0330	R/W
AHB_DMA_IN_CRC_INIT_DATA_CHO_REG	RX channel 0 CRC initial value configuration register	0x0334	R/W
AHB_DMA_RX_CRC_WIDTH_CHO_REG	RX channel 0 CRC result width configuration register	0x0338	R/W
AHB_DMA_IN_CRC_CLEAR_CHO_REG	RX channel 0 CRC result clear register	0x033C	R/W
AHB_DMA_IN_CRC_FINAL_RESULT_CHO_REG	RX channel 0 CRC result register	0x0340	RO
AHB_DMA_RX_CRC_EN_WR_DATA_CHO_REG	RX channel 0 CRC intermediate result mask register	0x0344	R/W
AHB_DMA_RX_CRC_EN_ADDR_CHO_REG	RX channel 0 CRC data input mask target register	0x0348	R/W
AHB_DMA_RX_CRC_DATA_EN_WR_DATA_CHO_REG	RX channel 0 CRC data input mask register	0x034C	R/W
AHB_DMA_RX_CRC_DATA_EN_ADDR_CHO_REG	RX channel 0 CRC data input mask target register	0x0350	R/W
AHB_DMA_RX_CH_ARB_WEIGHT_CHO_REG	RX channel 0 arbitration weight configuration register	0x0354	R/W
AHB_DMA_RX_ARB_WEIGHT_OPT_DIR_CHO_REG	RX channel 0 weight arbitration optimization enable register	0x0358	R/W
AHB_DMA_IN_CRC_INIT_DATA_CH1_REG	RX channel 1 CRC initial value configuration register	0x035C	R/W
AHB_DMA_RX_CRC_WIDTH_CH1_REG	RX channel 1 CRC result width configuration register	0x0360	R/W
AHB_DMA_IN_CRC_CLEAR_CH1_REG	RX channel 1 CRC result clear register	0x0364	R/W
AHB_DMA_IN_CRC_FINAL_RESULT_CH1_REG	RX channel 1 CRC result register	0x0368	RO
AHB_DMA_RX_CRC_EN_WR_DATA_CH1_REG	RX channel 1 CRC intermediate result mask register	0x036C	R/W
AHB_DMA_RX_CRC_EN_ADDR_CH1_REG	RX channel 1 CRC data input mask target register	0x0370	R/W
AHB_DMA_RX_CRC_DATA_EN_WR_DATA_CH1_REG	RX channel 1 CRC data input mask register	0x0374	R/W
AHB_DMA_RX_CRC_DATA_EN_ADDR_CH1_REG	RX channel 1 CRC data input mask target register	0x0378	R/W
AHB_DMA_RX_CH_ARB_WEIGHT_CH1_REG	RX channel 1 arbitration weight configuration register	0x037C	R/W
AHB_DMA_RX_ARB_WEIGHT_OPT_DIR_CH1_REG	RX channel 1 weight arbitration optimization enable register	0x0380	R/W

Name	Description	Address	Access
AHB_DMA_IN_CRC_INIT_DATA_CH2_REG	RX channel 2 CRC initial value configuration register	0x0384	R/W
AHB_DMA_RX_CRC_WIDTH_CH2_REG	RX channel 2 CRC result width configuration register	0x0388	R/W
AHB_DMA_IN_CRC_CLEAR_CH2_REG	RX channel 2 CRC result clear register	0x038C	R/W
AHB_DMA_IN_CRC_FINAL_RESULT_CH2_REG	RX channel 2 CRC result register	0x0390	RO
AHB_DMA_RX_CRC_EN_WR_DATA_CH2_REG	RX channel 2 CRC intermediate result mask register	0x0394	R/W
AHB_DMA_RX_CRC_EN_ADDR_CH2_REG	RX channel 2 CRC data input mask target register	0x0398	R/W
AHB_DMA_RX_CRC_DATA_EN_WR_DATA_CH2_REG	RX channel 2 CRC data input mask register	0x039C	R/W
AHB_DMA_RX_CRC_DATA_EN_ADDR_CH2_REG	RX channel 2 CRC data input mask target register	0x03A0	R/W
AHB_DMA_RX_CH_ARB_WEIGHT_CH2_REG	RX channel 2 arbitration weight configuration register	0x03A4	R/W
AHB_DMA_RX_ARB_WEIGHT_OPT_DIR_CH2_REG	RX channel 2 weight arbitration optimization enable register	0x03A8	R/W
AHB_DMA_IN_LINK_ADDR_CHO_REG	Linked list descriptor configuration register of RX channel 0	0x03AC	R/W
AHB_DMA_IN_LINK_ADDR_CH1_REG	Linked list descriptor configuration register of RX channel 1	0x03B0	R/W
AHB_DMA_IN_LINK_ADDR_CH2_REG	Linked list descriptor configuration register of RX channel 2	0x03B4	R/W
AHB_DMA_OUT_LINK_ADDR_CHO_REG	Linked list descriptor configuration register of TX channel 0	0x03B8	R/W
AHB_DMA_OUT_LINK_ADDR_CH1_REG	Linked list descriptor configuration register of TX channel 1	0x03BC	R/W
AHB_DMA_OUT_LINK_ADDR_CH2_REG	Linked list descriptor configuration register of TX channel 2	0x03C0	R/W
AHB_DMA_INTR_MEM_START_ADDR_REG	Accessible address space start address configuration register	0x03C4	R/W
AHB_DMA_INTR_MEM_END_ADDR_REG	Accessible address space end address configuration register	0x03C8	R/W
AHB_DMA_ARB_TIMEOUT_REG	Weight arbitration timeout configuration register	0x03DC	R/W
AHB_DMA_WEIGHT_EN_REG	Weight arbitration enable register	0x0400	R/W
AHB_DMA_MODULE_CLK_EN_REG	Force clock enable register for all internal AHB DMA modules	0x0404	R/W
Version Register			
AHB_DMA_DATE_REG	Version control register	0x0068	R/W
Status Registers			
AHB_DMA_INFIFO_STATUS_CHO_REG	RX channel 0 FIFO status	0x0078	RO
AHB_DMA_IN_STATE_CHO_REG	RX channel 0 status	0x0084	RO
AHB_DMA_IN_SUC_EOF_DES_ADDR_CHO_REG	Receive descriptor address when EOF occurs on RX channel 0	0x0088	RO
AHB_DMA_IN_ERR_EOF_DES_ADDR_CHO_REG	Receive descriptor address when errors occur on RX channel 0	0x008C	RO

Name	Description	Address	Access
AHB_DMA_IN_DSCR_CHO_REG	Address of the next receive descriptor pointed by the current pre-read receive descriptor on RX channel 0	0x0090	RO
AHB_DMA_IN_DSCR_BFO_CHO_REG	Address of the current pre-read receive descriptor on RX channel 0	0x0094	RO
AHB_DMA_IN_DSCR_BF1_CHO_REG	Address of the previous pre-read receive descriptor on RX channel 0	0x0098	RO
AHB_DMA_OUTFIFO_STATUS_CHO_REG	TX channel 0 FIFO status	0x00D8	RO
AHB_DMA_OUT_STATE_CHO_REG	TX channel 0 status	0x00E4	RO
AHB_DMA_OUT_EOF_DES_ADDR_CHO_REG	Transmit descriptor address when EOF occurs on TX channel 0	0x00E8	RO
AHB_DMA_OUT_EOF_BFR_DES_ADDR_CHO_REG	The last transmit descriptor address when EOF occurs on TX channel 0	0x00EC	RO
AHB_DMA_OUT_DSCR_CHO_REG	Address of the next transmit descriptor pointed by the current pre-read transmit descriptor on TX channel 0	0x00F0	RO
AHB_DMA_OUT_DSCR_BFO_CHO_REG	Address of the current pre-read transmit descriptor on TX channel 0	0x00F4	RO
AHB_DMA_OUT_DSCR_BF1_CHO_REG	Address of the previous pre-read transmit descriptor on TX channel 0	0x00F8	RO
AHB_DMA_INFIFO_STATUS_CH1_REG	RX channel 1 FIFO status	0x0138	RO
AHB_DMA_IN_STATE_CH1_REG	RX channel 1 status	0x0144	RO
AHB_DMA_IN_SUC_EOF_DES_ADDR_CH1_REG	Receive descriptor address when EOF occurs on RX channel 1	0x0148	RO
AHB_DMA_IN_ERR_EOF_DES_ADDR_CH1_REG	Receive descriptor address when errors occur on RX channel 1	0x014C	RO
AHB_DMA_IN_DSCR_CH1_REG	Address of the next receive descriptor pointed by the current pre-read receive descriptor on RX channel 1	0x0150	RO
AHB_DMA_IN_DSCR_BFO_CH1_REG	Address of the current pre-read receive descriptor on RX channel 1	0x0154	RO
AHB_DMA_IN_DSCR_BF1_CH1_REG	Address of the previous pre-read receive descriptor on RX channel 1	0x0158	RO
AHB_DMA_OUTFIFO_STATUS_CH1_REG	TX channel 1 FIFO status	0x0198	RO
AHB_DMA_OUT_STATE_CH1_REG	TX channel 1 status	0x01A4	RO

Name	Description	Address	Access
AHB_DMA_OUT_EOF_DES_ADDR_CH1_REG	Transmit descriptor address when EOF occurs on TX channel 1	0x01A8	RO
AHB_DMA_OUT_EOF_BFR_DES_ADDR_CH1_REG	The last transmit descriptor address when EOF occurs on TX channel 1	0x01AC	RO
AHB_DMA_OUT_DSCR_CH1_REG	Address of the next transmit descriptor pointed by the current pre-read transmit descriptor on TX channel 1	0x01B0	RO
AHB_DMA_OUT_DSCR_BFO_CH1_REG	Address of the current pre-read transmit descriptor on TX channel 1	0x01B4	RO
AHB_DMA_OUT_DSCR_BF1_CH1_REG	Address of the previous pre-read transmit descriptor on TX channel 1	0x01B8	RO
AHB_DMA_INFIFO_STATUS_CH2_REG	RX channel 2 FIFO status	0x01F8	RO
AHB_DMA_IN_STATE_CH2_REG	RX channel 2 status	0x0204	RO
AHB_DMA_IN_SUC_EOF_DES_ADDR_CH2_REG	Receive descriptor address when EOF occurs on RX channel 2	0x0208	RO
AHB_DMA_IN_ERR_EOF_DES_ADDR_CH2_REG	Receive descriptor address when errors occur on RX channel 2	0x020C	RO
AHB_DMA_IN_DSCR_CH2_REG	Address of the next receive descriptor pointed by the current pre-read receive descriptor on RX channel 2	0x0210	RO
AHB_DMA_IN_DSCR_BFO_CH2_REG	Address of the current pre-read receive descriptor on RX channel 2	0x0214	RO
AHB_DMA_IN_DSCR_BF1_CH2_REG	Address of the previous pre-read receive descriptor on RX channel 2	0x0218	RO
AHB_DMA_OUTFIFO_STATUS_CH2_REG	TX channel 2 FIFO status	0x0258	RO
AHB_DMA_OUT_STATE_CH2_REG	TX channel 2 status	0x0264	RO
AHB_DMA_OUT_EOF_DES_ADDR_CH2_REG	Transmit descriptor address when EOF occurs on TX channel 2	0x0268	RO
AHB_DMA_OUT_EOF_BFR_DES_ADDR_CH2_REG	The last transmit descriptor address when EOF occurs on TX channel 2	0x026C	RO
AHB_DMA_OUT_DSCR_CH2_REG	Address of the next transmit descriptor pointed by the current pre-read transmit descriptor on TX channel 2	0x0270	RO
AHB_DMA_OUT_DSCR_BFO_CH2_REG	Address of the current pre-read transmit descriptor on TX channel 2	0x0274	RO

Name	Description	Address	Access
AHB_DMA_OUT_DSCR_BF1_CH2_REG	Address of the previous pre-read transmit descriptor on TX channel 2	0x0278	RO
AHB_DMA_AHBINF_RESP_ERR_STATUS0_REG	Address of the transfer for which AHB DMA received an AHB bus error response	0x0408	RO
AHB_DMA_AHBINF_RESP_ERR_STATUS1_REG	Transfer type, ID, and channel ID of the transfer for which AHB DMA received an AHB bus error response	0x040C	RO
AHB_DMA_IN_DONE_DES_ADDR_CHO_REG	Address of the completed descriptor for RX channel 0	0x0410	RO
AHB_DMA_OUT_DONE_DES_ADDR_CHO_REG	Address of the completed descriptor for TX channel 0	0x0414	RO
AHB_DMA_IN_DONE_DES_ADDR_CH1_REG	Address of the completed descriptor for RX channel 1	0x0418	RO
AHB_DMA_OUT_DONE_DES_ADDR_CH1_REG	Address of the completed descriptor for TX channel 1	0x041C	RO
AHB_DMA_IN_DONE_DES_ADDR_CH2_REG	Address of the completed descriptor for RX channel 2	0x0420	RO
AHB_DMA_OUT_DONE_DES_ADDR_CH2_REG	Address of the completed descriptor for TX channel 2	0x0424	RO
Priority Registers			
AHB_DMA_IN_PRI_CHO_REG	Priority register of RX channel 0	0x009C	R/W
AHB_DMA_OUT_PRI_CHO_REG	Priority register of TX channel 0	0x00FC	R/W
AHB_DMA_IN_PRI_CH1_REG	Priority register of RX channel 1	0x015C	R/W
AHB_DMA_OUT_PRI_CH1_REG	Priority register of TX channel 1	0x01BC	R/W
AHB_DMA_IN_PRI_CH2_REG	Priority register of RX channel 2	0x021C	R/W
AHB_DMA_OUT_PRI_CH2_REG	Priority register of TX channel 2	0x027C	R/W
Peripheral Select Registers			
AHB_DMA_IN_PERI_SEL_CHO_REG	Peripheral selection register of RX channel 0	0x00A0	R/W
AHB_DMA_OUT_PERI_SEL_CHO_REG	Peripheral selection register of TX channel 0	0x0100	R/W
AHB_DMA_IN_PERI_SEL_CH1_REG	Peripheral selection register of RX channel 1	0x0160	R/W
AHB_DMA_OUT_PERI_SEL_CH1_REG	Peripheral selection register of TX channel 1	0x01C0	R/W
AHB_DMA_IN_PERI_SEL_CH2_REG	Peripheral selection register of RX channel 2	0x0220	R/W
AHB_DMA_OUT_PERI_SEL_CH2_REG	Peripheral selection register of TX channel 2	0x0280	R/W

4.8.2 GDMA-AXI Register Summary

The addresses in this section are relative to GDMA-AXI base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
Interrupt Registers			
AXI_DMA_IN_INT_RAW_CHO_REG	RX channel 0 raw interrupt status register	0x0000	R/WTC/SS
AXI_DMA_IN_INT_ST_CHO_REG	RX channel 0 masked interrupt status register	0x0004	RO
AXI_DMA_IN_INT_ENA_CHO_REG	RX channel 0 interrupt enable register	0x0008	R/W
AXI_DMA_IN_INT_CLR_CHO_REG	RX channel 0 interrupt clear register	0x000C	WT
AXI_DMA_IN_INT_RAW_CH1_REG	RX channel 1 raw interrupt status register	0x0068	R/WTC/SS
AXI_DMA_IN_INT_ST_CH1_REG	RX channel 1 masked interrupt status register	0x006C	RO
AXI_DMA_IN_INT_ENA_CH1_REG	RX channel 1 interrupt enable register	0x0070	R/W
AXI_DMA_IN_INT_CLR_CH1_REG	RX channel 1 interrupt clear register	0x0074	WT
AXI_DMA_IN_INT_RAW_CH2_REG	RX channel 2 raw interrupt status register	0x00D0	R/WTC/SS
AXI_DMA_IN_INT_ST_CH2_REG	RX channel 2 masked interrupt status register	0x00D4	RO
AXI_DMA_IN_INT_ENA_CH2_REG	RX channel 2 interrupt enable register	0x00D8	R/W
AXI_DMA_IN_INT_CLR_CH2_REG	RX channel 2 interrupt clear register	0x00DC	WT
AXI_DMA_OUT_INT_RAW_CHO_REG	TX channel 0 raw interrupt status register	0x0138	R/WTC/SS
AXI_DMA_OUT_INT_ST_CHO_REG	TX channel 0 masked interrupt status register	0x013C	RO
AXI_DMA_OUT_INT_ENA_CHO_REG	TX channel 0 interrupt enable register	0x0140	R/W
AXI_DMA_OUT_INT_CLR_CHO_REG	TX channel 0 interrupt clear register	0x0144	WT
AXI_DMA_OUT_INT_RAW_CH1_REG	TX channel 1 raw interrupt status register	0x01A0	R/WTC/SS
AXI_DMA_OUT_INT_ST_CH1_REG	TX channel 1 masked interrupt status register	0x01A4	RO
AXI_DMA_OUT_INT_ENA_CH1_REG	TX channel 1 interrupt enable register	0x01A8	R/W
AXI_DMA_OUT_INT_CLR_CH1_REG	TX channel 1 interrupt clear register	0x01AC	WT
AXI_DMA_OUT_INT_RAW_CH2_REG	TX channel 2 raw interrupt status register	0x0208	R/WTC/SS
AXI_DMA_OUT_INT_ST_CH2_REG	TX channel 2 masked interrupt status register	0x020C	RO
AXI_DMA_OUT_INT_ENA_CH2_REG	TX channel 2 interrupt enable register	0x0210	R/W

Name	Description	Address	Access
AXI_DMA_OUT_INT_CLR_CH2_REG	TX channel 2 interrupt clear register	0x0214	WT
Configuration Registers			
AXI_DMA_IN_CONFO_CHO_REG	Configuration register 0 of RX channel 0	0x0010	R/W
AXI_DMA_IN_CONF1_CHO_REG	Configuration register 1 of RX channel 0	0x0014	R/W
AXI_DMA_IN_POP_CHO_REG	Pop control register of RX channel 0	0x001C	varies
AXI_DMA_IN_LINK1_CHO_REG	Linked list descriptor configuration and control register 1 of RX channel 0	0x0020	varies
AXI_DMA_IN_LINK2_CHO_REG	Linked list descriptor configuration and control register 2 of RX channel 0	0x0024	R/W
AXI_DMA_IN_CRC_INIT_DATA_CHO_REG	RX channel 0 CRC initial value configuration register	0x0048	R/W
AXI_DMA_RX_CRC_WIDTH_CHO_REG	RX channel 0 CRC result width configuration register	0x004C	R/W
AXI_DMA_IN_CRC_CLEAR_CHO_REG	RX channel 0 CRC result clear register	0x0050	R/W
AXI_DMA_IN_CRC_FINAL_RESULT_CHO_REG	RX channel 0 CRC result register	0x0054	RO
AXI_DMA_RX_CRC_EN_ADDR_CHO_REG	TX channel 0 CRC intermediate result mask target register	0x005C	R/W
AXI_DMA_RX_CRC_DATA_EN_WR_DATA_CHO_REG	CRC RX channel 0 CRC intermediate result mask register	0x0060	R/W
AXI_DMA_RX_CRC_DATA_EN_ADDR_CHO_REG	RX channel 0 CRC data input mask target register	0x0064	R/W
AXI_DMA_IN_CONFO_CH1_REG	Configuration register 0 of RX channel 1	0x0078	R/W
AXI_DMA_IN_CONF1_CH1_REG	Configuration register 1 of RX channel 1	0x007C	R/W
AXI_DMA_IN_POP_CH1_REG	Pop control register of RX channel 1	0x0084	varies
AXI_DMA_IN_LINK1_CH1_REG	Linked list descriptor configuration and control register 1 of RX channel 1	0x0088	varies
AXI_DMA_IN_LINK2_CH1_REG	Linked list descriptor configuration and control register 2 of RX channel 1	0x008C	R/W
AXI_DMA_IN_CRC_INIT_DATA_CH1_REG	RX channel 1 CRC initial value configuration register	0x00B0	R/W
AXI_DMA_RX_CRC_WIDTH_CH1_REG	RX channel 1 CRC result width configuration register	0x00B4	R/W
AXI_DMA_IN_CRC_CLEAR_CH1_REG	RX channel 1 CRC result clear register	0x00B8	R/W
AXI_DMA_IN_CRC_FINAL_RESULT_CH1_REG	RX channel 1 CRC result register	0x00BC	RO
AXI_DMA_RX_CRC_EN_ADDR_CH1_REG	RX channel 1 CRC intermediate result mask target register	0x00C4	R/W
AXI_DMA_RX_CRC_DATA_EN_WR_DATA_CH1_REG	CRC RX channel 1 CRC intermediate result mask register	0x00C8	R/W

Name	Description	Address	Access
AXI_DMA_RX_CRC_DATA_EN_ADDR_CH1_REG	RX channel 1 CRC data input mask target register	0x00CC	R/W
AXI_DMA_IN_CONFO_CH2_REG	Configuration register 0 of RX channel 2	0x00E0	R/W
AXI_DMA_IN_CONF1_CH2_REG	Configuration register 1 of RX channel 2	0x00E4	R/W
AXI_DMA_IN_POP_CH2_REG	Pop control register of RX channel 2	0x00EC	varies
AXI_DMA_IN_LINK1_CH2_REG	Linked list descriptor configuration and control register 1 of RX channel 2	0x00F0	varies
AXI_DMA_IN_LINK2_CH2_REG	Linked list descriptor configuration and control register 2 of RX channel 2	0x00F4	R/W
AXI_DMA_IN_CRC_INIT_DATA_CH2_REG	RX channel 2 CRC initial value configuration register	0x0118	R/W
AXI_DMA_RX_CRC_WIDTH_CH2_REG	RX channel 2 CRC result width configuration register	0x011C	R/W
AXI_DMA_IN_CRC_CLEAR_CH2_REG	RX channel 2 CRC result clear register	0x0120	R/W
AXI_DMA_IN_CRC_FINAL_RESULT_CH2_REG	RX channel 2 CRC result register	0x0124	RO
AXI_DMA_RX_CRC_EN_ADDR_CH2_REG	RX channel 2 CRC intermediate result mask target register	0x012C	R/W
AXI_DMA_RX_CRC_DATA_EN_WR_DATA_CH2_REG	CRC RX channel 2 CRC intermediate result mask register	0x0130	R/W
AXI_DMA_RX_CRC_DATA_EN_ADDR_CH2_REG	RX channel 2 CRC data input mask target register	0x0134	R/W
AXI_DMA_OUT_CONFO_CHO_REG	Configuration register 0 of TX channel 0	0x0148	R/W
AXI_DMA_OUT_CONF1_CHO_REG	Configuration register 1 of TX channel 0	0x014C	R/W
AXI_DMA_OUT_PUSH_CHO_REG	Push control register of TX channel 0	0x0154	varies
AXI_DMA_OUT_LINK1_CHO_REG	Linked list descriptor configuration and control register 1 of TX channel 0	0x0158	varies
AXI_DMA_OUT_LINK2_CHO_REG	Linked list descriptor configuration and control register 2 of TX channel 0	0x015C	R/W
AXI_DMA_OUT_CRC_INIT_DATA_CHO_REG	TX channel 0 CRC initial value configuration register	0x0180	R/W
AXI_DMA_TX_CRC_WIDTH_CHO_REG	TX channel 0 CRC result width configuration register	0x0184	R/W
AXI_DMA_OUT_CRC_CLEAR_CHO_REG	TX channel 0 CRC result clear register	0x0188	R/W
AXI_DMA_OUT_CRC_FINAL_RESULT_CHO_REG	TX channel 0 CRC result register	0x018C	RO
AXI_DMA_TX_CRC_EN_WR_DATA_CHO_REG	CRC TX channel 0 CRC intermediate result mask register	0x0190	R/W
AXI_DMA_TX_CRC_EN_ADDR_CHO_REG	TX channel 0 CRC intermediate result mask target register	0x0194	R/W
AXI_DMA_TX_CRC_DATA_EN_WR_DATA_CHO_REG	TX channel 0 CRC data input mask register	0x0198	R/W

Name	Description	Address	Access
AXI_DMA_TX_CRC_DATA_EN_ADDR_CHO_REG	TX channel 0 CRC data input mask target register	0x019C	R/W
AXI_DMA_OUT_CONFO_CH1_REG	Configuration register 0 of TX channel 1	0x01B0	R/W
AXI_DMA_OUT_CONF1_CH1_REG	Configuration register 1 of TX channel 1	0x01B4	R/W
AXI_DMA_OUT_PUSH_CH1_REG	Push control register of TX channel 1	0x01BC	varies
AXI_DMA_OUT_LINK1_CH1_REG	Linked list descriptor configuration and control register 1 of TX channel 1	0x01C0	varies
AXI_DMA_OUT_LINK2_CH1_REG	Linked list descriptor configuration and control register 2 of TX channel 1	0x01C4	R/W
AXI_DMA_OUT_CRC_INIT_DATA_CH1_REG	TX channel 1 CRC initial value configuration register	0x01E8	R/W
AXI_DMA_TX_CRC_WIDTH_CH1_REG	TX channel 1 CRC result width configuration register	0x01EC	R/W
AXI_DMA_OUT_CRC_CLEAR_CH1_REG	TX channel 1 CRC result clear register	0x01F0	R/W
AXI_DMA_OUT_CRC_FINAL_RESULT_CH1_REG	TX channel 1 CRC result register	0x01F4	RO
AXI_DMA_TX_CRC_EN_WR_DATA_CH1_REG	CRC TX channel 1 CRC intermediate result mask register	0x01F8	R/W
AXI_DMA_TX_CRC_EN_ADDR_CH1_REG	TX channel 1 CRC intermediate result mask target register	0x01FC	R/W
AXI_DMA_TX_CRC_DATA_EN_WR_DATA_CH1_REG	TX channel 1 CRC data input mask target register	0x0200	R/W
AXI_DMA_TX_CRC_DATA_EN_ADDR_CH1_REG	TX channel 1 CRC data input mask target register	0x0204	R/W
AXI_DMA_OUT_CONFO_CH2_REG	Configuration register 0 of TX channel 2	0x0218	R/W
AXI_DMA_OUT_CONF1_CH2_REG	Configuration register 1 of TX channel 2	0x021C	R/W
AXI_DMA_OUT_PUSH_CH2_REG	Push control register of TX channel 2	0x0224	varies
AXI_DMA_OUT_LINK1_CH2_REG	Linked list descriptor configuration and control register of TX channel 2	0x0228	varies
AXI_DMA_OUT_LINK2_CH2_REG	Linked list descriptor configuration and control register 2 1of TX channel 2	0x022C	R/W
AXI_DMA_OUT_CRC_INIT_DATA_CH2_REG	TX channel 2 CRC initial value configuration register	0x0250	R/W
AXI_DMA_TX_CRC_WIDTH_CH2_REG	TX channel 2 CRC result width configuration register	0x0254	R/W
AXI_DMA_OUT_CRC_CLEAR_CH2_REG	TX channel 2 CRC result clear register	0x0258	R/W
AXI_DMA_OUT_CRC_FINAL_RESULT_CH2_REG	TX channel 2 CRC result register	0x025C	RO
AXI_DMA_TX_CRC_EN_WR_DATA_CH2_REG	CRC TX channel 2 CRC intermediate result mask register	0x0260	R/W
AXI_DMA_TX_CRC_EN_ADDR_CH2_REG	TX channel 2 CRC intermediate result mask target register	0x0264	R/W

Name	Description	Address	Access
AXI_DMA_TX_CRC_DATA_EN_WR_DATA_CH2_REG	TX channel 2 CRC data input mask target register	0x0268	R/W
AXI_DMA_TX_CRC_DATA_EN_ADDR_CH2_REG	TX channel 2 CRC data input mask target register	0x026C	R/W
AXI_DMA_ARB_TIMEOUT_REG	Arbitration timeout configuration register	0x0270	R/W
AXI_DMA_WEIGHT_EN_REG	Weight arbitration enable register	0x0274	R/W
AXI_DMA_IN_MEM_CONF_REG	Internal memory power configuration register for RX channel	0x0278	R/W
AXI_DMA_INTR_MEM_START_ADDR_REG	Accessible internal memory start address configuration register	0x027C	R/W
AXI_DMA_INTR_MEM_END_ADDR_REG	Accessible internal memory end address configuration register	0x0280	R/W
AXI_DMA_EXTR_MEM_START_ADDR_REG	Accessible external memory start address configuration register	0x0284	R/W
AXI_DMA_EXTR_MEM_END_ADDR_REG	Accessible external memory end address configuration register	0x0288	R/W
AXI_DMA_MISC_CONF_REG	Miscellaneous register	0x02A8	R/W
AXI_DMA_LINK_SWITCH_STATE_REG	Linked list switching configuration register	0x02DC	R/W
AXI_DMA_RX_CRC_EN_WR_DATA_CHO_REG	CRC RX channel 0 CRC intermediate result mask register	0x0058	R/W
AXI_DMA_RX_CRC_EN_WR_DATA_CH1_REG	CRC RX channel 1 CRC intermediate result mask register	0x00C0	R/W
AXI_DMA_RX_CRC_EN_WR_DATA_CH2_REG	CRC RX channel 2 CRC intermediate result mask register	0x0128	R/W
Status Registers			
AXI_DMA_INFIFO_STATUS_CHO_REG	RX channel 0 FIFO status register	0x0018	RO
AXI_DMA_IN_STATE_CHO_REG	RX channel 0 status register	0x0028	RO
AXI_DMA_IN_SUC_EOF_DES_ADDR_CHO_REG	Receive descriptor address when EOF occurs on RX channel 0	0x002C	RO
AXI_DMA_IN_ERR_EOF_DES_ADDR_CHO_REG	Receive descriptor address when errors occur on RX channel 0	0x0030	RO
AXI_DMA_IN_DSCR_CHO_REG	Address of the next receive descriptor pointed by the current pre-read receive descriptor on RX channel 0	0x0034	RO
AXI_DMA_IN_DSCR_BFO_CHO_REG	Address of the current pre-read receive descriptor on RX channel 0	0x0038	RO
AXI_DMA_IN_DSCR_BF1_CHO_REG	Address of the previous pre-read receive descriptor on RX channel 0	0x003C	RO
AXI_DMA_INFIFO_STATUS_CH1_REG	RX channel 1 FIFO status register	0x0080	RO
AXI_DMA_IN_STATE_CH1_REG	RX channel 1 status register	0x0090	RO
AXI_DMA_IN_SUC_EOF_DES_ADDR_CH1_REG	Receive descriptor address when EOF occurs on RX channel 1	0x0094	RO
AXI_DMA_IN_ERR_EOF_DES_ADDR_CH1_REG	Receive descriptor address when errors occur on RX channel 1	0x0098	RO

Name	Description	Address	Access
AXI_DMA_IN_DSCR_CH1_REG	Address of the next receive descriptor pointed by the current pre-read receive descriptor on RX channel 1	0x009C	RO
AXI_DMA_IN_DSCR_BFO_CH1_REG	Address of the current pre-read receive descriptor on RX channel 1	0x00A0	RO
AXI_DMA_IN_DSCR_BF1_CH1_REG	Address of the previous pre-read receive descriptor on RX channel 1	0x00A4	RO
AXI_DMA_INFIFO_STATUS_CH2_REG	RX channel 2 FIFO status register	0x00E8	RO
AXI_DMA_IN_STATE_CH2_REG	RX channel 2 status register	0x00F8	RO
AXI_DMA_IN_SUC_EOF_DES_ADDR_CH2_REG	Receive descriptor address when EOF occurs on RX channel 2	0x00FC	RO
AXI_DMA_IN_ERR_EOF_DES_ADDR_CH2_REG	Receive descriptor address when errors occur on RX channel 2	0x0100	RO
AXI_DMA_IN_DSCR_CH2_REG	Address of the next receive descriptor pointed by the current pre-read receive descriptor on RX channel 2	0x0104	RO
AXI_DMA_IN_DSCR_BFO_CH2_REG	Address of the current pre-read receive descriptor on RX channel 2	0x0108	RO
AXI_DMA_IN_DSCR_BF1_CH2_REG	Address of the previous pre-read receive descriptor on RX channel 2	0x010C	RO
AXI_DMA_OUTFIFO_STATUS_CHO_REG	TX channel 0 FIFO status	0x0150	RO
AXI_DMA_OUT_STATE_CHO_REG	TX channel 0 status	0x0160	RO
AXI_DMA_OUT_EOF_DES_ADDR_CHO_REG	Transmit descriptor address when EOF occurs on TX channel 0	0x0164	RO
AXI_DMA_OUT_EOF_BFR_DES_ADDR_CHO_REG	The last transmit descriptor address when EOF occurs on TX channel 0	0x0168	RO
AXI_DMA_OUT_DSCR_CHO_REG	Address of the next transmit descriptor pointed by the current pre-read transmit descriptor on TX channel 0	0x016C	RO
AXI_DMA_OUT_DSCR_BFO_CHO_REG	Address of the current pre-read transmit descriptor on TX channel 0	0x0170	RO
AXI_DMA_OUT_DSCR_BF1_CHO_REG	Address of the previous pre-read transmit descriptor on TX channel 0	0x0174	RO
AXI_DMA_OUTFIFO_STATUS_CH1_REG	TX channel 0 FIFO status	0x01B8	RO
AXI_DMA_OUT_STATE_CH1_REG	TX channel 1 status	0x01C8	RO

Name	Description	Address	Access
AXI_DMA_OUT_EOF_DES_ADDR_CH1_REG	Transmit descriptor address when EOF occurs on TX channel 1	0x01CC	RO
AXI_DMA_OUT_EOF_BFR_DES_ADDR_CH1_REG	The last transmit descriptor address when EOF occurs on TX channel 1	0x01D0	RO
AXI_DMA_OUT_DSCR_CH1_REG	Address of the next transmit descriptor pointed by the current pre-read transmit descriptor on TX channel 1	0x01D4	RO
AXI_DMA_OUT_DSCR_BFO_CH1_REG	Address of the current pre-read transmit descriptor on TX channel 1	0x01D8	RO
AXI_DMA_OUT_DSCR_BF1_CH1_REG	Address of the previous pre-read transmit descriptor on TX channel 1	0x01DC	RO
AXI_DMA_OUTFIFO_STATUS_CH2_REG	TX channel 2 FIFO status	0x0220	RO
AXI_DMA_OUT_STATE_CH2_REG	TX channel 2 status	0x0230	RO
AXI_DMA_OUT_EOF_DES_ADDR_CH2_REG	Transmit descriptor address when EOF occurs on TX channel 2	0x0234	RO
AXI_DMA_OUT_EOF_BFR_DES_ADDR_CH2_REG	The last transmit descriptor address when EOF occurs on TX channel 2	0x0238	RO
AXI_DMA_OUT_DSCR_CH2_REG	Address of the next transmit descriptor pointed by the current pre-read transmit descriptor on TX channel 2	0x023C	RO
AXI_DMA_OUT_DSCR_BFO_CH2_REG	Address of the current pre-read transmit descriptor on TX channel 2	0x0240	RO
AXI_DMA_OUT_DSCR_BF1_CH2_REG	Address of the previous pre-read transmit descriptor on TX channel 2	0x0244	RO
AXI_DMA_IN_RESET_AVAIL_CHO_REG	RX channel 0 reset status register	0x028C	RO
AXI_DMA_IN_RESET_AVAIL_CH1_REG	RX channel 1 reset status register	0x0290	RO
AXI_DMA_IN_RESET_AVAIL_CH2_REG	RX channel 2 reset status register	0x0294	RO
AXI_DMA_OUT_RESET_AVAIL_CHO_REG	TX channel 0 reset status register	0x0298	RO
AXI_DMA_OUT_RESET_AVAIL_CH1_REG	TX channel 1 reset status register	0x029C	RO
AXI_DMA_OUT_RESET_AVAIL_CH2_REG	TX channel 2 reset status register	0x02A0	RO
Priority Registers			
AXI_DMA_IN_PRI_CHO_REG	Priority register of RX channel 0	0x0040	R/W
AXI_DMA_IN_PRI_CH1_REG	Priority register of RX channel 1	0x00A8	R/W

Name	Description	Address	Access
AXI_DMA_IN_PRI_CH2_REG	Priority register of RX channel 2	0x0110	R/W
AXI_DMA_OUT_PRI_CHO_REG	Priority register of TX channel 0	0x0178	R/W
AXI_DMA_OUT_PRI_CH1_REG	Priority register of TX channel 1	0x01E0	R/W
AXI_DMA_OUT_PRI_CH2_REG	Priority register of TX channel 2	0x0248	R/W
Peripheral Select Registers			
AXI_DMA_IN_PERI_SEL_CHO_REG	Peripheral selection register of RX channel 0	0x0044	R/W
AXI_DMA_IN_PERI_SEL_CH1_REG	Peripheral selection register of RX channel 1	0x00AC	R/W
AXI_DMA_IN_PERI_SEL_CH2_REG	Peripheral selection register of RX channel 2	0x0114	R/W
AXI_DMA_OUT_PERI_SEL_CHO_REG	Peripheral selection register of TX channel 0	0x017C	R/W
AXI_DMA_OUT_PERI_SEL_CH1_REG	Peripheral selection register of TX channel 1	0x01E4	R/W
AXI_DMA_OUT_PERI_SEL_CH2_REG	Peripheral selection register of TX channel 2	0x024C	R/W
Version Register			
AXI_DMA_DATE_REG	Version control register	0x02D8	R/W

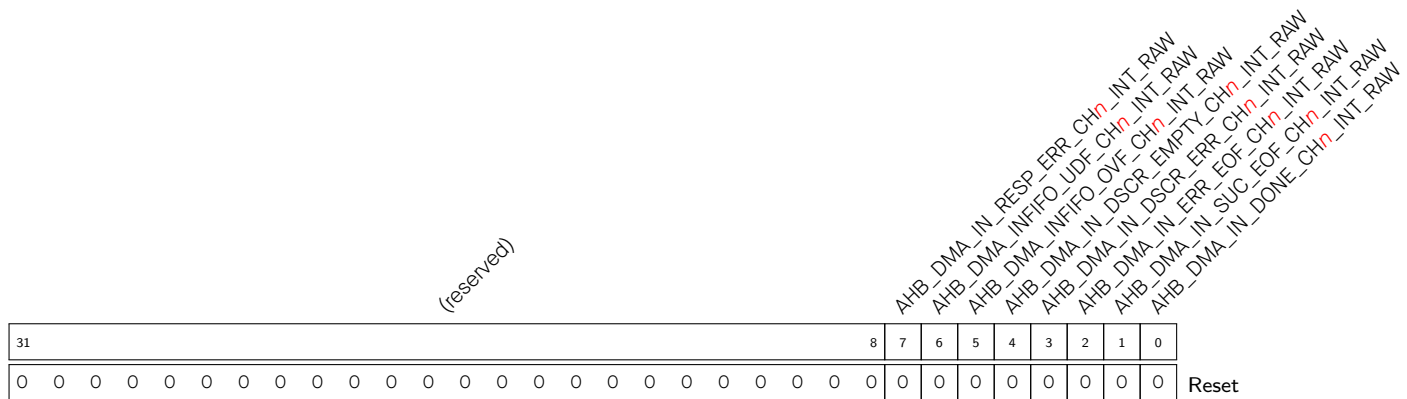
4.9 Registers

4.9.1 GDMA-AHB Registers

The addresses in this section are relative to GDMA-AHB base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

For how to program reserved fields, please refer to Section [Programming Reserved Register Field](#).

Register 4.1. AHB_DMA_IN_INT_RAW_CH n _REG (n : 0-2) (0x0000+0x10* n)



AHB_DMA_IN_DONE_CH n _INT_RAW The raw interrupt status of AHB_DMA_IN_DONE_CH n _INT. (R/WTC/SS)

AHB_DMA_IN_SUC_EOF_CH n _INT_RAW The raw interrupt status of AHB_DMA_IN_SUC_EOF_CH n _INT. For UHCI, this bit turns to 1 when the last data byte pointed by one receive descriptor has been received and no data error is detected for RX channel n . (R/WTC/SS)

AHB_DMA_IN_ERR_EOF_CH n _INT_RAW The raw interrupt status of AHB_DMA_IN_ERR_EOF_CH n _INT. Valid only for UHCI. (R/WTC/SS)

AHB_DMA_IN_DSCR_ERR_CH n _INT_RAW The raw interrupt status of AHB_DMA_IN_DSCR_ERR_CH n _INT. (R/WTC/SS)

AHB_DMA_IN_DSCR_EMPTY_CH n _INT_RAW The raw interrupt status of AHB_DMA_IN_DSCR_EMPTY_CH n _INT. (R/WTC/SS)

AHB_DMA_IN_INFIFO_OVF_CH n _INT_RAW The raw interrupt status of AHB_DMA_IN_INFIFO_OVF_CH n _INT. (R/WTC/SS)

AHB_DMA_IN_INFIFO_UDF_CH n _INT_RAW The raw interrupt status of AHB_DMA_IN_INFIFO_UDF_CH n _INT. (R/WTC/SS)

AHB_DMA_IN_RESP_ERR_CH n _INT_RAW The raw interrupt status of AHB_DMA_IN_RESP_ERR_CH n _INT. (R/WTC/SS)

Register 4.2. AHB_DMA_IN_INT_ST_CH n _REG (n : 0-2) (0x0004+0x10* n)

(reserved)

31	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0

Reset

AHB_DMA_IN_RESP_ERR_CHn_INT_ST
 AHB_DMA_IN_FIFO_OVF_CHn_INT_ST
 AHB_DMA_IN_DSCR_ERR_CHn_INT_ST
 AHB_DMA_IN_DSCR_EMPTY_CHn_INT_ST
 AHB_DMA_IN_ERR_EOF_CHn_INT_ST
 AHB_DMA_IN_SUC_EOF_CHn_INT_ST
 AHB_DMA_IN_DONE_CHn_INT_ST

AHB_DMA_IN_DONE_CH n _INT_ST The masked interrupt status of AHB_DMA_IN_DONE_CH n _INT.
(RO)

AHB_DMA_IN_SUC_EOF_CH_nINT_ST The masked interrupt status of AHB_DMA_IN_SUC_EOF_CH_nINT. (RO)

AHB_DMA_IN_ERR_EOF_CH*n***_INT_ST** The masked interrupt status of AHB_DMA_IN_ERR_EOF_CH*n*_INT. (RO)

AHB_DMA_IN_DSCR_ERR_CH n _INT_ST The masked interrupt status of AHB_DMA_IN_DSCR_ERR_CH n _INT. (RO)

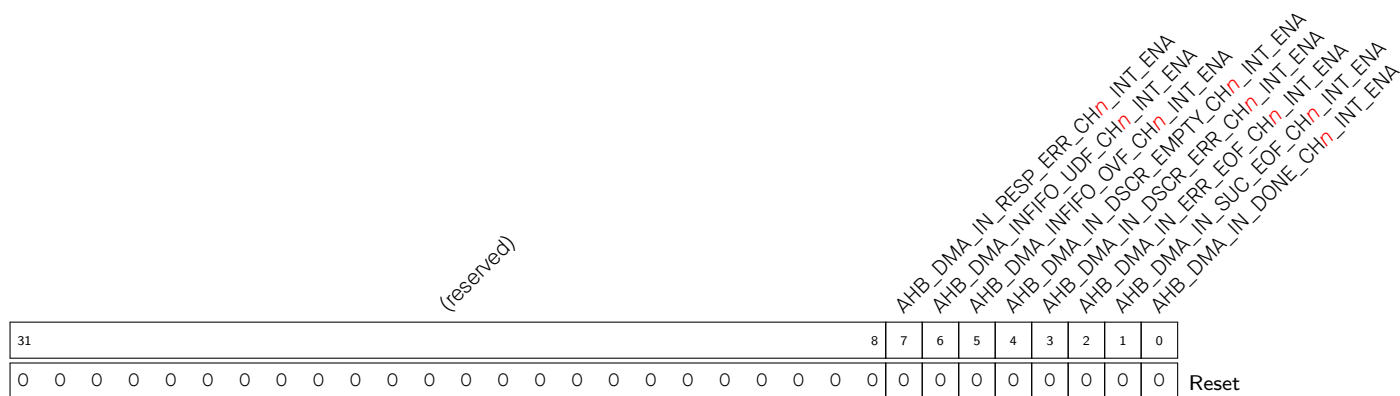
AHB_DMA_IN_DSCR_EMPTY_CH n _INT_ST The masked interrupt status of
AHB_DMA_IN_DSCR_EMPTY_CH n _INT. (RO)

AHB_DMA_INFIFO_OVF_CH n _INT_ST The masked interrupt status of AHB_DMA_INFIFO_OVF_CH n _INT. (RO)

AHB_DMA_INFIFO_UDF_CH n _INT_ST The masked interrupt status of AHB_DMA_INFIFO_UDF_CH n _INT. (RO)

AHB_DMA_IN_RESP_ERR_CH n _INT_ST The masked interrupt status of AHB_DMA_IN_RESP_ERR_CH n _INT. (RO)

Register 4.3. AHB_DMA_IN_INT_ENA_CH n _REG (n : 0-2) (0x0008+0x10* n)



AHB_DMA_IN_DONE_CH_n_INT_ENA Write 1 to enable AHB_DMA_IN_DONE_CH_n_INT. (R/W)

AHB_DMA_IN_SUC_EOF_CH***n***_INT_ENA Write 1 to enable AHB_DMA_IN_SUC_EOF_CH***n***_INT.
(R/W)

AHB_DMA_IN_ERR_EOF_CH***n***_INT_ENA Write 1 to enable AHB_DMA_IN_ERR_EOF_CH***n***_INT.
(R/W)

AHB_DMA_IN_DSCR_ERR_CH*n*_INT_ENA Write 1 to enable AHB_DMA_IN_DSCR_ERR_CH*n*_INT.
(R/W)

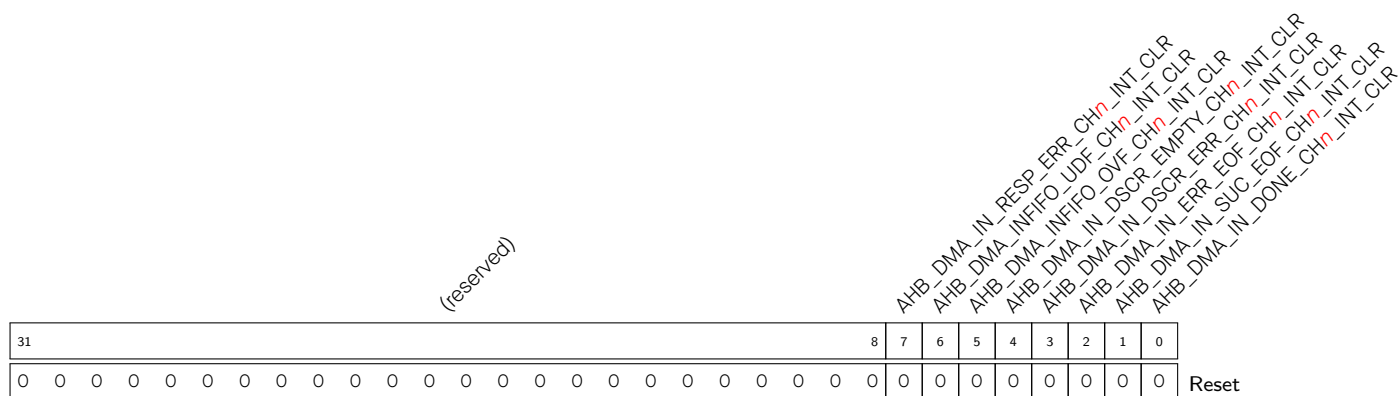
AHB_DMA_IN_DSCR_EMPTY_CH n _INT_ENA Write 1 to enable AHB_DMA_IN_DSCR_EMPTY_CH n _INT.
(R/W)

AHB_DMA_INFIFO_OVF_CH_n_INT_ENA Write 1 to enable AHB_DMA_INFIFO_OVF_CH_n_INT. (R/W)

AHB_DMA_INFIFO_UDF_CH_n_INT_ENA Write 1 to enable AHB_DMA_INFIFO_UDF_CH_n_INT. (R/W)

AHB_DMA_IN_RESP_ERR_CH***n***_INT_ENA Write 1 to enable AHB_DMA_IN_RESP_ERR_CH***n***_INT.
(R/W)

Register 4.4. AHB_DMA_IN_INT_CLR_CH n _REG (n : 0-2) (0x000C+0x10* n)



AHB_DMA_IN_DONE_CH_n_INT_CLR Write 1 to clear AHB_DMA_IN_DONE_CH_n_INT. (WT)

AHB_DMA_IN_SUC_EOF_CH_n_INT_CLR Write 1 to clear AHB_DMA_IN_SUC_EOF_CH_n_INT. (WT)

AHB_DMA_IN_ERR_EOF_CH_n_INT_CLR Write 1 to clear AHB_DMA_IN_ERR_EOF_CH_n_INT. (WT)

AHB_DMA_IN_DSCR_ERR_CH n _INT_CLR Write 1 to clear AHB_DMA_IN_DSCR_ERR_CH n _INT.
(WT)

AHB_DMA_IN_DSCR_EMPTY_CH n _INT_CLR Write 1 to clear AHB_DMA_IN_DSCR_EMPTY_CH n _INT.
(WT)

AHB_DMA_INFIFO_OVF_CH_n_INT_CLR Write 1 to clear AHB_DMA_INFIFO_OVF_CH_n_INT. (WT)

AHB DMA INFIFO UDF CH_n INT CLR Write 1 to clear AHB DMA INFIFO UDF CH_n INT. (WT)

AHB_DMA_IN_RESP_ERR_CH_n INT CLR Write 1 to clear AHB_DMA_IN_RESP_ERR_CH_n INT. (WT)

Register 4.5. AHB_DMA_OUT_INT_RAW_CH n _REG (n : 0-2) (0x0030+0x10* n)

Register 0x00000000: AHB DMA OUT_DONE

Bit 31: (reserved)

Bit 7: AHB_DMA_OUT_DONE_CHn_INT_RAW

Bit 6: AHB_DMA_OUT_DONE_CHn_INT_RAW

Bit 5: AHB_DMA_OUT_DONE_CHn_INT_RAW

Bit 4: AHB_DMA_OUT_DONE_CHn_INT_RAW

Bit 3: AHB_DMA_OUT_DONE_CHn_INT_RAW

Bit 2: AHB_DMA_OUT_DONE_CHn_INT_RAW

Bit 1: AHB_DMA_OUT_DONE_CHn_INT_RAW

Bit 0: AHB_DMA_OUT_DONE_CHn_INT_RAW

Reset: 0

AHB_DMA_OUT_DONE_CH n _INT_RAW The raw interrupt status of AHB DMA OUT DONE CH n INT. (R/WTC/SS)

AHB_DMA_OUT_EOF_CH*n***_INT_RAW** The raw interrupt status of AHB_DMA_OUT_EOF_CH*n*_INT.
(R/WTC/SS)

AHB_DMA_OUT_DSCR_ERR_CH_n_INT_RAW The raw interrupt status of AHB_DMA_OUT_DSCR_ERR_CH_n INT. (R/WTC/SS)

AHB_DMA_OUT_TOTAL_EOF_CH*n***_INT_RAW** The raw interrupt status of AHB_DMA_OUT_TOTAL_EOF_CH*n*_INT. (R/WTC/SS)

AHB_DMA_OUTFIFO_OVF_CH n _INT_RAW The raw interrupt status of AHB_DMA_OUTFIFO_OVF_CH n _INT. (R/WTC/SS)

AHB_DMA_OUTFIFO_UDF_CH_n_INT_RAW The raw interrupt status of
AHB DMA_OUTFIFO_UDF_CH_n_INT. (R/WTC/SS)

AHB_DMA_OUT_RESP_ERR_CH*n*_INT_RAW The raw interrupt status of AHB_DMA_OUT_RESP_ERR_CH*n*_INT. (R/WTC/SS)

Register 4.6. AHB_DMA_OUT_INT_ST_CH n _REG (n : 0-2) (0x0034+0x10* n)

(reserved)																															AHB_DMA_OUT_RESP_ERR_CH _n _INT_ST AHB_DMA_OUT_FIFO_UDF_CH _n _INT_ST AHB_DMA_OUT_FIFO_OVF_CH _n _INT_ST AHB_DMA_OUT_TOTAL_EOF_CH _n _INT_ST AHB_DMA_OUT_DSCR_ERR_CH _n _INT_ST AHB_DMA_OUT_DONE_CH _n _INT_ST															
31																															7	6	5	4	3	2	1	0								
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset													

AHB_DMA_OUT_DONE_CH n _INT_ST The masked interrupt status of AHB_DMA_OUT_DONE_CH n _INT. (RO)

AHB_DMA_OUT_EOF_CH n _INT_ST The masked interrupt status of AHB_DMA_OUT_EOF_CH n _INT. (RO)

AHB_DMA_OUT_DSCR_ERR_CH n _INT_ST The masked interrupt status of AHB_DMA_OUT_DSCR_ERR_CH n _INT. (RO)

AHB_DMA_OUT_TOTAL_EOF_CH n _INT_ST The masked interrupt status of AHB_DMA_OUT_TOTAL_EOF_CH n _INT. (RO)

AHB_DMA_OUT_FIFO_OVF_CH n _INT_ST The masked interrupt status of AHB_DMA_OUT_FIFO_OVF_CH n _INT. (RO)

AHB_DMA_OUT_FIFO_UDF_CH n _INT_ST The masked interrupt status of AHB_DMA_OUT_FIFO_UDF_CH n _INT. (RO)

AHB_DMA_OUT_RESP_ERR_CH n _INT_ST The masked interrupt status of AHB_DMA_OUT_RESP_ERR_CH n _INT. (RO)

Register 4.7. AHB_DMA_OUT_INT_ENA_CH n _REG (n : 0-2) (0x0038+0x10* n)

AHB_DMA_OUT_DONE_CH n _INT_ENA Write 1 to enable AHB_DMA_OUT_DONE_CH n _INT. (R/W)

AHB_DMA_OUT_EOF_CH_n_INT_ENA Write 1 to enable AHB_DMA_OUT_EOF_CH_n_INT. (R/W)

AHB_DMA_OUT_DSCR_ERR_CH n _INT_ENA Write 1 to enable AHB_DMA_OUT_DSCR_ERR_CH n _INT.
(R/W)

AHB_DMA_OUT_TOTAL_EOF_CH n _INT_ENA Write 1 to enable AHB_DMA_OUT_TOTAL_EOF_CH n _INT.
(R/W)

AHB_DMA_OUTFIFO_OVF_CH**n****_INT_ENA** Write 1 to enable AHB_DMA_OUTFIFO_OVF_CH**n**_INT.
(R/W)

AHB_DMA_OUTFIFO_UDF_CH n _INT_ENA Write 1 to enable AHB_DMA_OUTFIFO_UDF_CH n _INT.
(R/W)

AHB_DMA_OUT_RESP_ERR_CH*n***_INT_ENA** Write 1 to enable AHB_DMA_OUT_RESP_ERR_CH*n*_INT.
(R/W)

Register 4.8. AHB_DMA_OUT_INT_CLR_CH n _REG (n : 0-2) (0x003C+0x10* n)

(reserved)																																AHB_DMA_OUT_RESP_ERR_CH _n _INT_CLR AHB_DMA_OUTFIFO_UDF_CH _n _INT_CLR AHB_DMA_OUTFIFO_OVF_CH _n _INT_CLR AHB_DMA_OUT_TOTAL_EOF_CH _n _INT_CLR AHB_DMA_OUT_DSCR_ERR_CH _n _INT_CLR AHB_DMA_OUT_EOF_CH _n _INT_CLR AHB_DMA_OUT_DONE_CH _n _INT_CLR								
31																															7	6	5	4	3	2	1	0		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset					

AHB_DMA_OUT_DONE_CH n _INT_CLR Write 1 to clear AHB_DMA_OUT_DONE_CH n _INT. (WT)

AHB_DMA_OUT_EOF_CH n _INT_CLR Write 1 to clear AHB_DMA_OUT_EOF_CH n _INT. (WT)

AHB_DMA_OUT_DSCR_ERR_CH n _INT_CLR Write 1 to clear AHB_DMA_OUT_DSCR_ERR_CH n _INT.
(WT)

AHB_DMA_OUT_TOTAL_EOF_CH n _INT_CLR Write 1 to clear AHB_DMA_OUT_TOTAL_EOF_CH n _INT.
(WT)

AHB_DMA_OUTFIFO_OVF_CH n _INT_CLR Write 1 to clear AHB_DMA_OUTFIFO_OVF_CH n _INT. (WT)

AHB_DMA_OUTFIFO_UDF_CH n _INT_CLR Write 1 to clear AHB_DMA_OUTFIFO_UDF_CH n _INT. (WT)

AHB_DMA_OUT_RESP_ERR_CH n _INT_CLR Write 1 to clear AHB_DMA_OUT_RESP_ERR_CH n _INT.
(WT)

Register 4.9. AHB_DMA_AHB_TEST_REG (0x0060)

(reserved)																												AHB_DMA_AHB_TESTADDR (reserved) AHB_DMA_AHB_TESTMODE						
31																												6	5	4	3	2	0	
0 0																												0	0	0		Reset		

AHB_DMA_AHB_TESTMODE Reserved. (R/W)

AHB_DMA_AHB_TESTADDR Reserved. (R/W)

(reserved)

AHB_DMA_CLK_EN
AHB_DMA_ARB_PRI_DIS
(reserved)
AHB_DMA_AHBM_RST_INTER

(R/W)

(R/W)

Register 4.11. AHB_DMA_IN_CONFO_CH n _REG (n : 0-2) (0x0070+0xC0* n)

(reserved)																															AHB_DMA_IN_ETM_EN_CHn AHB_DMA_IN_ETM_EN_CHn AHB_DMA_IN_ETM_EN_CHn AHB_DMA_IN_ETM_EN_CHn AHB_DMA_IN_ETM_EN_CHn AHB_DMA_IN_ETM_EN_CHn							
31																															6	5	4	3	2	1	0	Reset
0 0																															0	0	0	0	0	0	0	

AHB_DMA_IN_RST_CH n Write 1 and then 0 to reset RX channel n FSM and RX FIFO pointer.(R/W)

AHB_DMA_IN_LOOP_TEST_CH_n Reserved. (R/W)

AHB_DMA_INDSR_BURST_EN_CH*n* Configures whether to enable INCR burst transfer for RX channel *n* to read descriptors.

0: Disable

1: Enable

(R/W)

AHB_DMA_IN_DATA_BURST_EN_CH n Configures whether to enable INCR burst transfer for RX channel n .

0: Disable

1: Enable

(R/W)

AHB_DMA_MEM_TRANS_EN_CH n ($n = 1 \sim 2$) Configures whether to enable memory-to-memory data transfer. Channel 0 does not support this feature.

0: Disable

1: Enable

(R/W)

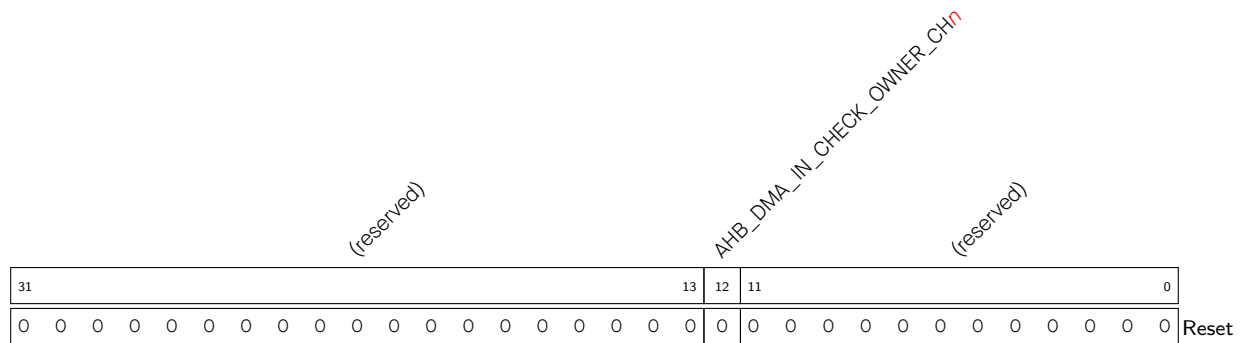
AHB_DMA_IN_ETM_EN_CH n Configures whether to enable ETM control for RX channel n .

0: Disable

1: Enable

(R/W)

Register 4.12. AHB_DMA_IN_CONF1_CH n _REG (n : 0-2) (0x0074+0xC0* n)



AHB_DMA_IN_CHECK_OWNER_CHn Configures whether to enable owner bit check for RX channel

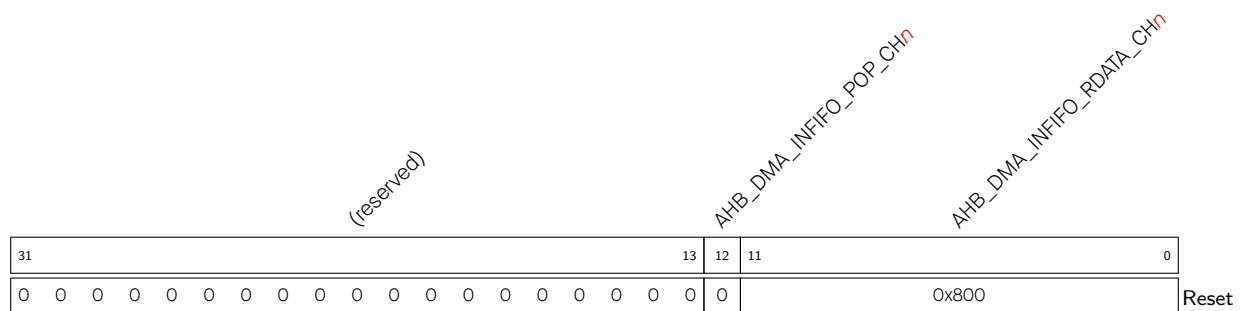
 $n.$

0: Disable

1: Enable

(R/W)

Register 4.13. AHB_DMA_IN_POP_CH n _REG (n : 0-2) (0x007C+0xC0* n)



AHB_DMA_INFIFO_RDATA_CH_n Represents the data popped from AHB DMA RX FIFO. (RO)

AHB_DMA_INFIFO_POP_CH n Configures whether to pop data from AHB DMA RX FIFO.

0: Invalid. No effect

1: Pop

(WT)

Register 4.15. AHB_DMA_OUT_CONFO_CH n _REG (n : 0-2) (0x00D0+0xC0* n)

(reserved)																												AHB_DMA_OUT_ETM_EN_CH _n								AHB_DMA_OUT_DATA_BURST_EN_CH _n								AHB_DMA_OUTDSCR_BURST_EN_CH _n								AHB_DMA_OUT_EOF_MODE_CH _n								AHB_DMA_OUT_AUTO_WRRBACK_CH _n								AHB_DMA_OUT_LOOP_TEST_CH _n								AHB_DMA_OUT_RST_CH _n							
31																												7		6		5		4		3		2		1		0		Reset																																							
0																												0		0		0		0		1		0		0		0																																									

AHB_DMA_OUT_RST_CH n Configures the reset state of TX channel n FSM and TX FIFO pointer.

0: Release reset

1: Reset

(R/W)

AHB_DMA_OUT_LOOP_TEST_CH n Reserved. (R/W)

AHB_DMA_OUT_AUTO_WRBK_CH n Configures whether to enable automatic outlink write-back when all the data in TX FIFO has been transmitted.

0: Disable

1: Enable

(R/W)

AHB_DMA_OUT_EOF_MODE_CH n Configures when to generate EOF flag.

0: EOF flag for TX channel n is generated when data to be transmitted has been pushed into FIFO in AHB DMA.

1: EOF flag for TX channel n is generated when data to be transmitted has been popped from FIFO in AHB DMA.

(R/W)

AHB_DMA_OUTDSCR_BURST_EN_CH n Configures whether to enable INCR burst transfer for TX channel n reading descriptors.

0: Disable

1: Enable

(R/W)

AHB_DMA_OUT_DATA_BURST_EN_CH n Configures whether to enable INCR burst transfer for TX channel n .

0: Disable

1: Enable

(R/W)

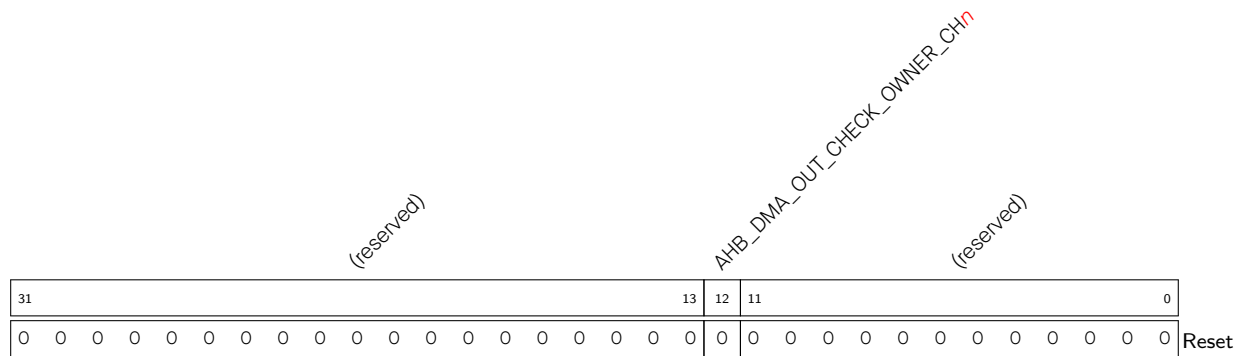
AHB_DMA_OUT_ETM_EN_CH n Configures whether to enable ETM control for TX channel n .

0: Disable

1: Enable

(R/W)

Register 4.16. AHB_DMA_OUT_CONF1_CH n _REG (n : 0-2) (0x00D4+0xC0* n)



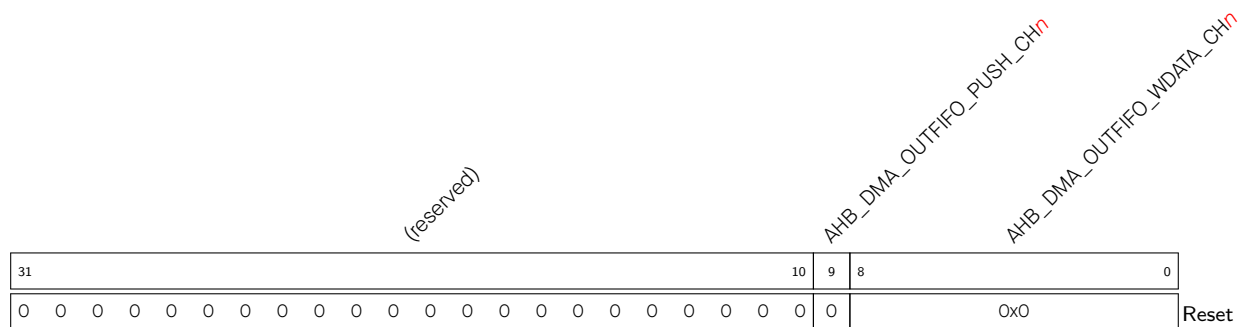
AHB_DMA_OUT_CHECK_OWNER_CH*n* Configures whether to enable owner bit check for TX channel *n*.

0: Disable

1: Enable

(R/W)

Register 4.17. AHB_DMA_OUT_PUSH_CH n _REG (n : 0-2) (0x00DC+0xC0* n)



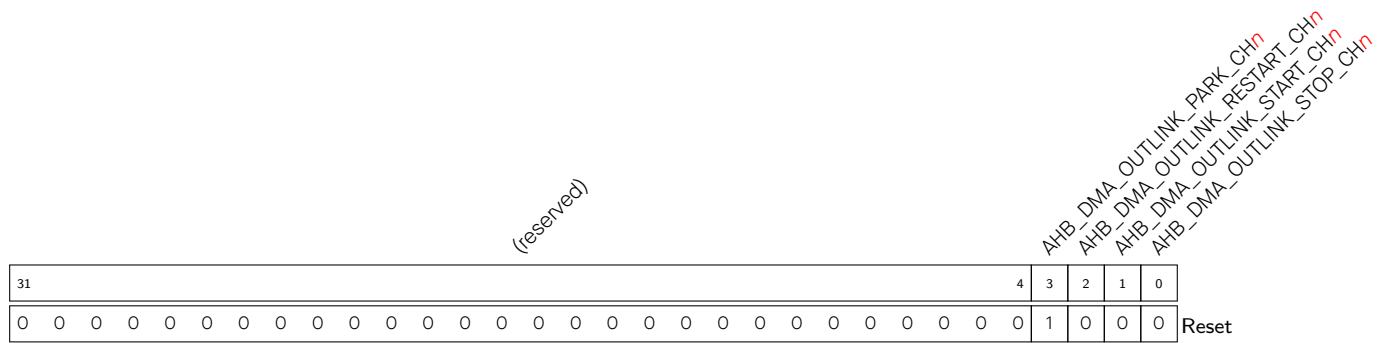
AHB_DMA_OUTFIFO_WDATA_CH n Represents the data that need to be pushed into AHB DMA TX FIFO. (R/W)

AHB_DMA_OUTFIFO_PUSH_CH_n Configures whether to push data into AHB DMA TX FIFO.

0: Invalid. No effect

1: Push

(WT)

Register 4.18. AHB_DMA_OUT_LINK_CH n _REG (n : 0-2) (0x00E0+0xC0* n)

AHB_DMA_OUTLINK_STOP_CH n Configures whether to stop TX channel n from transmitting data.

0: Invalid. No effect

1: Stop

(WT)

AHB_DMA_OUTLINK_START_CH n Configures whether to enable TX channel n for data transfer.

0: Disable

1: Enable

(WT)

AHB_DMA_OUTLINK_RESTART_CH n Configures whether to restart TX channel n for AHB DMA transfer.

0: Invalid. No effect

1: Restart

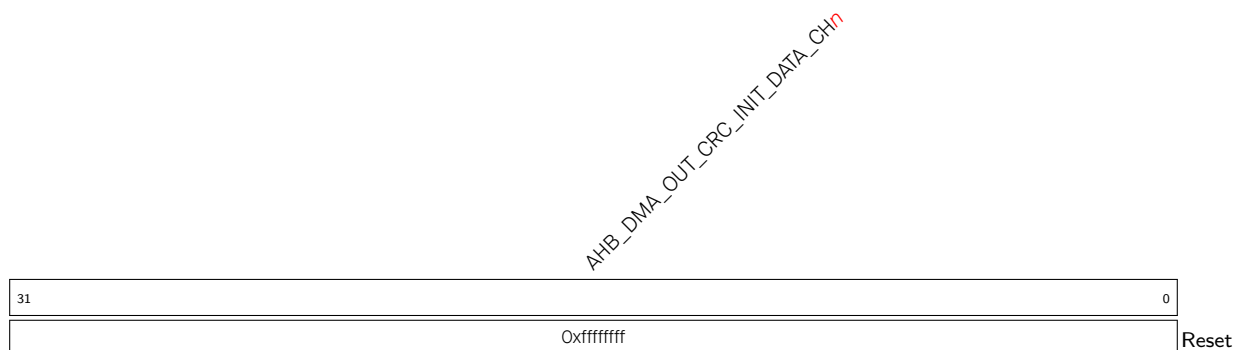
(WT)

AHB_DMA_OUTLINK_PARK_CH n Represents the status of the transmit descriptor's FSM.

0: Running

1: Idle

(RO)

Register 4.19. AHB_DMA_OUT_CRC_INIT_DATA_CH n _REG (n : 0-2) (0x02BC+0x28* n)

AHB_DMA_OUT_CRC_INIT_DATA_CH n Configures the CRC initial value for TX channel n . (R/W)

Register 4.20. AHB_DMA_TX_CRC_WIDTH_CH n _REG (n : 0-2) (0x02C0+0x28* n)

[illegible]

AHB_DMA_TX_CRC_WIDTH_CH n Configures the CRC result width for TX channel n . 0: ≤ 8 bits

1: ≤ 16 bits

2: ≤ 24 bits

3: ≤ 32 bits

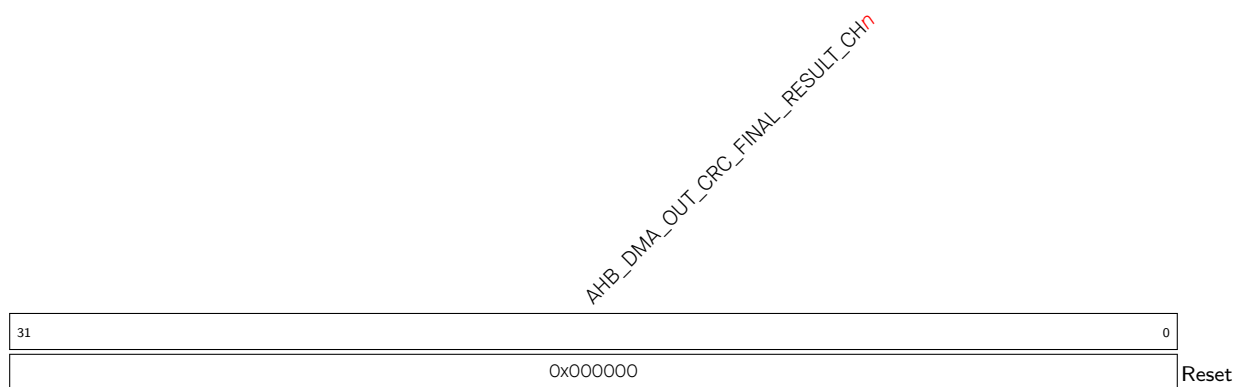
(R/W)

AHB_DMA_TX_CRC_LAUTCH_FLGA_CH n Write 1 and then 0 to latch the values of
 AHB_DMA_TX_CRC_EN_ADDR_CH n , AHB_DMA_TX_CRC_DATA_EN_ADDR_CH n ,
 AHB_DMA_TX_CRC_EN_WR_DATA_CH n , and AHB_DMA_TX_CRC_DATA_EN_WR_DATA_CH n .
 (R/W)

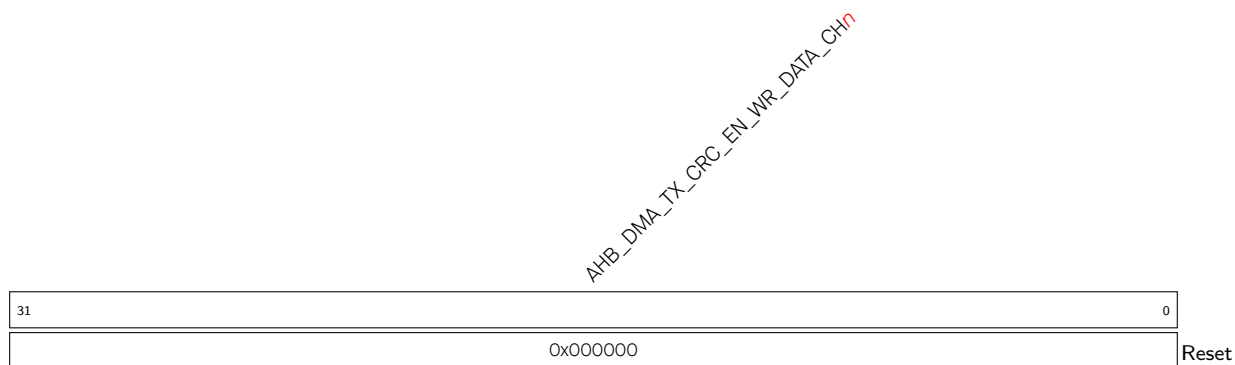
Register 4.21. AHB_DMA_OUT_CRC_CLEAR_CH n _REG (n : 0-2) (0x02C4+0x28* n)

Diagram illustrating the AHB-DMA Control Register (AHB_DMA_CTL) structure. The register is 32 bits wide, with bit 31 labeled '1' and bit 0 labeled '0'. The register is divided into three sections: a 1-bit field for 'AHB_DMA_CTL[31]', a 30-bit field for 'AHB_DMA_CTL[30:1]' (labeled '(reserved)'), and a 1-bit field for 'AHB_DMA_CTL[0]'.

AHB_DMA_OUT_CRC_CLEAR_CH_n_REG Reserved. (R/W)

Register 4.22. AHB_DMA_OUT_CRC_FINAL_RESULT_CH n _REG (n : 0-2) (0x02C8+0x28* n)

AHB_DMA_OUT_CRC_FINAL_RESULT_CH n Represents the CRC result for TX channel n . (RO)

Register 4.23. AHB_DMA_TX_CRC_EN_WR_DATA_CH n _REG (n : 0-2) (0x02CC+0x28* n)

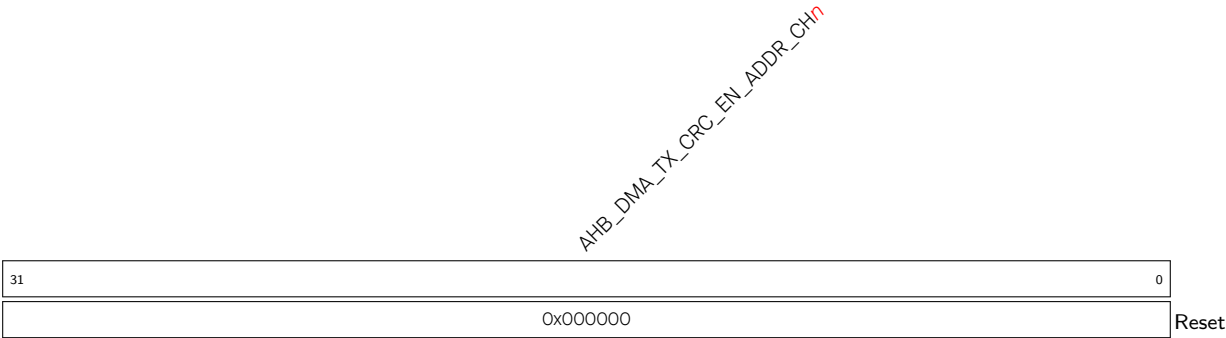
AHB_DMA_TX_CRC_EN_WR_DATA_CH n Configures whether to include each bit of the intermediate result in the CRC calculation matrix for TX channel n .

0: Not include

1: Include

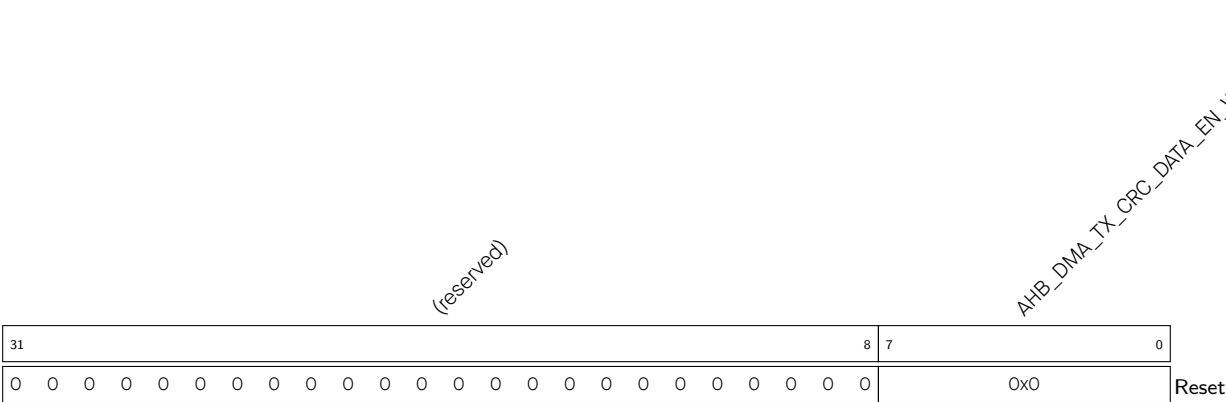
(R/W)

Register 4.24. AHB_DMA_TX_CRC_EN_ADDR_CH n _REG (n : 0-2) (0x02D0+0x28* n)



AHB_DMA_TX_CRC_EN_ADDR_CH n Configures at which bit of the CRC result the AHB_DMA_TX_CRC_EN_WR_DATA_CH n _REG register targets for TX channel n . (R/W)

Register 4.25. AHB_DMA_TX_CRC_DATA_EN_WR_DATA_CH n _REG (n : 0-2) (0x02D4+0x28* n)

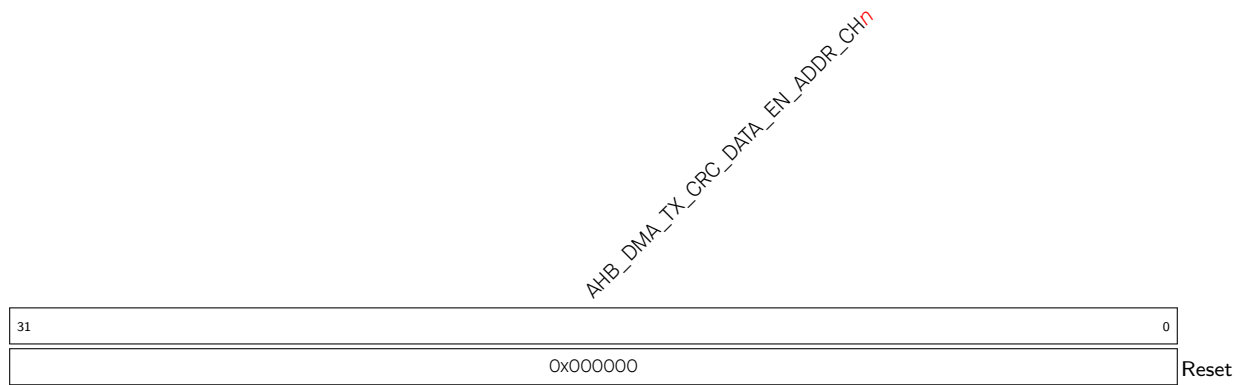


AHB_DMA_TX_CRC_DATA_EN_WR_DATA_CH n Configures whether to include each bit of the data in the CRC calculation matrix for TX channel n .

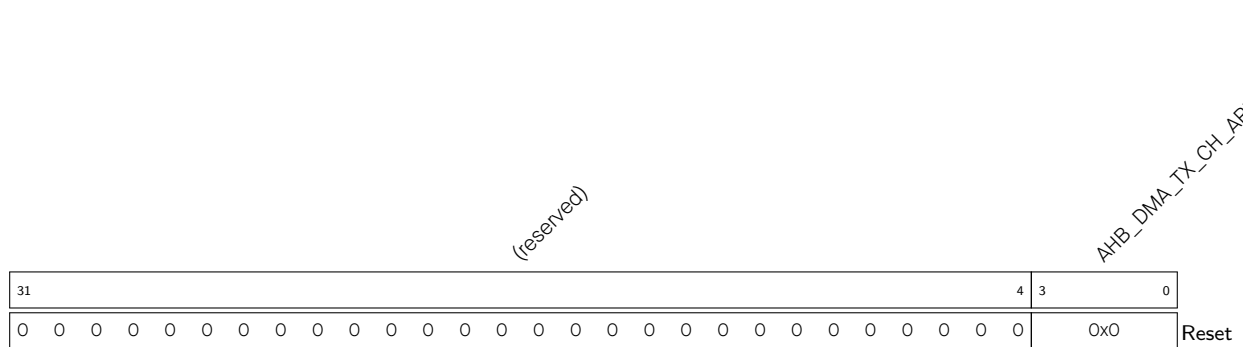
0: Not include

1: Include

(R/W)

Register 4.26. AHB_DMA_TX_CRC_DATA_EN_ADDR_CH n _REG (n : 0-2) (0x02D8+0x28* n)

AHB_DMA_TX_CRC_DATA_EN_ADDR_CH n Configures at which bit of the CRC result the AHB_DMA_TX_CRC_DATA_EN_WR_DATA_CH n _REG register targets for TX channel n . (R/W)

Register 4.27. AHB_DMA_TX_CH_ARB_WEIGHT_CH n _REG (n : 0-2) (0x02DC+0x28* n)

AHB_DMA_TX_CH_ARB_WEIGHT_CH n Configures the weight (i.e the number of tokens) of TX channel n .

Value range: 0 ~ 15. (R/W)

Register 4.28. AHB_DMA_TX_ARB_WEIGHT_OPT_DIR_CH*n*_REG (*n*: 0-2) (0x02E0+0x28**n*)

[illegible]

AHB_DMA_TX_ARB_WEIGHT_OPT_DIR_CH*n* Configures whether to enable weight optimization for TX channel *n*.

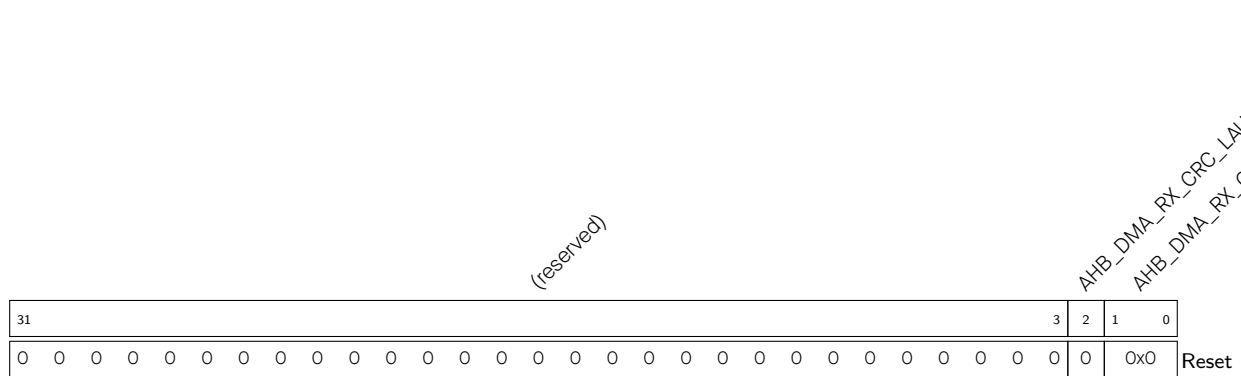
0: Enable

1: Disable (R/W)

Register 4.29. AHB_DMA_IN_CRC_INIT_DATA_CH n _REG (n : 0-2) (0x0334+0x28* n)

Diagram of the AHB-DMA_IN_CRC_INIT_DATA_CH1n register. The register is 32 bits wide, with bit 31 on the left and bit 0 on the right. The value 0xffffffff is shown in the register. The label AHB_DMA_IN_CRC_INIT_DATA_CH1n is written diagonally across the register.

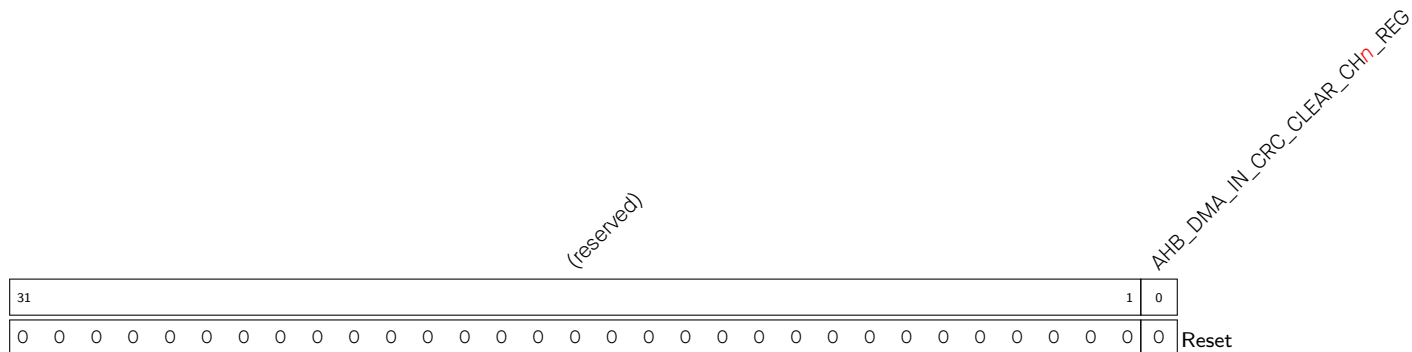
AHB_DMA_IN_CRC_INIT_DATA_CH n Configures the CRC initial value for RX channel n . (R/W)

Register 4.30. AHB_DMA_RX_CRC_WIDTH_CH n _REG (n : 0-2) (0x0338+0x28* n)

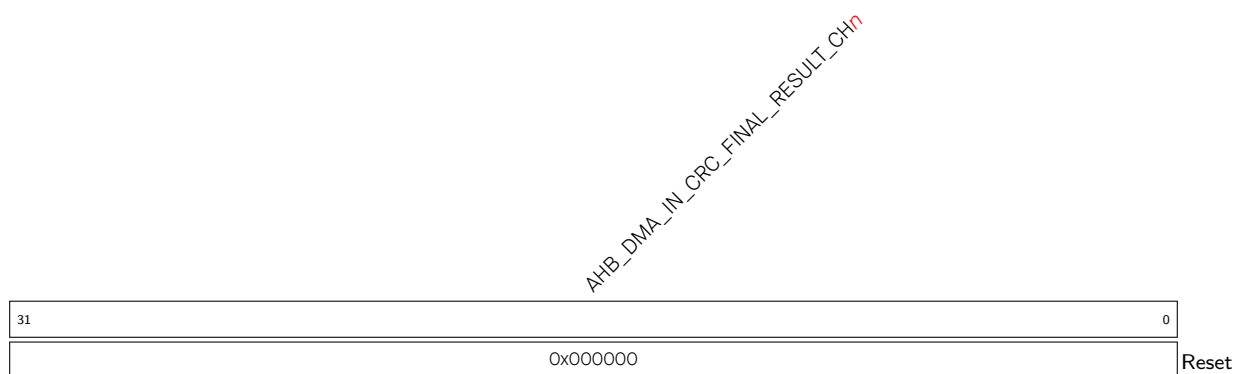
AHB_DMA_RX_CRC_WIDTH_CH n Configures the CRC result width for RX channel n .

- 0: $\leq$ 8 bits
- 1: $\leq$ 16 bits
- 2: $\leq$ 24 bits
- 3: $\leq$ 32 bits
- (R/W)

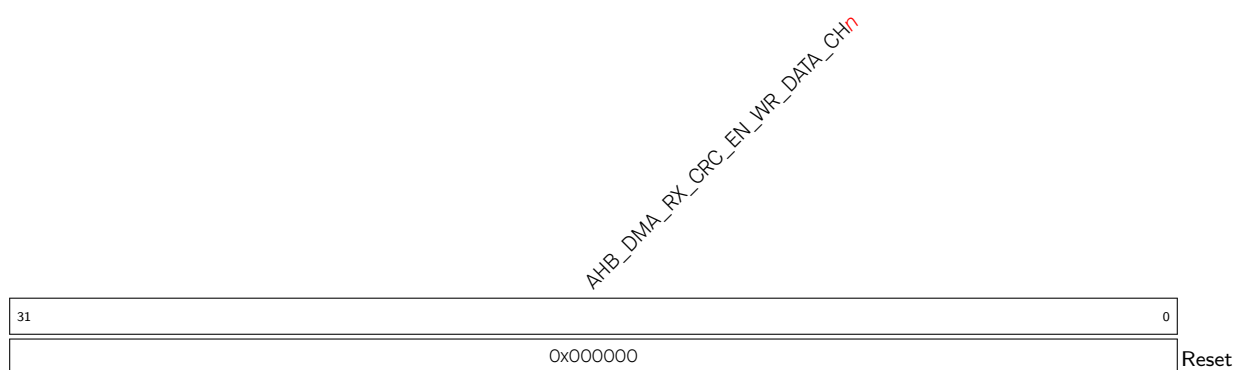
AHB_DMA_RX_CRC_LAUTCH_FLGA_CH n Write 1 and then 0 to latch the values of AHB_DMA_RX_CRC_EN_ADDR_CH n , AHB_DMA_RX_CRC_DATA_EN_ADDR_CH n , AHB_DMA_RX_CRC_EN_WR_DATA_CH n , and AHB_DMA_RX_CRC_DATA_EN_WR_DATA_CH n .
(R/W)

Register 4.31. AHB_DMA_IN_CRC_CLEAR_CH n _REG (n : 0-2) (0x033C+0x28* n)

AHB_DMA_IN_CRC_CLEAR_CH n _REG Reserved. (R/W)

Register 4.32. AHB_DMA_IN_CRC_FINAL_RESULT_CH n _REG (n : 0-2) (0x0340+0x28* n)

AHB_DMA_IN_CRC_FINAL_RESULT_CH n Represents the CRC result for RX channel n . (RO)

Register 4.33. AHB_DMA_RX_CRC_EN_WR_DATA_CH n _REG (n : 0-2) (0x0344+0x28* n)

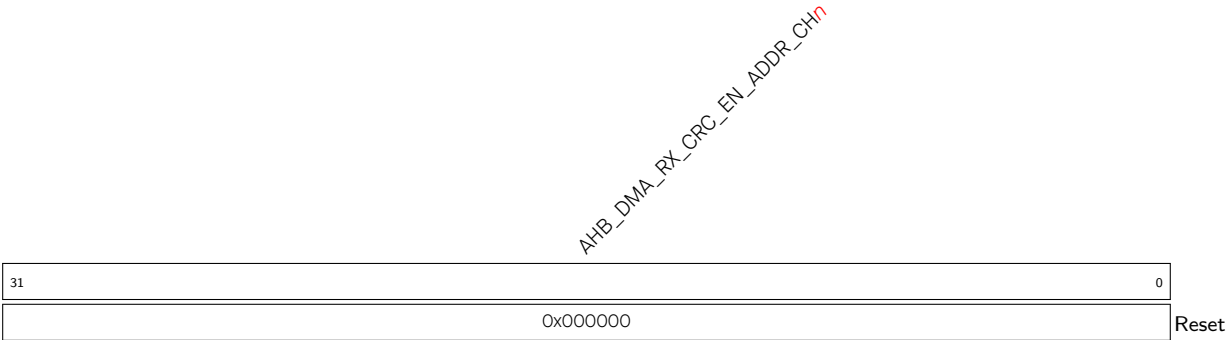
AHB_DMA_RX_CRC_EN_WR_DATA_CH n Configures whether to include each bit of the intermediate result in the CRC calculation matrix for RX channel n .

0: Not include

1: Include

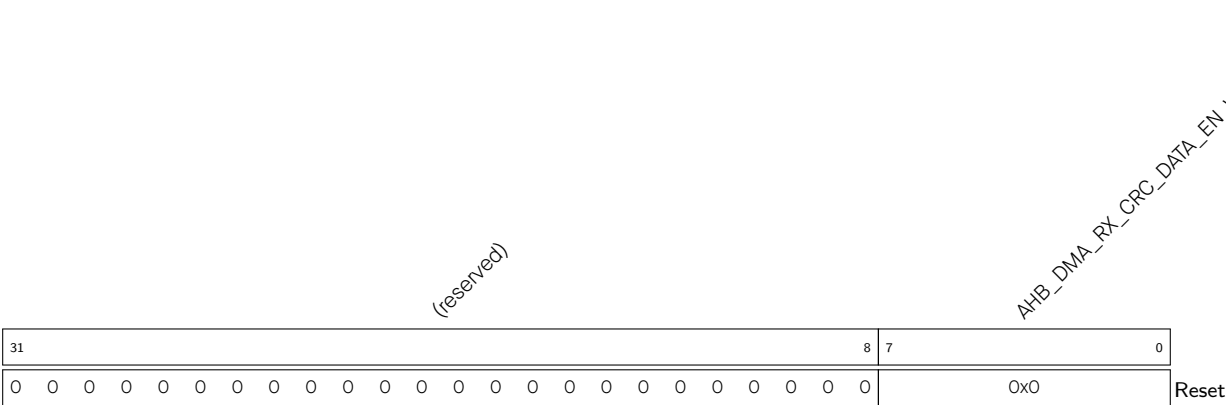
(R/W)

Register 4.34. AHB_DMA_RX_CRC_EN_ADDR_CH n _REG (n : 0-2) (0x0348+0x28* n)



AHB_DMA_RX_CRC_EN_ADDR_CH n Configures at which bit of the CRC result the AHB_DMA_RX_CRC_EN_WR_DATA_CH n _REG register targets for RX channel n . (R/W)

Register 4.35. AHB_DMA_RX_CRC_DATA_EN_WR_DATA_CH n _REG (n : 0-2) (0x034C+0x28* n)



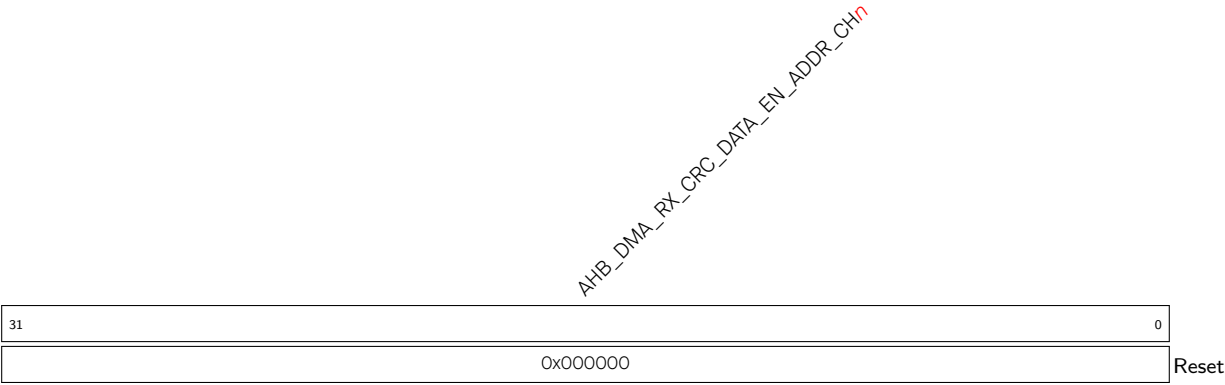
AHB_DMA_RX_CRC_DATA_EN_WR_DATA_CH n Configures whether to include each bit of the data in the CRC calculation matrix for RX channel n .

0: Not include

1: Include

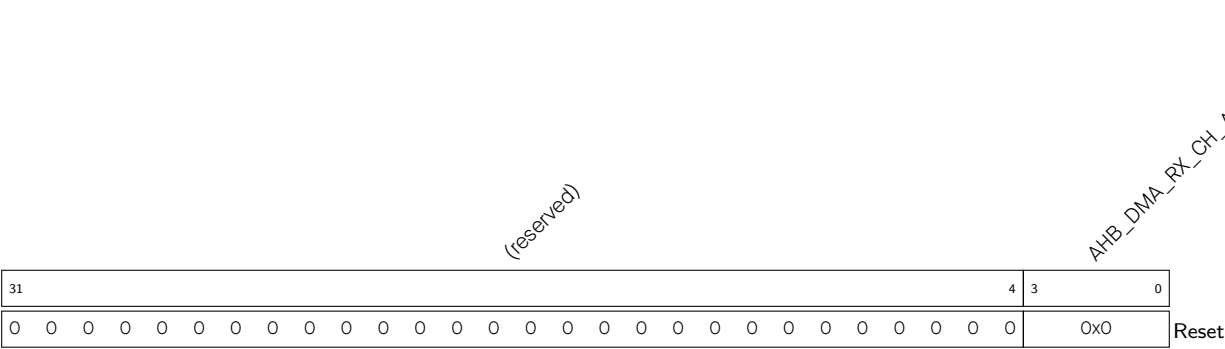
(R/W)

Register 4.36. AHB_DMA_RX_CRC_DATA_EN_ADDR_CH n _REG (n : 0-2) (0x0350+0x28* n)

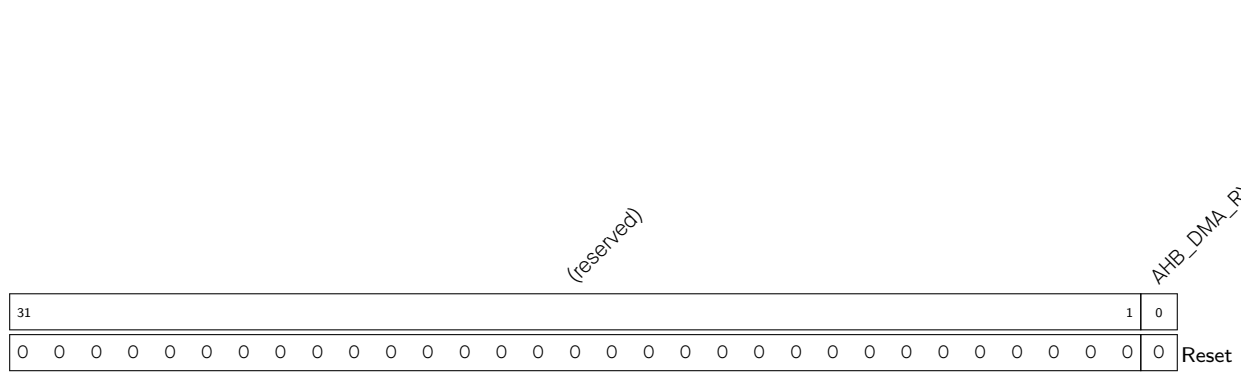


AHB_DMA_RX_CRC_DATA_EN_ADDR_CH n Configures at which bit of the CRC result the AHB_DMA_RX_CRC_DATA_EN_WR_DATA_CH n _REG register targets for RX channel n . (R/W)

Register 4.37. AHB_DMA_RX_CH_ARB_WEIGHT_CH n _REG (n : 0-2) (0x0354+0x28* n)



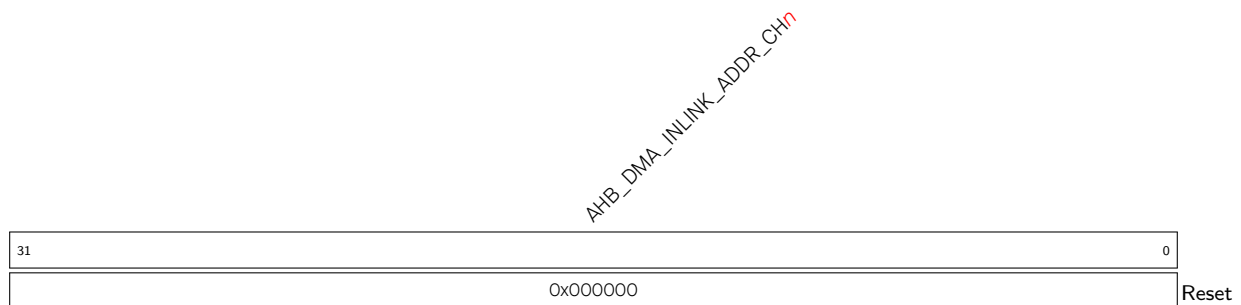
AHB_DMA_RX_CH_ARB_WEIGHT_CH n Configures the weight (i.e the number of tokens) of RX channel n .
Value range: 0 ~ 15. (R/W)

Register 4.38. AHB_DMA_RX_ARB_WEIGH_OPT_DIR_CH n _REG (n : 0-2) (0x0358+0x28* n)

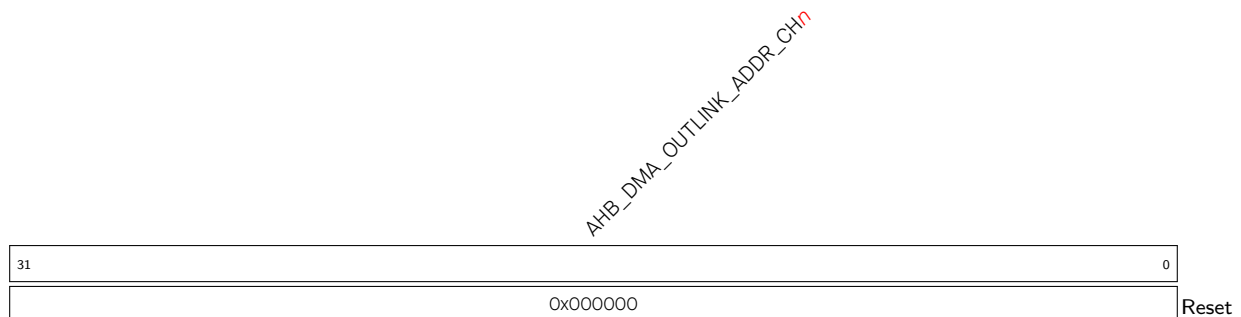
AHB_DMA_RX_ARB_WEIGH_OPT_DIR_CH n Configures whether to enable weight optimization for RX channel n .

0: Enable

1: Disable (R/W)

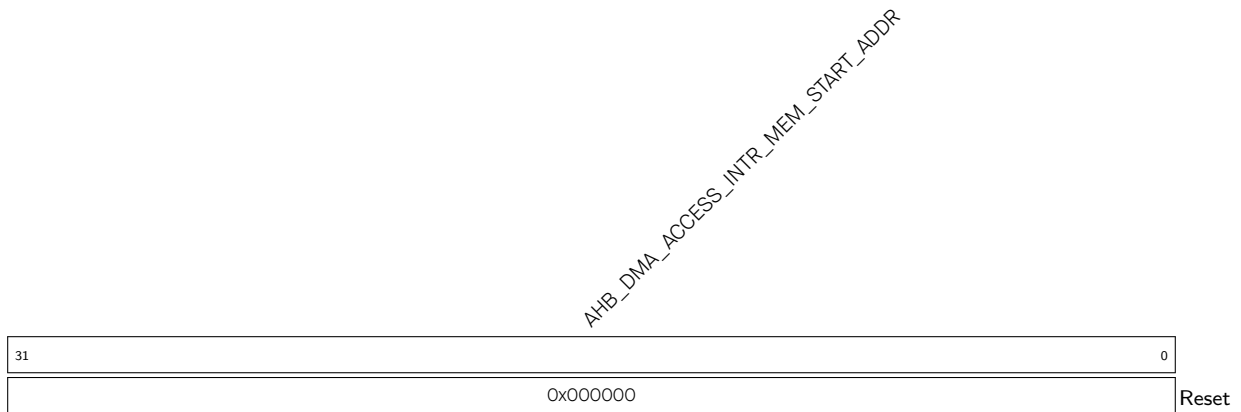
Register 4.39. AHB_DMA_IN_LINK_ADDR_CH n _REG (n : 0-2) (0x03AC+0x4* n)

AHB_DMA_INLINK_ADDR_CH n Represents the first receive descriptor's address. (R/W)

Register 4.40. AHB_DMA_OUT_LINK_ADDR_CH n _REG (n : 0-2) (0x03B8+0x4* n)

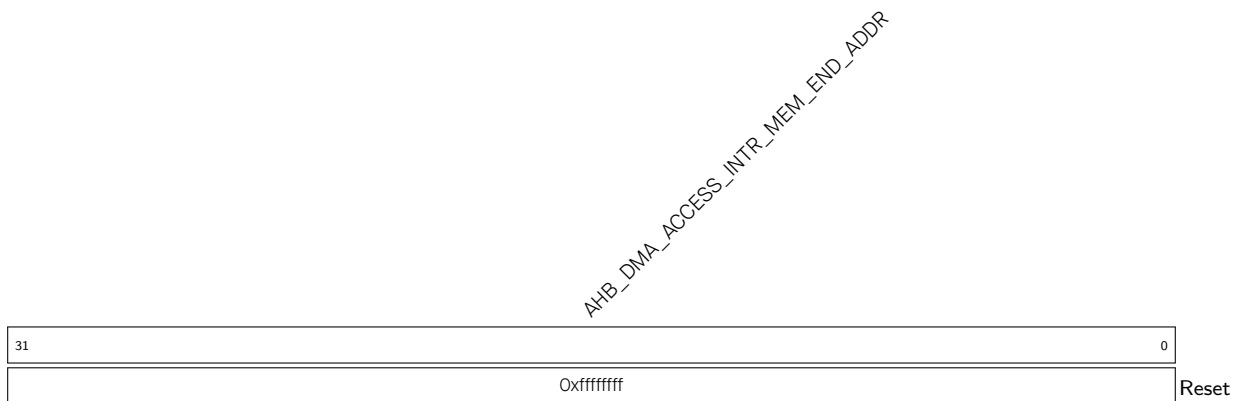
AHB_DMA_OUTLINK_ADDR_CH n Represents the first transmit descriptor's address. (R/W)

Register 4.41. AHB_DMA_INTR_MEM_START_ADDR_REG (0x03C4)



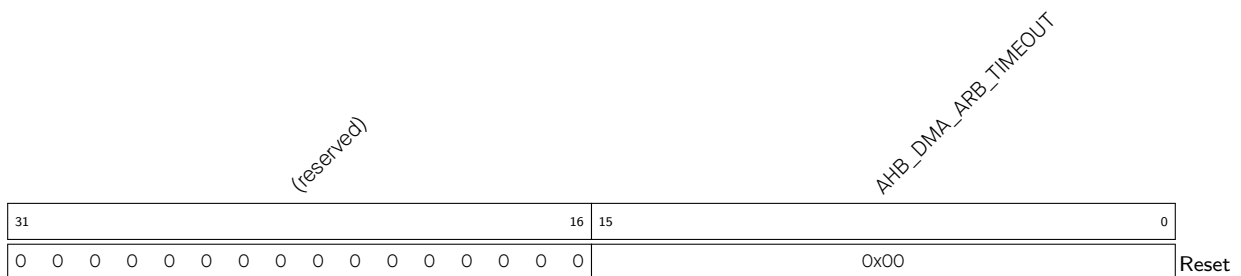
AHB_DMA_ACCESS_INTR_MEM_START_ADDR Configures the start address of accessible address space. (R/W)

Register 4.42. AHB_DMA_INTR_MEM_END_ADDR_REG (0x03C8)



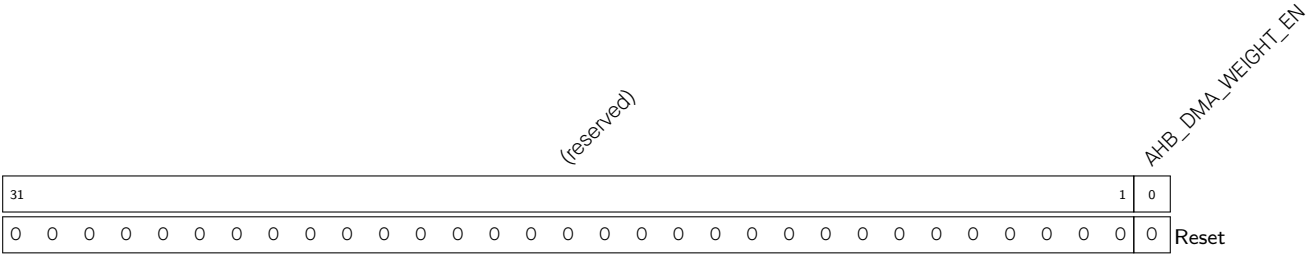
AHB_DMA_ACCESS_INTR_MEM_END_ADDR Configures the end address of accessible address space. (R/W)

Register 4.43. AHB_DMA_ARB_TIMEOUT_REG (0x03DC)



AHB_DMA_ARB_TIMEOUT Configures the time slot of weight arbitration. Measurement unit: AHB bus clock cycle. (R/W)

Register 4.44. AHB_DMA_WEIGHT_EN_REG (0x0400)

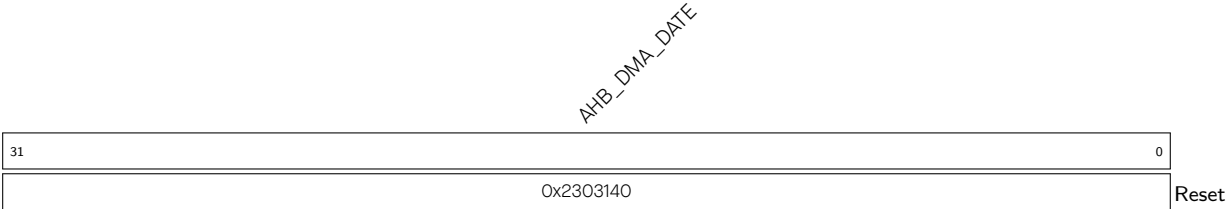


AHB_DMA_WEIGHT_EN Configures whether to enable weight arbitration.

- 0: Disable
- 1: Enable

(R/W)

Register 4.45. AHB_DMA_DATE_REG (0x0068)



AHB_DMA_DATE Version control register. (R/W)

Register 4.46. AHB_DMA_INFIFO_STATUS_CH n _REG (n : 0-2) (0x0078+0xC0* n)

[illegible]

AHB_DMA_INFIFO_FULL_CH_n Represents whether L1 RX FIFO is full.

0: Not Full

1: Full

(RO)

AHB_DMA_INFIFO_EMPTY_CH_n Represents whether L1 RX FIFO is empty.

0: Not empty

1: Empty

(RO)

AHB_DMA_INFIFO_CNT_CH n Represents the number of data bytes in L1 RX FIFO for RX channel n .

(RO)

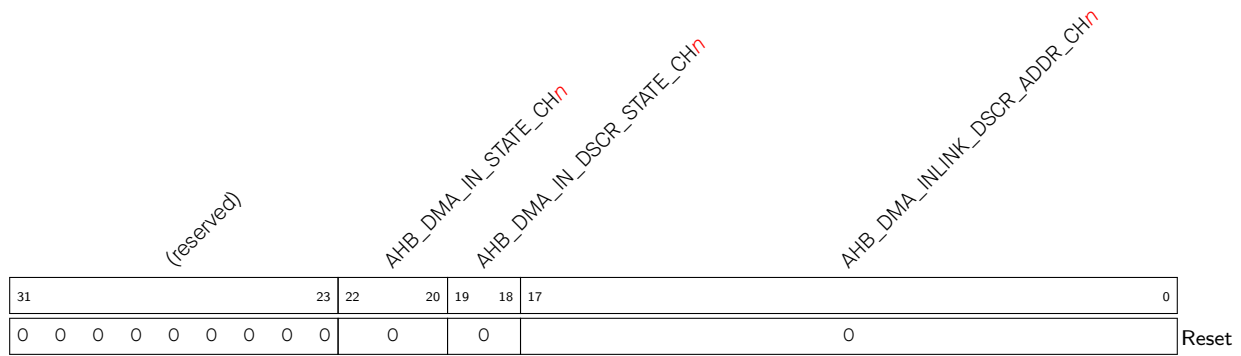
AHB_DMA_IN_REMAIN_UNDER_1B_CHn Reserved. (RO)

AHB_DMA_IN_REMAIN_UNDER_2B_CHn Reserved. (RO)

AHB_DMA_IN_REMAIN_UNDER_3B_CHn Reserved. (RO)

AHB_DMA_IN_REMAIN_UNDER_4B_CH_n Reserved. (RO)

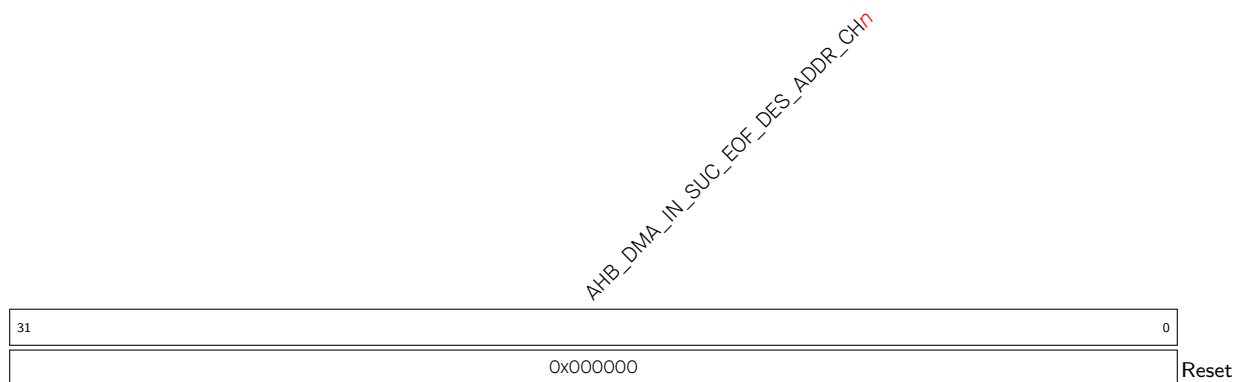
AHB_DMA_IN_BUF_HUNGRY_CH_n Reserved. (RO)

Register 4.47. AHB_DMA_IN_STATE_CH n _REG (n : 0-2) (0x0084+0xC0* n)

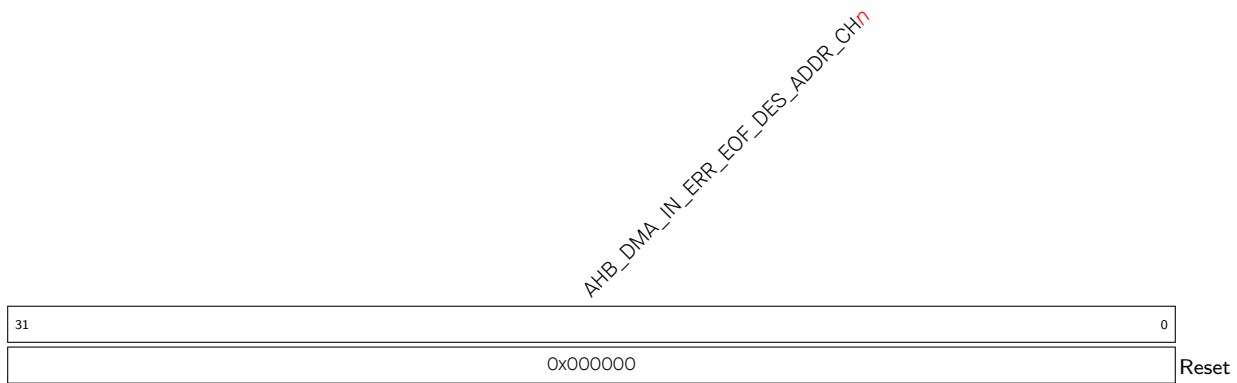
AHB_DMA_INLINK_DSCR_ADDR_CH n Represents the lower 18 bits of the next receive descriptor address that is pre-read (but not processed yet). If the current receive descriptor is the last descriptor, then this field represents the address of the current receive descriptor. (RO)

AHB_DMA_IN_DSCR_STATE_CH n Reserved. (RO)

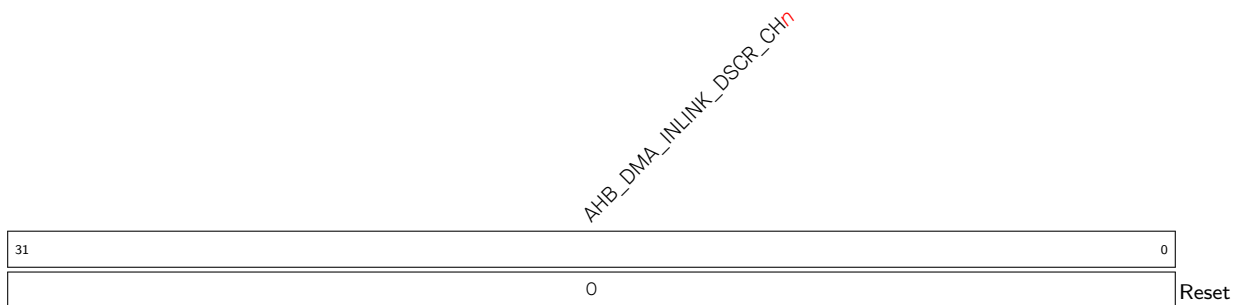
AHB_DMA_IN_STATE_CH n Reserved. (RO)

Register 4.48. AHB_DMA_IN_SUC_EOF_DES_ADDR_CH n _REG (n : 0-2) (0x0088+0xC0* n)

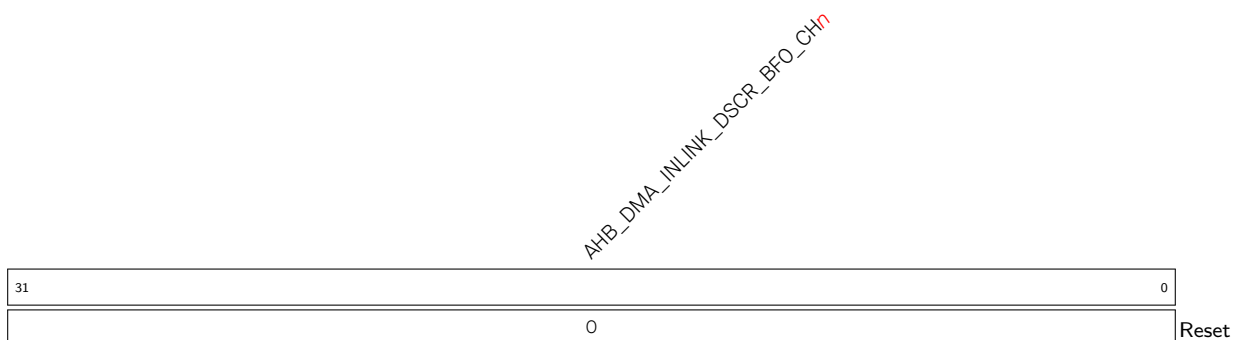
AHB_DMA_IN_SUC_EOF_DES_ADDR_CH n Represents the address of the receive descriptor when the EOF bit in this descriptor is 1. (RO)

Register 4.49. AHB_DMA_IN_ERR_EOF_DES_ADDR_CH_{*n*}_REG (*n*: 0-2) (0x008C+0xC0n*)**

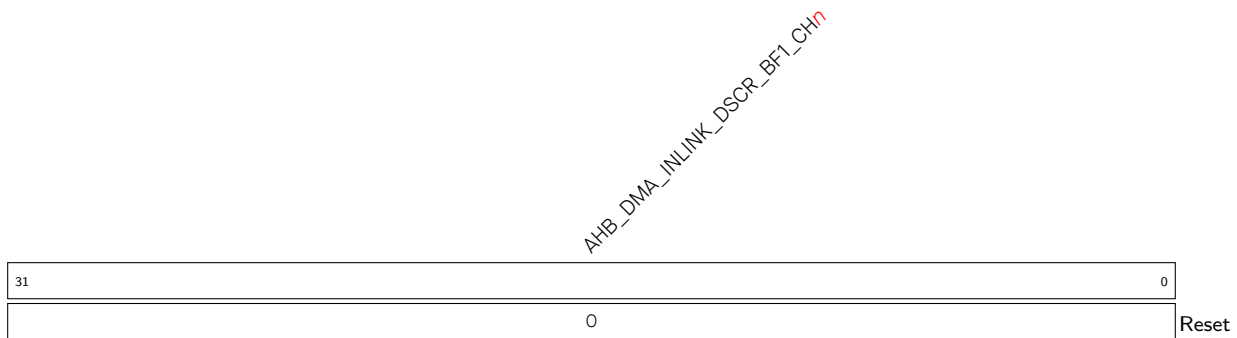
AHB_DMA_IN_ERR_EOF_DES_ADDR_CH_{*n*} Represents the address of the receive descriptor when there are some errors in the currently received data. Valid only for UHCI. (RO)

Register 4.50. AHB_DMA_INLINK_DSCR_CH_{*n*}_REG (*n*: 0-2) (0x0090+0xC0n*)**

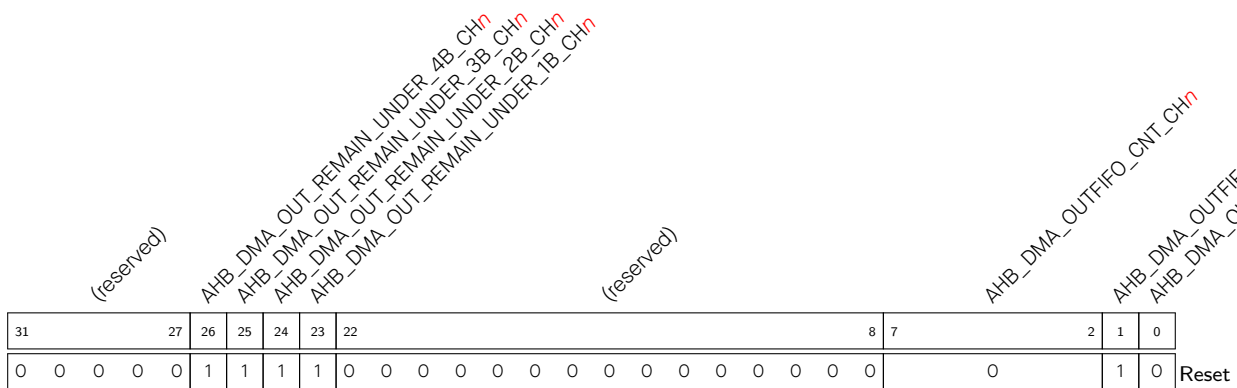
AHB_DMA_INLINK_DSCR_CH_{*n*} Represents the address of the next receive descriptor x+1 pointed by the current receive descriptor that is pre-read. (RO)

Register 4.51. AHB_DMA_IN_DSCR_BFO_CH_{*n*}_REG (*n*: 0-2) (0x0094+0xC0n*)**

AHB_DMA_INLINK_DSCR_BFO_CH_{*n*} Represents the address of the current receive descriptor x that is pre-read. (RO)

Register 4.52. AHB_DMA_IN_DSCR_BF1_CH_{*n*}_REG (*n*: 0-2) (0x0098+0xC0n*)**

AHB_DMA_INLINK_DSCR_BF1_CH_{*n*} Represents the address of the previous receive descriptor x-1 that is pre-read. (RO)

Register 4.53. AHB_DMA_OUTFIFO_STATUS_CH_{*n*}_REG (*n*: 0-2) (0x00D8+0xC0n*)**

AHB_DMA_OUTFIFO_FULL_CH_{*n*} Represents whether L1 TX FIFO is full.

0: Not Full

1: Full

(RO)

AHB_DMA_OUTFIFO_EMPTY_CH_{*n*} Represents whether L1 TX FIFO is empty.

0: Not empty

1: Empty

(RO)

AHB_DMA_OUTFIFO_CNT_CH_{*n*} Represents the number of data bytes in L1 TX FIFO for TX channel *n*. (RO)

AHB_DMA_OUT_REMAIN_UNDER_1B_CH_{*n*} Reserved. (RO)

AHB_DMA_OUT_REMAIN_UNDER_2B_CH_{*n*} Reserved. (RO)

AHB_DMA_OUT_REMAIN_UNDER_3B_CH_{*n*} Reserved. (RO)

AHB_DMA_OUT_REMAIN_UNDER_4B_CH_{*n*} Reserved. (RO)

Register 4.54. AHB_DMA_OUT_STATE_CH_{*n*}_REG (*n*: 0-2) (0x00E4+0xC0n*)**

(reserved)										AHB_DMA_OUT_STATE_CH _{<i>n</i>}					AHB_DMA_OUT_DSCR_STATE_CH _{<i>n</i>}					AHB_DMA_OUTLINK_DSCR_ADDR_CH _{<i>n</i>}																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																									
31									23	22			20	19	18	17															0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																														
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

AHB_DMA_OUTLINK_DSCR_ADDR_CH_{*n*} Represents the lower 18 bits of the next transmit descriptor address that is pre-read (but not processed yet). If the current transmit descriptor is the last descriptor, then this field represents the address of the current transmit descriptor. (RO)

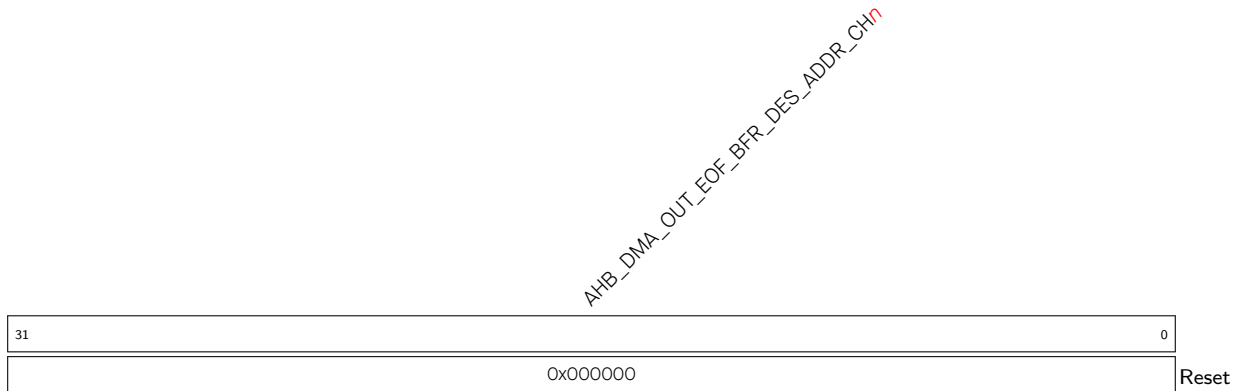
AHB_DMA_OUT_DSCR_STATE_CH_{*n*} Reserved. (RO)

AHB_DMA_OUT_STATE_CH_{*n*} Reserved. (RO)

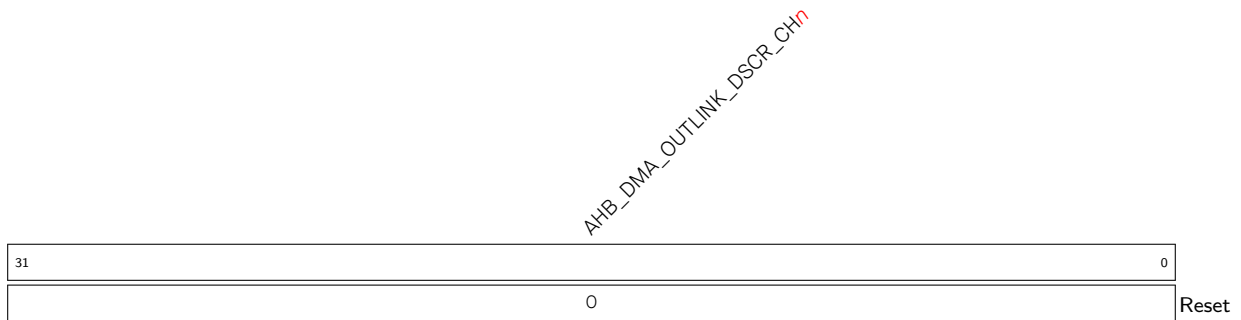
Register 4.55. AHB_DMA_OUT_EOF_DES_ADDR_CH_{*n*}_REG (*n*: 0-2) (0x00E8+0xC0n*)**

AHB_DMA_OUT_EOF_DES_ADDR_CH ⁿ																															
31																															0
0x000000																															
Reset																															

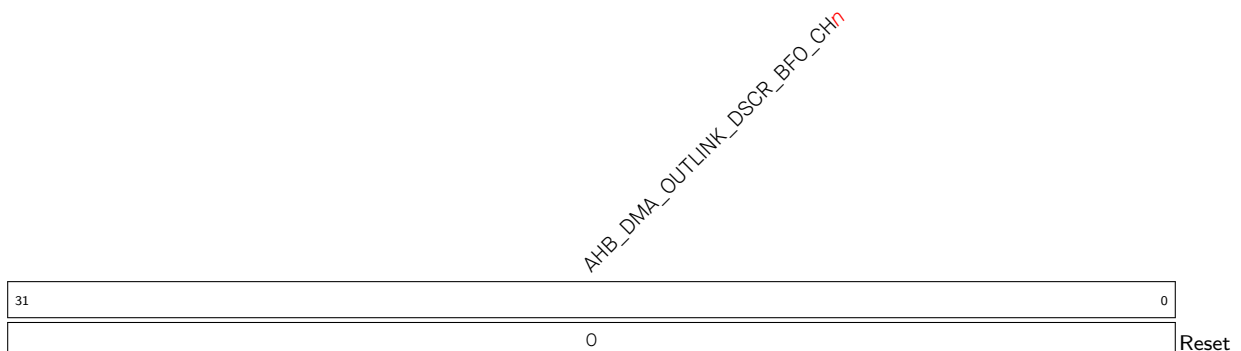
AHB_DMA_OUT_EOF_DES_ADDR_CH_{*n*} Represents the address of the transmit descriptor when the EOF bit in this descriptor is 1. (RO)

Register 4.56. AHB_DMA_OUT_EOF_BFR_DES_ADDR_CH_{*n*}_REG (*n*: 0-2) (0x00EC+0xC0n*)**

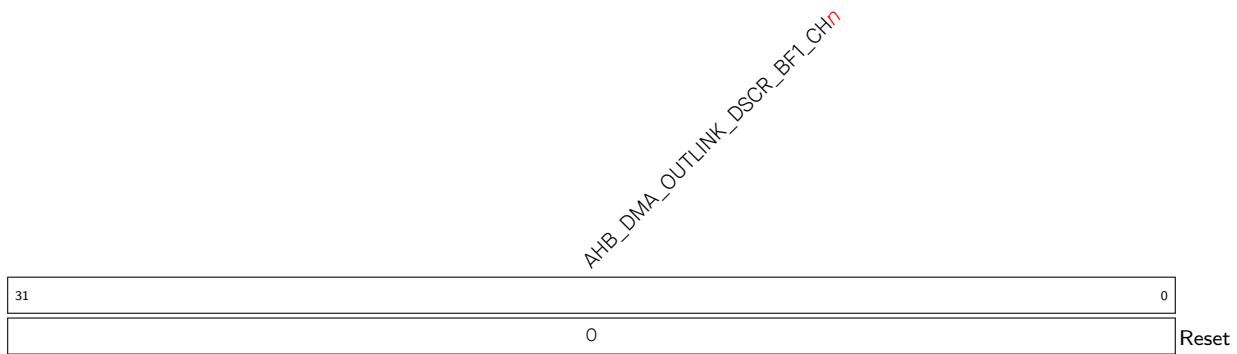
AHB_DMA_OUT_EOF_BFR_DES_ADDR_CH_{*n*} Represents the address of the transmit descriptor before the last transmit descriptor. (RO)

Register 4.57. AHB_DMA_OUT_DSCR_CH_{*n*}_REG (*n*: 0-2) (0x00F0+0xC0n*)**

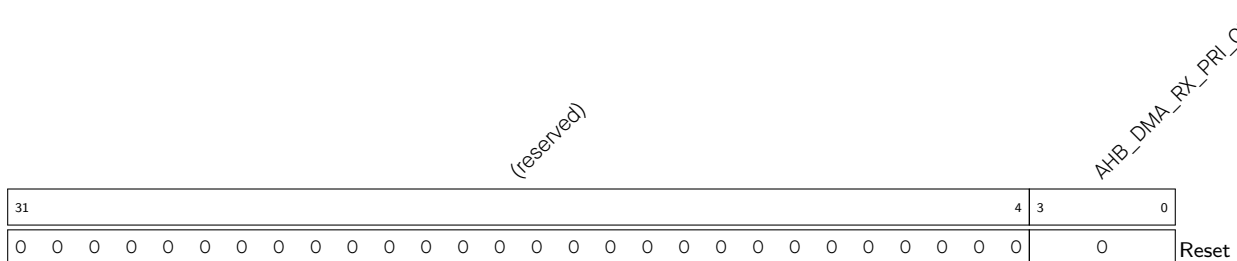
AHB_DMA_OUTLINK_DSCR_CH_{*n*} Represents the address of the next transmit descriptor y+1 pointed by the current transmit descriptor that is pre-read. (RO)

Register 4.58. AHB_DMA_OUT_DSCR_BFO_CH_{*n*}_REG (*n*: 0-2) (0x00F4+0xC0n*)**

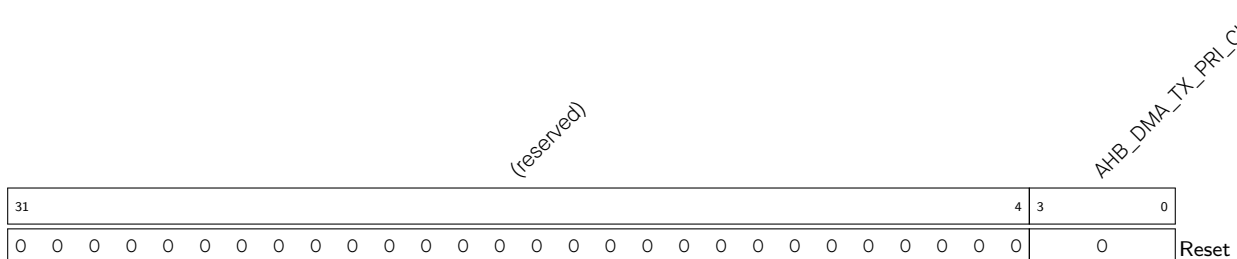
AHB_DMA_OUTLINK_DSCR_BFO_CH_{*n*} Represents the address of the current transmit descriptor y that is pre-read. (RO)

Register 4.59. AHB_DMA_OUT_DSCR_BF1_CH_{*n*}_REG (*n*: 0-2) (0x00F8+0xC0n*)**

AHB_DMA_OUTLINK_DSCR_BF1_CH_{*n*} Represents the address of the previous transmit descriptor y-1 that is pre-read. (RO)

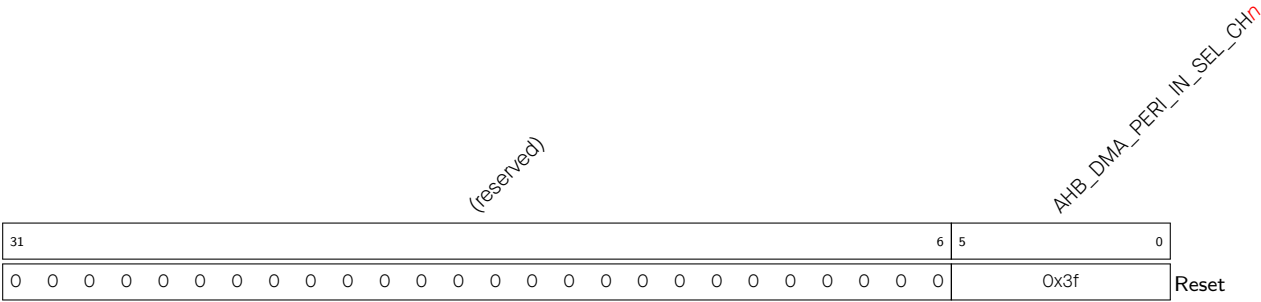
Register 4.60. AHB_DMA_IN_PRI_CH_{*n*}_REG (*n*: 0-2) (0x009C+0xC0n*)**

AHB_DMA_RX_PRI_CH_{*n*} Configures the priority of RX channel *n*. The larger the value, the higher the priority.
Value range: 0 ~ 5 (R/W)

Register 4.61. AHB_DMA_OUT_PRI_CH_{*n*}_REG (*n*: 0-2) (0x00FC+0xC0n*)**

AHB_DMA_TX_PRI_CH_{*n*} Configures the priority of TX channel *n*. The larger the value, the higher the priority.
Value range: 0 ~ 5 (R/W)

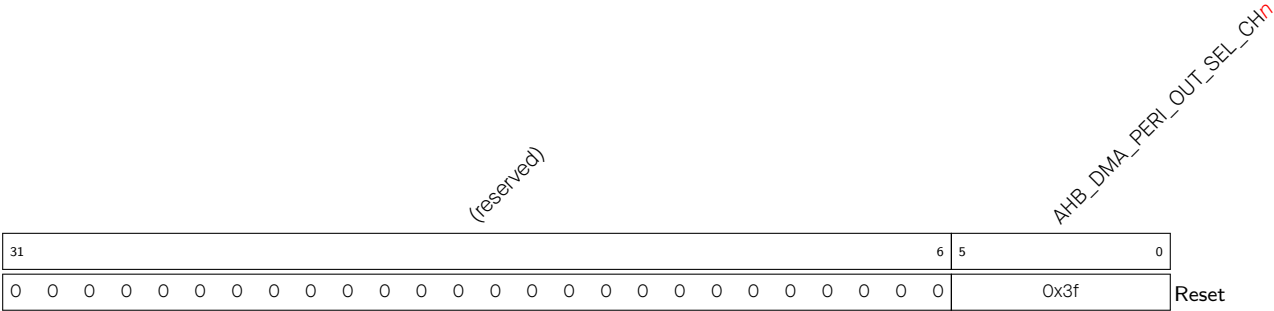
Register 4.62. AHB_DMA_IN_PERI_SEL_CHn_REG (n: 0-2) (0x00A0+0xC0*n)



AHB_DMA_IN_PERI_SEL_CHn Configures the peripheral connected to RX channel n.

- 0: I3C
 - 1: Dummy-1
 - 2: UHCI
 - 3: I2S0
 - 4: I2S1
 - 5: I2S2
 - 6: Dummy-6
 - 7: Dummy-7
 - 8: ADC
 - 9: Dummy-9
 - 10: RMT
 - 11 ~ 15: Dummy-11 ~ Dummy-15
 - 16 ~ 63: Invalid
- (R/W)

Register 4.63. AHB_DMA_OUT_PERI_SEL_CHn_REG (n: 0-2) (0x0100+0xC0*n)



AHB_DMA_OUT_PERI_SEL_CHn Configures the peripheral connected to TX channel n.

- 0: I3C
 - 1: Dummy-1
 - 2: UHCI
 - 3: I2S0
 - 4: I2S1
 - 5: I2S2
 - 6: Dummy-6
 - 7: Dummy-7
 - 8: ADC
 - 9: Dummy-9
 - 10: RMT
 - 11 ~ 15: Dummy-11 ~ Dummy-15
 - 16 ~ 63: Invalid
- (R/W)

Register 4.64. AHB_DMA_MODULE_CLK_EN_REG (0x0404)

(reserved)				AHB_DMA_AHBIF_CLK_EN				AHB_DMA_CMD_ARB_CLK_EN				(reserved)				AHB_DMA_IN_CTRL_CLK_EN				AHB_DMA_IN_DSCR_CLK_EN				AHB_DMA_OUT_CTRL_CLK_EN				AHB_DMA_OUT_DSCR_CLK_EN				AHB_DMA_AHB_APB_SYNC_CLK_EN			
31	29	28	27	26											15	14	12	11	9	8	6	5	3	2	0										
0	0	0	0x0	0x0	0	0	0	0	0	0	0	0	0	0	0	0x7	0x7	0x7	0x7	0x7	0x7	Reset													

AHB_DMA_AHB_APB_SYNC_CLK_EN Configures whether to forcibly enable the clock for the AHB DMA register configuration synchronization module.

Bit 0 corresponds to TX and RX channel 0, bit 1 corresponds to channel TX and RX 1, and so on.

0: Not forcibly enable

1: Forcibly enable

(R/W)

AHB_DMA_OUT_DSCR_CLK_EN Configures whether to forcibly enable the clock for the AHB DMA TX descriptor processing module.

Bit 0 corresponds to TX channel 0, bit 1 corresponds to TX channel 1, and so on.

0: Not forcibly enable

1: Forcibly enable

(R/W)

AHB_DMA_OUT_CTRL_CLK_EN Configures whether to forcibly enable the clock for the AHB DMA TX data processing module.

Bit 0 corresponds to TX channel 0, bit 1 corresponds to TX channel 1, and so on.

0: Not forcibly enable

1: Forcibly enable

(R/W)

AHB_DMA_IN_DSCR_CLK_EN Configures whether to forcibly enable the clock for the AHB DMA RX descriptor processing module.

Bit 0 corresponds to RX channel 0, bit 1 corresponds to TX channel 1, and so on.

0: Not forcibly enable

1: Forcibly enable

(R/W)

AHB_DMA_IN_CTRL_CLK_EN Configures whether to forcibly enable the clock for the AHB DMA RX data processing module.

Bit 0 corresponds to RX channel 0, bit 1 corresponds to TX channel 1, and so on.

0: Not forcibly enable

1: Forcibly enable

(R/W)

Continued on the next page...

Register 4.64. AHB_DMA_MODULE_CLK_EN_REG (0x0404)

Continued from the previous page...

AHB_DMA_CMD_ARB_CLK_EN Configures whether to forcibly enable the clock for the AHB DMA arbitration module.
0: Not forcibly enable
1: Forcibly enable
(R/W)

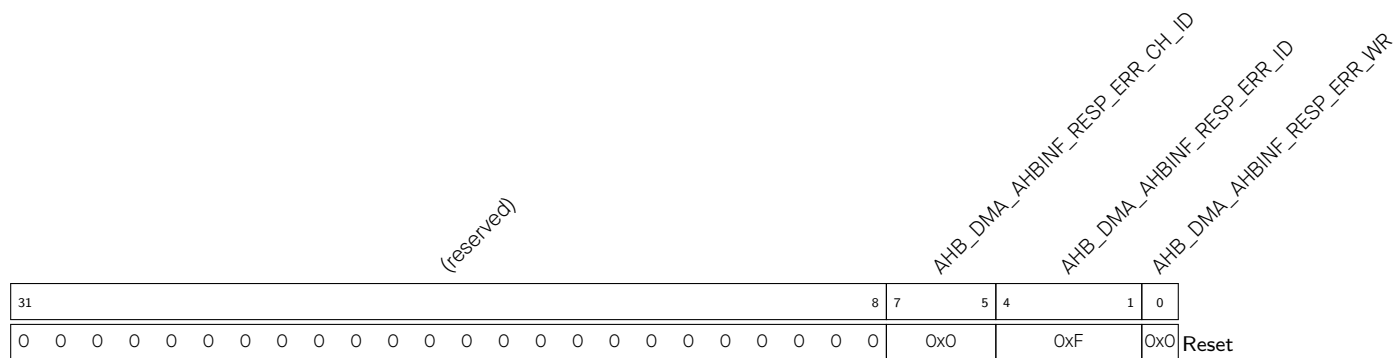
AHB_DMA_AHBMIF_CLK_EN Configures whether to forcibly enable the clock for the AHB DMA bus interface processing module.
0: Not forcibly enable
1: Forcibly enable
(R/W)

Register 4.65. AHB_DMA_AHBMIF_RESP_ERR_STATUS0_REG (0x0408)



AHB_DMA_AHBMIF_RESP_ERR_ADDR Represents the address of the current AHB bus transfer that triggers an error response. (RO)

Register 4.66. AHB_DMA_AHBINF_RESP_ERR_STATUS1_REG (0x040C)



AHB_DMA_AHBINF_RESP_ERR_CH_ID Represents the channel ID of the transfer that triggers the AHB bus error response.

Bits[0:1]: Channel number. Value range: 0 ~2

Bit[2]: Channel direction. 0: RX

1: TX

(RO)

AHB_DMA_AHBINF_RESP_ERR_ID Represents the ID of the transfer that triggers the AHB bus error response.

- 1: Transfer between GP-SPI and memory
- 2: Transfer between UHCI and memory
- 3: Transfer between I2S and memory
- 6: Transfer between AES and memory
- 7: Transfer between SHA and memory
- 8: Transfer between ADC and memory
- 9: Transfer between PARLIO and memory
- 11: Memory-to-memory transfer on channel 1
- 12: Memory-to-memory transfer on channel 2

Other values are invalid

(RO)

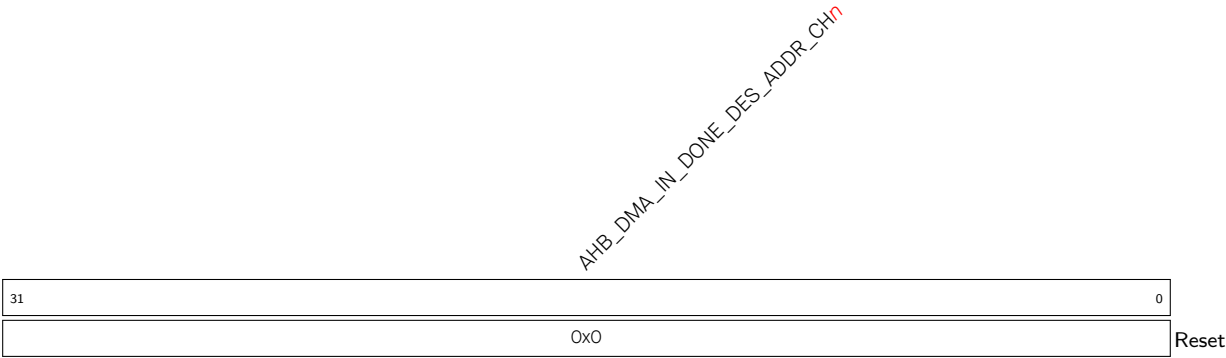
AHB_DMA_AHBINF_RESP_ERR_WR Represents the type of the transfer that triggered the AHB bus error response received by the AHB DMA.

0: Read

1: Write

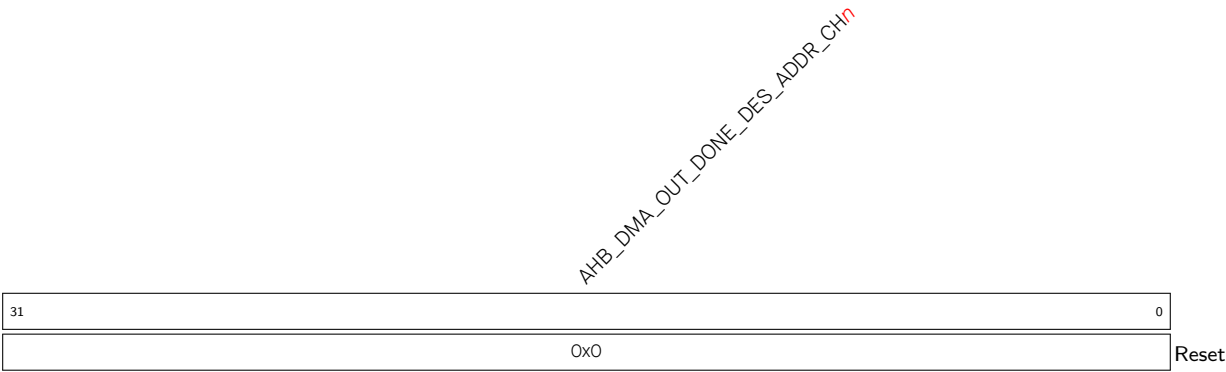
(RO)

Register 4.67. AHB_DMA_IN_DONE_DES_ADDR_CH n _REG(n : 0-2) (0x0410+0x08* n)



AHB_DMA_IN_DONE_DES_ADDR_CH n Represents the address of the descriptor corresponding to the completed RX transfer. (RO)

Register 4.68. AHB_DMA_OUT_DONE_DES_ADDR_CH n _REG(n : 0-2) (0x0414+0x08* n)



AHB_DMA_OUT_DONE_DES_ADDR_CH n Represents the address of the descriptor corresponding to the completed TX transfer. (RO)

4.9.2 GDMA-AXI Registers

The addresses in this section are relative to GDMA-AXI base address provided in Table 7.3-2 in Chapter 7 [System and Memory](#).

For how to program reserved fields, please refer to Section [Programming Reserved Register Field](#).

Register 4.69. AXI_DMA_IN_INT_RAW_CH n _REG (n : 0-2) (0x0000+0x68* n)

(reserved)																						AXI_DMA_INFIFO_L3_UDF_CH _n _INT_RAW AXI_DMA_INFIFO_L3_OVF_CH _n _INT_RAW AXI_DMA_INFIFO_L2_UDF_CH _n _INT_RAW AXI_DMA_INFIFO_L2_OVF_CH _n _INT_RAW AXI_DMA_INFIFO_L1_UDF_CH _n _INT_RAW AXI_DMA_INFIFO_L1_OVF_CH _n _INT_RAW AXI_DMA_IN_DSCR_EMPTY_CH _n _INT_RAW AXI_DMA_IN_ERR_EOF_CH _n _INT_RAW AXI_DMA_IN_SUC_EOF_CH _n _INT_RAW AXI_DMA_IN_DONE_CH _n _INT_RAW																			
31																						11	10	9	8	7	6	5	4	3	2	1	0								
0																						0	0	0	0	0	0	0	0	0	0	0	0	0	Reset						

AXI_DMA_IN_DONE_CH n _INT_RAW The raw interrupt status of AXI_DMA_IN_DONE_CH n _INT. (R/WTC/SS)

AXI_DMA_IN_SUC_EOF_CH n _INT_RAW The raw interrupt status of AXI_DMA_IN_SUC_EOF_CH n _INT. For UHCI this bit turns to 1 when the last data byte pointed by one receive descriptor has been received and no data error is detected for RX channel 0. (R/WTC/SS)

AXI_DMA_IN_ERR_EOF_CH n _INT_RAW The raw interrupt status of AXI_DMA_IN_ERR_EOF_CH n _INT. Valid only for UHCI. (R/WTC/SS)

AXI_DMA_IN_DSCR_ERR_CH n _INT_RAW The raw interrupt status of AXI_DMA_IN_DSCR_ERR_CH n _INT. (R/WTC/SS)

AXI_DMA_IN_DSCR_EMPTY_CH n _INT_RAW The raw interrupt status of AXI_DMA_IN_DSCR_EMPTY_CH n _INT. (R/WTC/SS)

AXI_DMA_INFIFO_L1_OVF_CH n _INT_RAW The raw interrupt status of AXI_DMA_INFIFO_L1_OVF_CH n _INT. (R/WTC/SS)

AXI_DMA_INFIFO_L1_UDF_CH n _INT_RAW The raw interrupt status of AXI_DMA_INFIFO_L1_UDF_CH n _INT. (R/WTC/SS)

AXI_DMA_INFIFO_L2_OVF_CH n _INT_RAW The raw interrupt status of AXI_DMA_INFIFO_L2_OVF_CH n _INT. (R/WTC/SS)

AXI_DMA_INFIFO_L2_UDF_CH n _INT_RAW The raw interrupt status of AXI_DMA_INFIFO_L2_UDF_CH n _INT. (R/WTC/SS)

AXI_DMA_INFIFO_L3_OVF_CH n _INT_RAW The raw interrupt status of AXI_DMA_INFIFO_L3_OVF_CH n _INT. (R/WTC/SS)

AXI_DMA_INFIFO_L3_UDF_CH n _INT_RAW The raw interrupt status of AXI_DMA_INFIFO_L3_UDF_CH n _INT. (R/WTC/SS)

Register 4.70. AXI_DMA_IN_INT_ST_CH n _REG (n : 0-2) (0x0004+0x68* n)

(reserved)																								AXI_DMA_IN_FIFO_L3_UDF_CH _n _INT_ST AXI_DMA_IN_FIFO_L3_OVF_CH _n _INT_ST AXI_DMA_IN_FIFO_L2_UDF_CH _n _INT_ST AXI_DMA_IN_FIFO_L2_OVF_CH _n _INT_ST AXI_DMA_IN_FIFO_L1_UDF_CH _n _INT_ST AXI_DMA_IN_FIFO_L1_OVF_CH _n _INT_ST AXI_DMA_IN_DSCR_EMPTY_CH _n _INT_ST AXI_DMA_IN_DSCR_ERR_EOF_CH _n _INT_ST AXI_DMA_IN_SUC_EOF_CH _n _INT_ST AXI_DMA_IN_DONE_CH _n _INT_ST									
31												11	10	9	8	7	6	5	4	3	2	1	0										
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset								

AXI_DMA_IN_DONE_CH n _INT_ST The masked interrupt status of AXI_DMA_IN_DONE_CH n _INT. (RO)

AXI_DMA_IN_SUC_EOF_CH n _INT_ST The masked interrupt status of AXI_DMA_IN_SUC_EOF_CH n _INT. (RO)

AXI_DMA_IN_ERR_EOF_CH n _INT_ST The masked interrupt status of AXI_DMA_IN_ERR_EOF_CH n _INT. (RO)

AXI_DMA_IN_DSCR_ERR_CH n _INT_ST The masked interrupt status of AXI_DMA_IN_DSCR_ERR_CH n _INT. (RO)

AXI_DMA_IN_DSCR_EMPTY_CH n _INT_ST The masked interrupt status of AXI_DMA_IN_DSCR_EMPTY_CH n _INT. (RO)

AXI_DMA_IN_FIFO_L2_OVF_CH n _INT_ST The masked interrupt status of AXI_DMA_IN_FIFO_L2_OVF_CH n _INT. (RO)

AXI_DMA_IN_FIFO_L2_UDF_CH n _INT_ST The masked interrupt status of AXI_DMA_IN_FIFO_L2_UDF_CH n _INT. (RO)

AXI_DMA_IN_FIFO_L1_OVF_CH n _INT_ST The masked interrupt status of AXI_DMA_IN_FIFO_L1_OVF_CH n _INT. (RO)

AXI_DMA_IN_FIFO_L1_UDF_CH n _INT_ST The masked interrupt status of AXI_DMA_IN_FIFO_L1_UDF_CH n _INT. (RO)

AXI_DMA_IN_FIFO_L3_OVF_CH n _INT_ST The masked interrupt status of AXI_DMA_IN_FIFO_L3_OVF_CH n _INT. (RO)

AXI_DMA_IN_FIFO_L3_UDF_CH n _INT_ST The masked interrupt status of AXI_DMA_IN_FIFO_L3_UDF_CH n _INT. (RO)

Register 4.71. AXI_DMA_IN_INT_ENA_CH n _REG (n : 0-2) (0x0008+0x68* n)

(reserved)																						AXI_DMA_INFIFO_L3_UDF_CHn_INT_ENA AXI_DMA_INFIFO_L3_OVF_CHn_INT_ENA AXI_DMA_INFIFO_L2_UDF_CHn_INT_ENA AXI_DMA_INFIFO_L2_OVF_CHn_INT_ENA AXI_DMA_INFIFO_L1_UDF_CHn_INT_ENA AXI_DMA_IN_DSCR_EMPTY_CHn_INT_ENA AXI_DMA_IN_ERR_EOF_CHn_INT_ENA AXI_DMA_IN_SUC_EOF_CHn_INT_ENA AXI_DMA_IN_DONE_CHn_INT_ENA											
31											11											10	9	8	7	6	5	4	3	2	1	0	
0											0											0	0	0	0	0	0	0	0	0	0	0	Reset

AXI_DMA_IN_DONE_CH n _INT_ENA Write 1 to enable AXI_DMA_IN_DONE_CH n _INT. (R/W)

AXI_DMA_IN_SUC_EOF_CH n _INT_ENA Write 1 to enable AXI_DMA_IN_SUC_EOF_CH n _INT. (R/W)

AXI_DMA_IN_ERR_EOF_CH n _INT_ENA Write 1 to enable AXI_DMA_IN_ERR_EOF_CH n _INT. (R/W)

AXI_DMA_IN_DSCR_ERR_CH n _INT_ENA Write 1 to enable AXI_DMA_IN_DSCR_ERR_CH n _INT.
(R/W)

AXI_DMA_IN_DSCR_EMPTY_CH n _INT_ENA Write 1 to enable AXI_DMA_IN_DSCR_EMPTY_CH n _INT.
(R/W)

AXI_DMA_INFIFO_L1_OVF_CH n _INT_ENA Write 1 to enable AXI_DMA_INFIFO_L1_OVF_CH n _INT.
(R/W)

AXI_DMA_INFIFO_L1_UDF_CH n _INT_ENA Write 1 to enable AXI_DMA_INFIFO_L1_UDF_CH n _INT.
(R/W)

AXI_DMA_INFIFO_L2_OVF_CH n _INT_ENA Write 1 to enable AXI_DMA_INFIFO_L2_OVF_CH n _INT.
(R/W)

AXI_DMA_INFIFO_L2_UDF_CH n _INT_ENA Write 1 to enable AXI_DMA_INFIFO_L2_UDF_CH n _INT.
(R/W)

AXI_DMA_INFIFO_L3_OVF_CH n _INT_ENA Write 1 to enable AXI_DMA_INFIFO_L3_OVF_CH n _INT.
(R/W)

AXI_DMA_INFIFO_L3_UDF_CH n _INT_ENA Write 1 to enable AXI_DMA_INFIFO_L3_UDF_CH n _INT.
(R/W)

Register 4.72. AXI_DMA_IN_INT_CLR_CH n _REG (n : 0-2) (0x000C+0x68* n)

(reserved)																						AXI_DMA_INFIFO_L3_UDF_CH _n _INT_CLR AXI_DMA_INFIFO_L3_OVF_CH _n _INT_CLR AXI_DMA_INFIFO_L2_UDF_CH _n _INT_CLR AXI_DMA_INFIFO_L2_OVF_CH _n _INT_CLR AXI_DMA_INFIFO_L1_UDF_CH _n _INT_CLR AXI_DMA_IN_DSCR_EMPTY_CH _n _INT_CLR AXI_DMA_IN_DSCR_ERR_CH _n _INT_CLR AXI_DMA_IN_SUC_EOF_CH _n _INT_CLR AXI_DMA_IN_DONE_CH _n _INT_CLR											
31											11											10	9	8	7	6	5	4	3	2	1	0	
0											0											0	0	0	0	0	0	0	0	0	0	0	Reset

AXI_DMA_IN_DONE_CH n _INT_CLR Write 1 to clear AXI_DMA_IN_DONE_CH n _INT. (WT)

AXI_DMA_IN_SUC_EOF_CH n _INT_CLR Write 1 to clear AXI_DMA_IN_SUC_EOF_CH n _INT. (WT)

AXI_DMA_IN_ERR_EOF_CH n _INT_CLR Write 1 to clear AXI_DMA_IN_ERR_EOF_CH n _INT. (WT)

AXI_DMA_IN_DSCR_ERR_CH n _INT_CLR Write 1 to clear AXI_DMA_IN_DSCR_ERR_CH n _INT. (WT)

AXI_DMA_IN_DSCR_EMPTY_CH n _INT_CLR Write 1 to clear AXI_DMA_IN_DSCR_EMPTY_CH n _INT.
(WT)

AXI_DMA_INFIFO_L1_OVF_CH n _INT_CLR Write 1 to clear AXI_DMA_INFIFO_L1_OVF_CH n _INT. (WT)

AXI_DMA_INFIFO_L1_UDF_CH n _INT_CLR Write 1 to clear AXI_DMA_INFIFO_L1_UDF_CH n _INT.
(WT)

AXI_DMA_INFIFO_L2_OVF_CH n _INT_CLR Write 1 to clear AXI_DMA_INFIFO_L2_OVF_CH n _INT.
(WT)

AXI_DMA_INFIFO_L2_UDF_CH n _INT_CLR Write 1 to clear AXI_DMA_INFIFO_L2_UDF_CH n _INT.
(WT)

AXI_DMA_INFIFO_L3_OVF_CH n _INT_CLR Write 1 to clear AXI_DMA_INFIFO_L3_OVF_CH n _INT.
(WT)

AXI_DMA_INFIFO_L3_UDF_CH n _INT_CLR Write 1 to clear AXI_DMA_INFIFO_L3_UDF_CH n _INT.
(WT)

Register 4.73. AXI_DMA_OUT_INT_RAW_CH n _REG (n : 0-2) (0x0138+0x68* n)

AXI_DMA_OUT_DONE_CH n _INT_RAW The raw interrupt status of AXI_DMA_OUT_DONE_CH n _INT.
(R/WTC/SS)

AXI_DMA_OUT_EOF_CH n _INT_RAW The raw interrupt status of AXI_DMA_OUT_EOF_CH n _INT.
(R/WTC/SS)

AXI_DMA_OUT_DSCR_ERR_CH_nINT_RAW The raw interrupt status of AXI_DMA_OUT_DSCR_ERR_CH_nINT. (R/WTC/SS)

AXI_DMA_OUT_TOTAL_EOF_CHn_INT_RAW The raw interrupt status of AXI_DMA_OUT_TOTAL_EOF_CHn_INT. (R/WTC/SS)

AXI_DMA_OUTFIFO_L1_OVF_CH n _INT_RAW The raw interrupt status of AXI_DMA_OUTFIFO_L1_OVF_CH n _INT. (R/WTC/SS)

AXI_DMA_OUTFIFO_L1_UDF_CH n _INT_RAW The raw interrupt status of AXI_DMA_OUTFIFO_L1_UDF_CH n _INT. (R/WTC/SS)

AXI_DMA_OUTFIFO_L2_OVF_CH_n_INT_RAW The raw interrupt status of AXI_DMA_OUTFIFO_L2_OVF_CH_n_INT. (R/WTC/SS)

AXI_DMA_OUTFIFO_L2_UDF_CH n _INT_RAW The raw interrupt status of AXI DMA OUTFIFO L2 UDF CH n INT. (R/WTC/SS)

AXI_DMA_OUTFIFO_L3_OVF_CH_n_INT_RAW The raw interrupt status of AXI DMA OUTFIFO L3 OVF CH_n INT. (R/WTC/SS)

AXI_DMA_OUTFIFO_L3_UDF_CH n _INT_RAW The raw interrupt status of AXI_DMA_OUTFIFO_L3_UDF_CH n _INT. (R/WTC/SS)

AXI_DMA_OUT_LINK_SWITCH_CH n _INT_RAW The raw interrupt status of AXI DMA OUT LINK SWITCH CH n INT. (R/WTC/SS)

Register 4.74. AXI_DMA_OUT_INT_ST_CH n _REG (n : 0-2) (0x013C+0x68* n)

(reserved)																						AXI_DMA_OUT_LINK_SWITCH_CH _n _INT_ST AXI_DMA_OUT_FIFO_L3_UDF_CH _n _INT_ST AXI_DMA_OUT_FIFO_L3_OVF_CH _n _INT_ST AXI_DMA_OUT_FIFO_L2_UDF_CH _n _INT_ST AXI_DMA_OUT_FIFO_L2_OVF_CH _n _INT_ST AXI_DMA_OUT_FIFO_L1_UDF_CH _n _INT_ST AXI_DMA_OUT_FIFO_L1_OVF_CH _n _INT_ST AXI_DMA_OUT_TOTAL_EOF_ERR_CH _n _INT_ST AXI_DMA_OUT_DSCR_ERR_CH _n _INT_ST AXI_DMA_OUT_DONE_CH _n _INT_ST												
31																						11	10	9	8	7	6	5	4	3	2	1	0	Reset
0 0																						0	0	0	0	0	0	0	0	0	0	0		

AXI_DMA_OUT_DONE_CH n _INT_ST The masked interrupt status of AXI_DMA_OUT_DONE_CH n _INT. (RO)

AXI_DMA_OUT_EOF_CH n _INT_ST The masked interrupt status of AXI_DMA_OUT_EOF_CH n _INT. (RO)

AXI_DMA_OUT_DSCR_ERR_CH n _INT_ST The masked interrupt status of AXI_DMA_OUT_DSCR_ERR_CH n _INT. (RO)

AXI_DMA_OUT_TOTAL_EOF_CH n _INT_ST The masked interrupt status of AXI_DMA_OUT_TOTAL_EOF_CH n _INT. (RO)

AXI_DMA_OUTFIFO_L1_OVF_CH n _INT_ST The masked interrupt status of AXI_DMA_OUTFIFO_L1_OVF_CH n _INT. (RO)

AXI_DMA_OUTFIFO_L1_UDF_CH n _INT_ST The masked interrupt status of AXI_DMA_OUTFIFO_L1_UDF_CH n _INT. (RO)

AXI_DMA_OUTFIFO_L2_OVF_CH n _INT_ST The masked interrupt status of AXI_DMA_OUTFIFO_L2_OVF_CH n _INT. (RO)

AXI_DMA_OUTFIFO_L2_UDF_CH n _INT_ST The masked interrupt status of AXI_DMA_OUTFIFO_L2_UDF_CH n _INT. (RO)

AXI_DMA_OUTFIFO_L3_OVF_CH n _INT_ST The masked interrupt status of AXI_DMA_OUTFIFO_L3_OVF_CH n _INT. (RO)

AXI_DMA_OUTFIFO_L3_UDF_CH n _INT_ST The masked interrupt status of AXI_DMA_OUTFIFO_L3_UDF_CH n _INT. (RO)

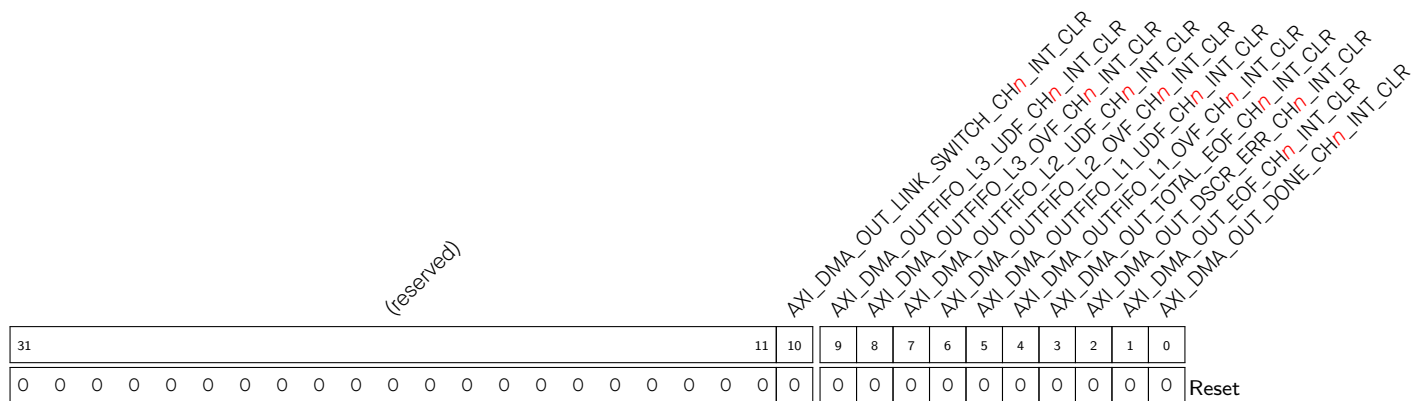
AXI_DMA_OUT_LINK_SWITCH_CH n _INT_ST The masked interrupt status of AXI_DMA_OUT_LINK_SWITCH_CH n _INT. (RO)

(reserved)

AXI_DMA_OUT_LINK_SWITCH_CHn_INT_ENA
AXI_DMA_OUT_FIFO_L3_UDF_CHn_INT_ENA
AXI_DMA_OUT_FIFO_L3_OVF_CHn_INT_ENA
AXI_DMA_OUT_FIFO_L2_UDF_CHn_INT_ENA
AXI_DMA_OUT_FIFO_L2_OVF_CHn_INT_ENA
AXI_DMA_OUT_FIFO_L1_UDF_CHn_INT_ENA
AXI_DMA_OUT_TOTAL_EOF_CHn_INT_ENA
AXI_DMA_OUT_DSCR_ERR_CHn_INT_ENA
AXI_DMA_OUT_DONE_CHn_INT_ENA

AXI_DMA_OUT_LINK_SWITCH_CH n _INT_ENA Write 1 to enable AXI_DMA_OUT_LINK_SWITCH_CH n _INT.
(RO)

Register 4.76. AXI_DMA_OUT_INT_CLR_CH n _REG (n : 0-2) (0x0144+0x68* n)



AXI_DMA_OUT_DONE_CH n _INT_CLR Write 1 to clear AXI_DMA_OUT_DONE_CH n _INT. (WT)

AXI_DMA_OUT_EOF_CH n _INT_CLR Write 1 to clear AXI_DMA_OUT_EOF_CH n _INT. (WT)

AXI_DMA_OUT_DSCR_ERR_CH_n_INT_CLR Write 1 to clear AXI_DMA_OUT_DSCR_ERR_CH_n_INT.
(WT)

AXI_DMA_OUT_TOTAL_EOF_CH_n_INT_CLR Write 1 to clear AXI_DMA_OUT_TOTAL_EOF_CH_n_INT.
(WT)

AXI_DMA_OUTFIFO_L1_OVF_CH n _INT_CLR Write 1 to clear AXI_DMA_OUTFIFO_L1_OVF_CH n _INT.
(WT)

AXI_DMA_OUTFIFO_L1_UDF_CH n _INT_CLR Write 1 to clear AXI_DMA_OUTFIFO_L1_UDF_CH n _INT.
(WT)

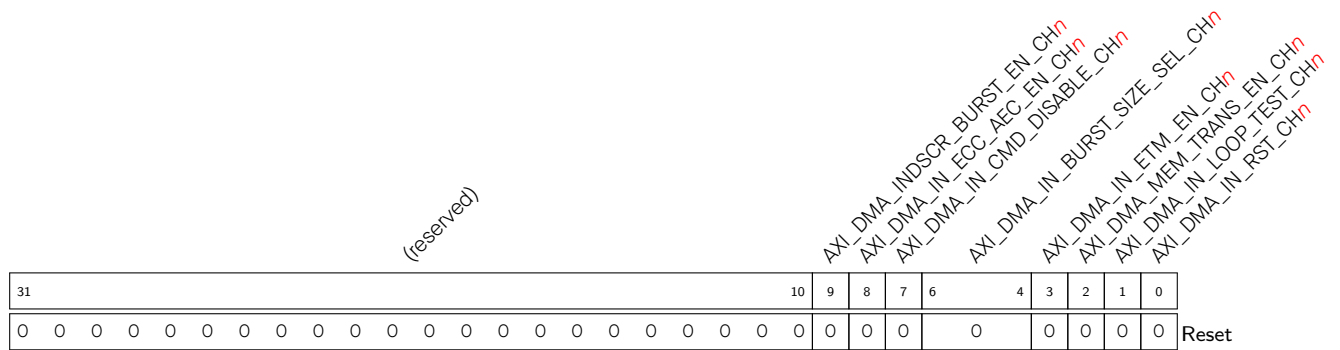
AXI_DMA_OUTFIFO_L2_OVF_CH n _INT_CLR Write 1 to clear AXI_DMA_OUTFIFO_L2_OVF_CH n _INT.
(WT)

AXI_DMA_OUTFIFO_L2_UDF_CH n _INT_CLR Write 1 to clear AXI_DMA_OUTFIFO_L2_UDF_CH n _INT.
(WT)

AXI_DMA_OUTFIFO_L3_OVF_CH n _INT_CLR Write 1 to clear AXI_DMA_OUTFIFO_L3_OVF_CH n _INT.
(WT)

AXI_DMA_OUTFIFO_L3_UDF_CH n _INT_CLR Write 1 to clear AXI_DMA_OUTFIFO_L3_UDF_CH n _INT.
(WT)

AXI_DMA_OUT_LINK_SWITCH_CH_n_INT_CLR Write 1 to clear AXI_DMA_OUT_LINK_SWITCH_CH_n_INT.
(RO)

Register 4.77. AXI_DMA_IN_CONFO_CH n _REG (n : 0-2) (0x0010+0x68* n)

AXI_DMA_IN_RST_CH n Write 1 and then 0 to reset RX channel n FSM and RX FIFO pointer.(R/W)

AXI_DMA_IN_LOOP_TEST_CH n Reserved. (R/W)

AXI_DMA_MEM_TRANS_EN_CH n Configures whether to enable memory-to-memory data transfer.

0: Disable

1: Enable

(R/W)

AXI_DMA_IN_ETM_EN_CH n Configures whether to enable ETM control for RX channel n .

0: Disable

1: Enable

(R/W)

AXI_DMA_IN_BURST_SIZE_SEL_CH n Configures the burst length for RX channel n .

0: 8 bytes

1: 16 bytes

2: 32 bytes

3: 64 bytes

4: 128 bytes

5 ~ 7: Invalid

(R/W)

AXI_DMA_IN_CMD_DISABLE_CH n Configures whether to disable command on RX channel n .

0: Enable

1: Disable

(R/W)

AXI_DMA_IN_ECC_AEC_EN_CH n Configures whether AXI DMA can access external memory space for ECC and AES via RX channel n .

0: Not access

1: Access

(R/W)

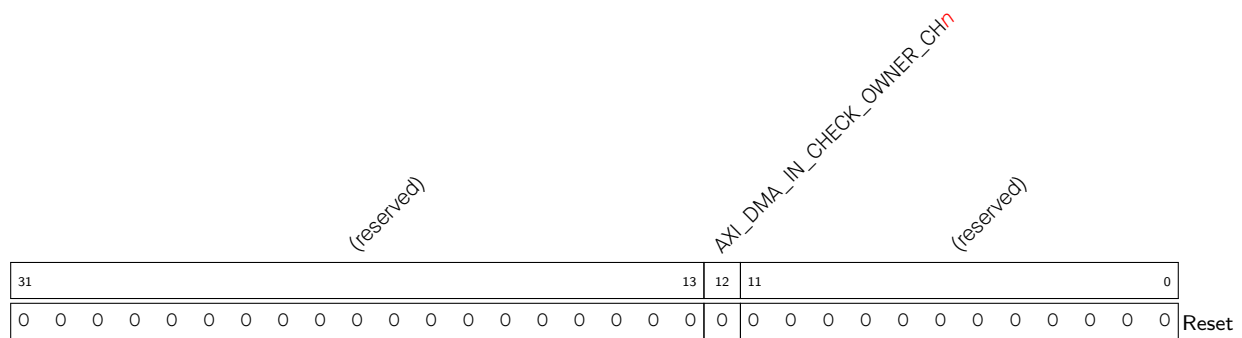
AXI_DMA_INDSR_BURST_EN_CH n Configures whether to enable INCR burst transfer for RX channel n to read descriptors when accessing internal memory.

0: Disable

1: Enable

(R/W)

Register 4.78. AXI_DMA_IN_CONF1_CH n _REG (n : 0-2) (0x0014+0x68* n)



AXI_DMA_IN_CHECK_OWNER_CHn Configures whether to enable owner bit check for RX channel

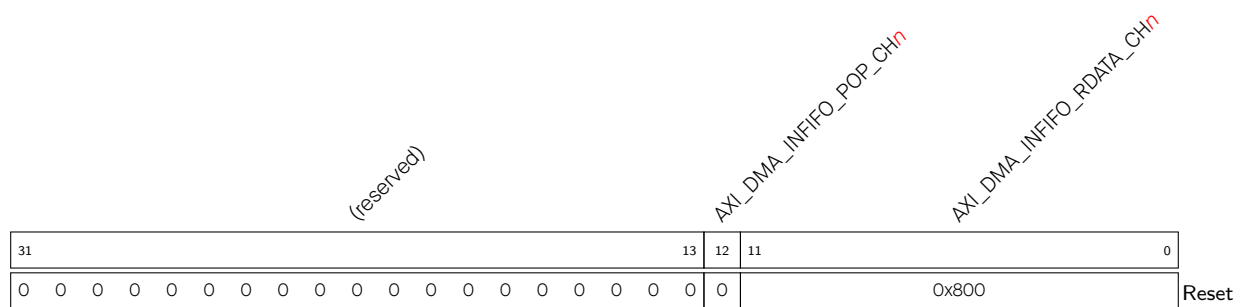
 $n.$

0: Disable

1: Enable

(R/W)

Register 4.79. AXI_DMA_IN_POP_CH n _REG (n : 0-2) (0x001C+0x68* n)



AXI_DMA_INFIFO_RDATA_CH_n Represents the data popped from AXI DMA RX FIFO. (RO)

AXI_DMA_INFIFO_POP_CH n Configures whether to pop data from AXI DMA RX FIFO.

0: Invalid. No effect

1: Pop

(WT)

Register 4.80. AXI_DMA_IN_LINK1_CH n _REG (n : 0-2) (0x0020+0x68* n)

(reserved)																												AXI_DMA_INLINK_PARK_CH _n AXI_DMA_INLINK_RESTART_CH _n AXI_DMA_INLINK_START_CH _n AXI_DMA_INLINK_STOP_CH _n AXI_DMA_INLINK_AUTO_RET_CH _n						
31																												5	4	3	2	1	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	Reset			

AXI_DMA_INLINK_AUTO_RET_CH n Configures whether to return to the current receive descriptor's address when there are some errors in current receiving data.

0: Not return

1: Return

(R/W)

AXI_DMA_INLINK_STOP_CH n Configures whether to stop RX channel n from receiving data.

0: Invalid. No effect

1: Stop

(WT)

AXI_DMA_INLINK_START_CH n Configures whether to enable RX channel n for data transfer.

0: Disable

1: Enable

(WT)

AXI_DMA_INLINK_RESTART_CH n Configures whether to restart RX channel n for AXI DMA transfer.

0: Invalid. No effect

1: Restart

(WT)

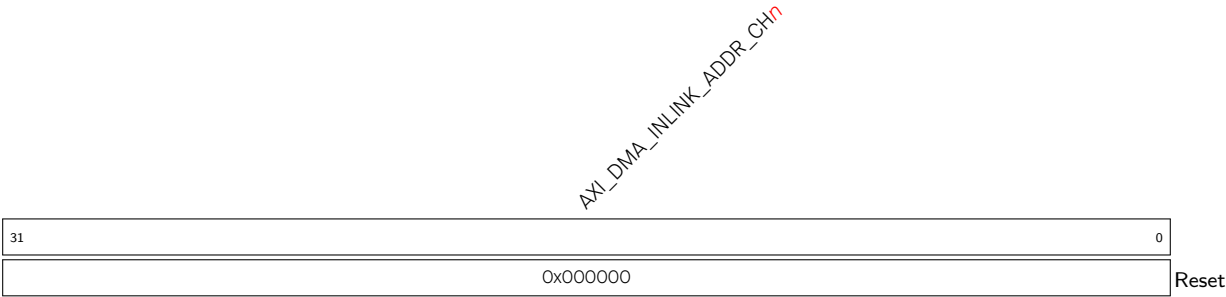
AXI_DMA_INLINK_PARK_CH n Represents the status of the receive descriptor's FSM.

0: Running

1: Idle

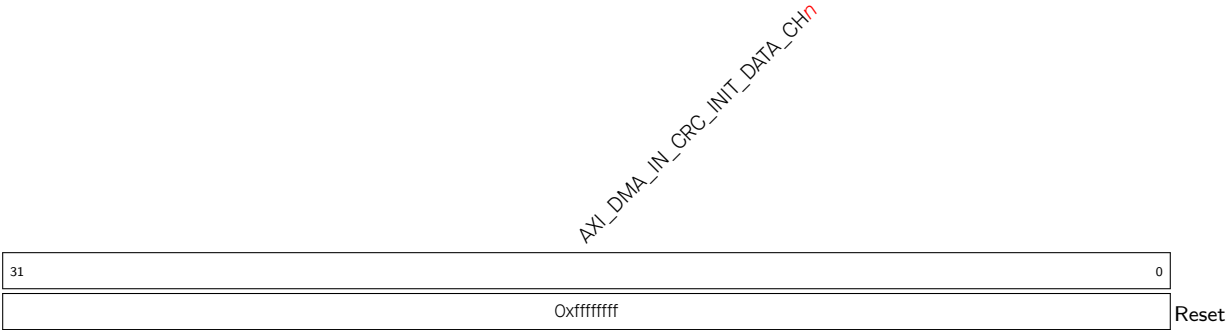
(RO)

Register 4.81. AXI_DMA_IN_LINK2_CH n _REG (n : 0-2) (0x0024+0x68* n)



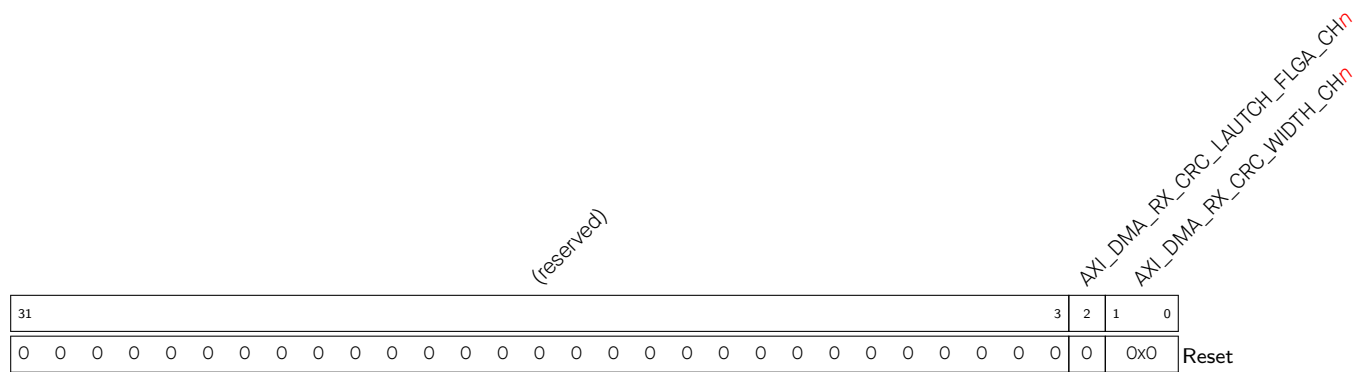
AXI_DMA_INLINK_ADDR_CH n Represents the first receive descriptor’s address. (R/W)

Register 4.82. AXI_DMA_IN_CRC_INIT_DATA_CH n _REG (n : 0-2) (0x0048+0x68* n)



AXI_DMA_IN_CRC_INIT_DATA_CH n Configures the CRC initial value for RX channel n . (R/W)

Register 4.83. AXI_DMA_RX_CRC_WIDTH_CH*n*_REG (*n*: 0-2) (0x004C+0x68**n*)



AXI_DMA_RX_CRC_WIDTH_CH n Configures the CRC result width for RX channel n . 0: ≤ 8 bits

1: ≤ 16 bits

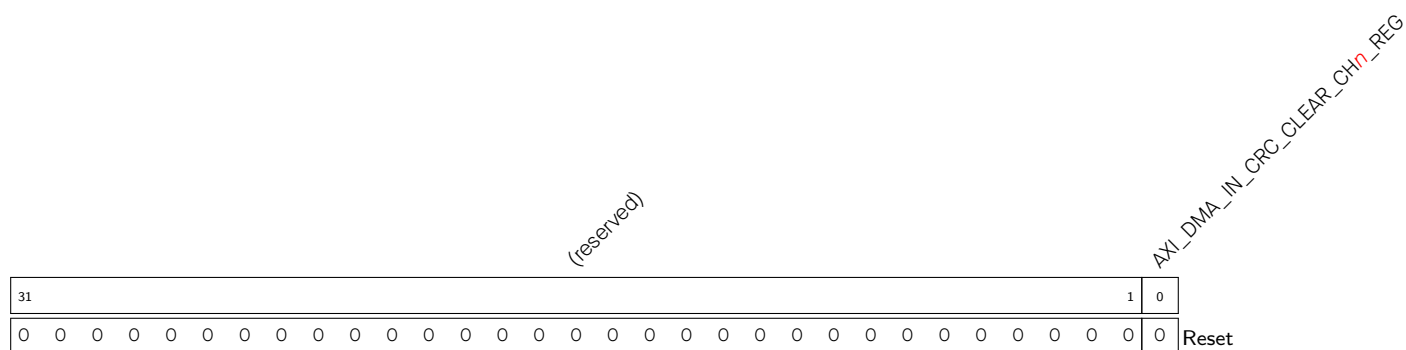
2: ≤ 24 bits

3: ≤ 32 bits

(R/W)

AXI_DMA_RX_CRC_LAUTCH_FLGA_CH n Write 1 and then 0 to latch the values of AXI_DMA_RX_CRC_EN_ADDR_CH n , AXI_DMA_RX_CRC_DATA_EN_ADDR_CH n , AXI_DMA_RX_CRC_EN_WR_DATA_CH n , and AXI_DMA_RX_CRC_DATA_EN_WR_DATA_CH n . (R/W)

Register 4.84. AXI_DMA_IN_CRC_CLEAR_CH n _REG (n : 0-2) (0x0050+0x68* n)

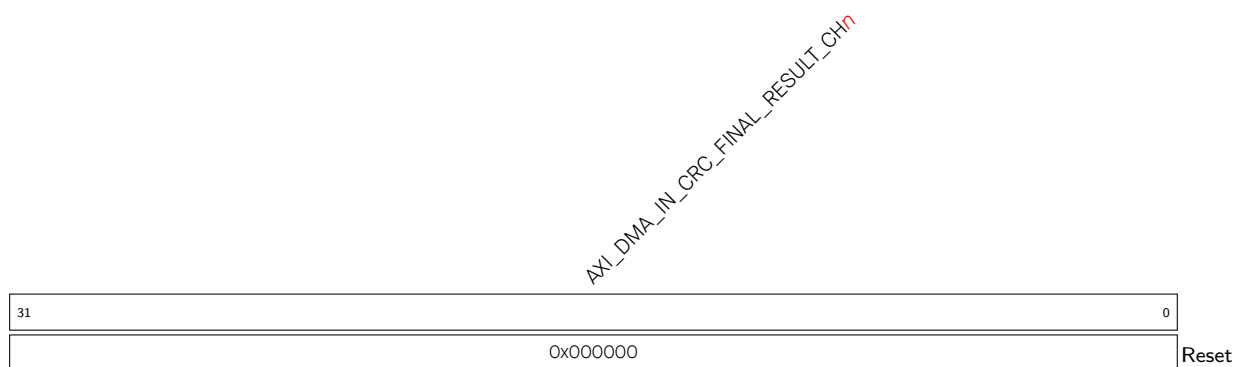


AXI_DMA_IN_CRC_CLEAR_CH n _REG Configures whether to clear the CRC result for RX channel n .

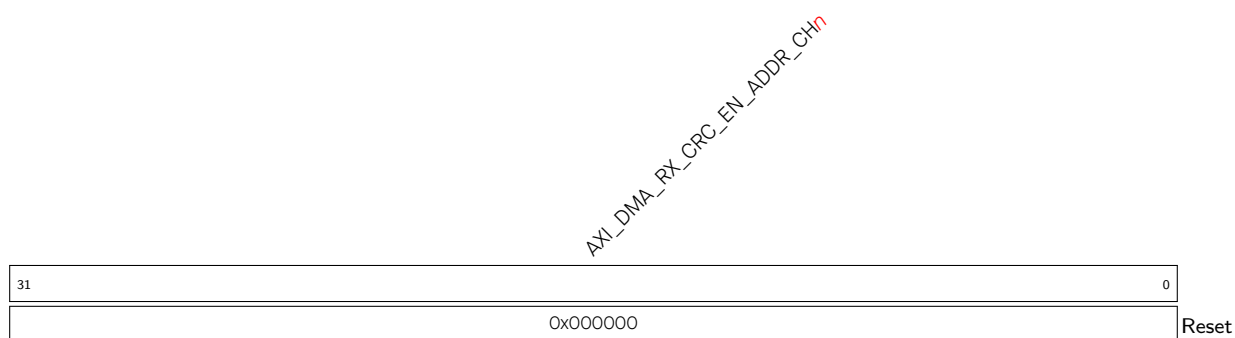
0: Not clear

1: Clear

(R/W)

Register 4.85. AXI_DMA_IN_CRC_FINAL_RESULT_CH n _REG (n : 0-2) (0x0054+0x68* n)

AXI_DMA_IN_CRC_FINAL_RESULT_CH n Represents the CRC result for RX channel n . (RO)

Register 4.86. AXI_DMA_RX_CRC_EN_ADDR_CH n _REG (n : 0-2) (0x005C+0x68* n)

AXI_DMA_RX_CRC_EN_ADDR_CH n Configures at which bit of the CRC result the AXI_DMA_RX_CRC_EN_WR_DATA_CH n _REG register targets for TX channel n . (R/W)

(reserved)

AXI_DMA_RX_CRC_DATA_EN_WR_DATA_CH1

(R/W)

AXI_DMA_RX_CRC_DATA_EN_ADDR_CH17

0x000000

ESP32-P4 TRM
PRELIMINARY

Register 4.89. AXI_DMA_OUT_CONFO_CH n _REG(n : 0-2) (0x0148+0x68* n)

(reserved)																						AXI_DMA_OUTDSCR_BURST_EN_CH ⁿ					AXI_DMA_OUT_ECC_AEC_EN_CH ⁿ					AXI_DMA_OUT_CMD_DISABLE_CH ⁿ					AXI_DMA_OUT_BURST_SIZE_SEL_CH ⁿ					AXI_DMA_OUT_ETM_EN_CH ⁿ					AXI_DMA_OUT_EOF_MODE_CH ⁿ					AXI_DMA_OUT_AUTO_WRRBACK_CH ⁿ					AXI_DMA_OUT_LOOP_RST_CH ⁿ																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																														
31											11											10	9	8	7	5		4	3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																							
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

AXI_DMA_OUT_RST_CH n Configures the reset state of TX channel n FSM and TX FIFO pointer.

0: Release reset

1: Reset

(R/W)

AXI_DMA_OUT_LOOP_TEST_CH n Reserved. (R/W)

AXI_DMA_OUT_AUTO_WRBK_CH n Configures whether to enable automatic outlink write-back when all the data in TX FIFO has been transmitted.

0: Disable

1: Enable

(R/W)

AXI_DMA_OUT_EOF_MODE_CH n Configures when to generate EOF flag.

0: EOF flag for TX channel n is generated when data to be transmitted has been pushed into FIFO in AXI DMA.

1: EOF flag for TX channel n is generated when data to be transmitted has been popped from FIFO in AXI DMA.

(R/W)

AXI_DMA_OUT_ETM_EN_CH n Configures whether to enable ETM control for TX channel n .

0: Disable

1: Enable

(R/W)

AXI_DMA_OUT_BURST_SIZE_SEL_CH n Configures the burst length for TX channel n .

0: 8 bytes

1: 16 bytes

2: 32 bytes

3: 64 bytes

4: 128 bytes

5 ~ 7: Invalid

(R/W)

AXI_DMA_OUT_CMD_DISABLE_CH n Configures whether to disable command on TX channel n .

0: Enable

1: Disable

(R/W)

Continued on the next page...

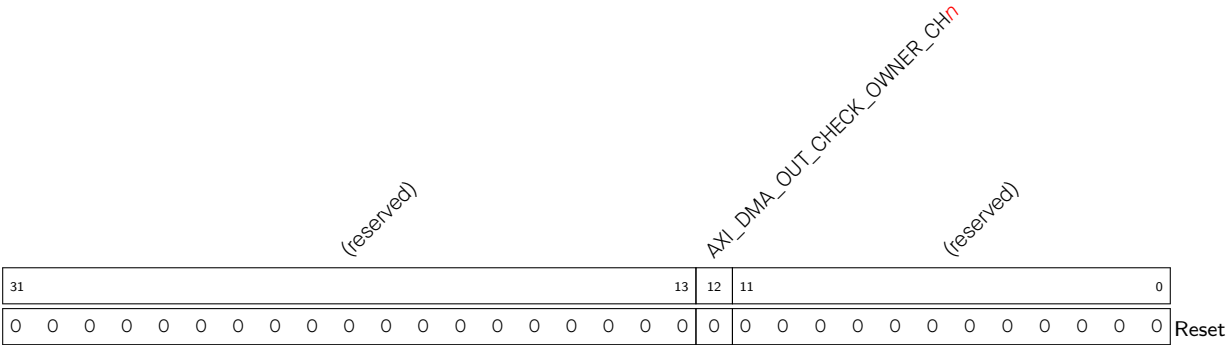
Register 4.89. AXI_DMA_OUT_CONFO_CH n _REG(n : 0-2) (0x0148+0x68* n)

Continued from the previous page...

AXI_DMA_OUT_ECC_AEC_EN_CH n Configures whether AXI DMA can access external memory space for ECC and AES via TX channel n .
0: Not access
1: Access
(R/W)

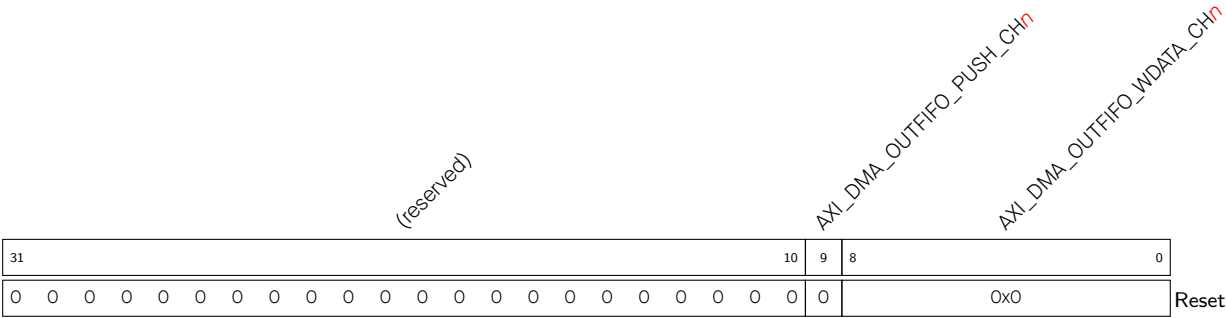
AXI_DMA_OUTDSCR_BURST_EN_CH n Configures whether to enable INCR burst transfer for TX channel n reading descriptors when accessing internal memory.
0: Disable
1: Enable
(R/W)

Register 4.90. AXI_DMA_OUT_CONF1_CH n _REG (n : 0-2) (0x014C+0x68* n)



AXI_DMA_OUT_CHECK_OWNER_CH n Configures whether to enable owner bit check for TX channel n .
0: Disable
1: Enable
(R/W)

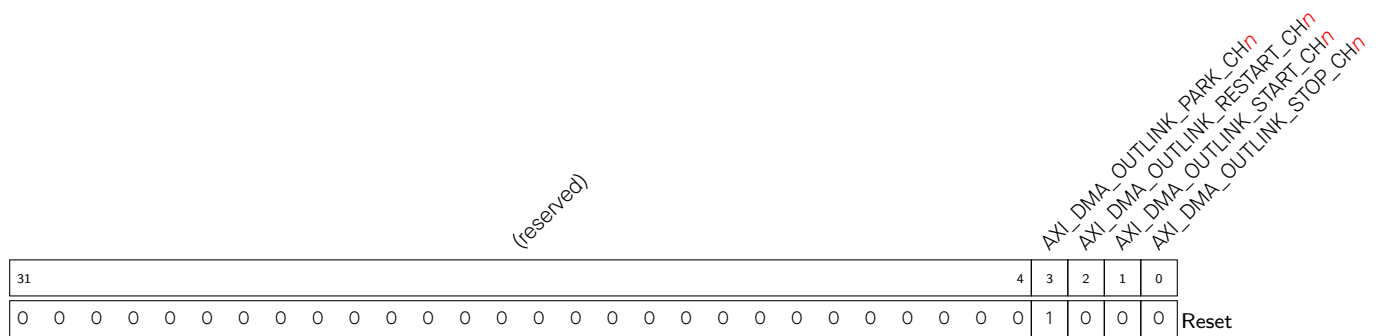
Register 4.91. AXI_DMA_OUT_PUSH_CHn_REG (n: 0-2) (0x0154+0x68*n)



AXI_DMA_OUTFIFO_WDATA_CHn Represents the data that need to be pushed into AXI DMA TX FIFO. (R/W)

AXI_DMA_OUTFIFO_PUSH_CHn Configures whether to push data into AXI DMA TX FIFO.

- 0: Invalid. No effect
- 1: Push (WT)

Register 4.92. AXI_DMA_OUT_LINK1_CH n _REG (n : 0-2) (0x0158+0x68* n)

AXI_DMA_OUTLINK_STOP_CH n Configures whether to stop TX channel n from transmitting data.

0: Invalid. No effect

1: Stop

(WT)

AXI_DMA_OUTLINK_START_CH n Configures whether to enable TX channel n for data transfer.

0: Disable

1: Enable

(WT)

AXI_DMA_OUTLINK_RESTART_CH n Configures whether to restart TX channel n for AXI DMA transfer.

0: Invalid. No effect

1: Restart

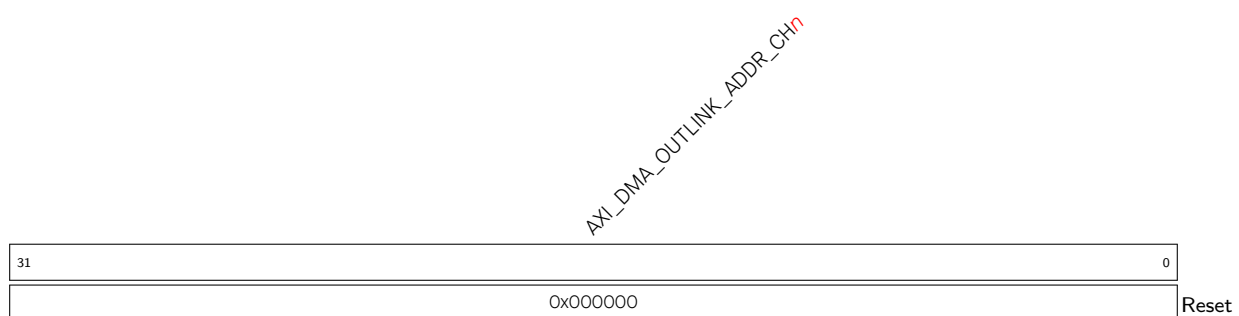
(WT)

AXI_DMA_OUTLINK_PARK_CH n Represents the status of the transmit descriptor's FSM.

0: Running

1: Idle

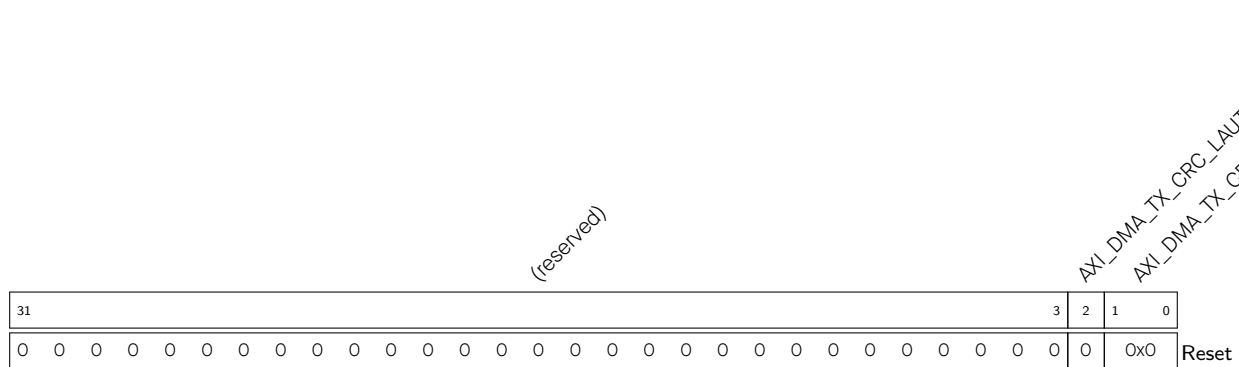
(RO)

Register 4.93. AXI_DMA_OUT_LINK2_CH n _REG (n : 0-2) (0x015C+0x68* n)

AXI_DMA_OUTLINK_ADDR_CH n Represents the first transmit descriptor's address. (R/W)

Register 4.94. AXI_DMA_OUT_CRC_INIT_DATA_CH n _REG (n : 0-2) (0x0180+0x68* n)

AXI_DMA_OUT_CRC_INIT_DATA_CH n Configures the CRC initial value for TX channel n . (R/W)

Register 4.95. AXI_DMA_TX_CRC_WIDTH_CH n _REG (n : 0-2) (0x0184+0x68* n)

AXI_DMA_TX_CRC_WIDTH_CH n Configures the CRC result width for TX channel n . 0: ≤ 8 bits

1: ≤ 16 bits

2: ≤ 24 bits

3: ≤ 32 bits

(R/W)

AXI_DMA_TX_CRC_LAUTCH_FLGA_CH n Write 1 and then 0 to latch the values of AXI_DMA_TX_CRC_EN_ADDR_CH n , AXI_DMA_TX_CRC_DATA_EN_ADDR_CH n , AXI_DMA_TX_CRC_EN_WR_DATA_CH n , and AXI_DMA_TX_CRC_DATA_EN_WR_DATA_CH n . (R/W)

Register 4.96. AXI_DMA_OUT_CRC_CLEAR_CH n _REG (n : 0-2) (0x0188+0x68* n)

[illegible]

AXI_DMA_OUT_CRC_CLEAR_CH n _REG Configures whether to clear the CRC result for TX channel n .

0: Not clear

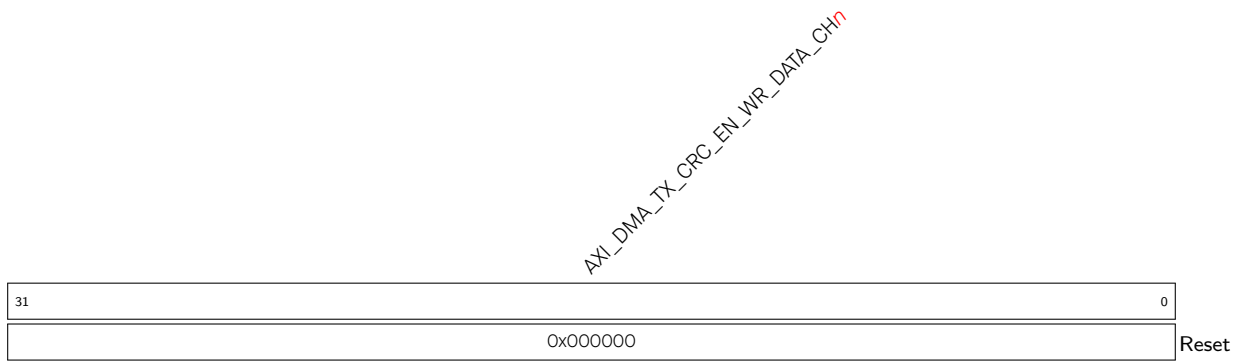
1: Clear

(R/W)

Register 4.97. AXI_DMA_OUT_CRC_FINAL_RESULT_CH n _REG (n : 0-2) (0x018C+0x68* n)

Diagram of the AXI DMA OUT CRC FINAL RESULT CHn register. The register is 32 bits wide, with bit 31 on the left and bit 0 on the right. The value 0x000000 is shown in the lower part of the register. A label 'AXI_DMA_OUT_CRC_FINAL_RESULT_CHn' is written diagonally across the register, with a red 'n' at the end. A 'Reset' label is at the bottom right.

AXI_DMA_OUT_CRC_FINAL_RESULT_CH n Represents the CRC result for TX channel n . (RO)

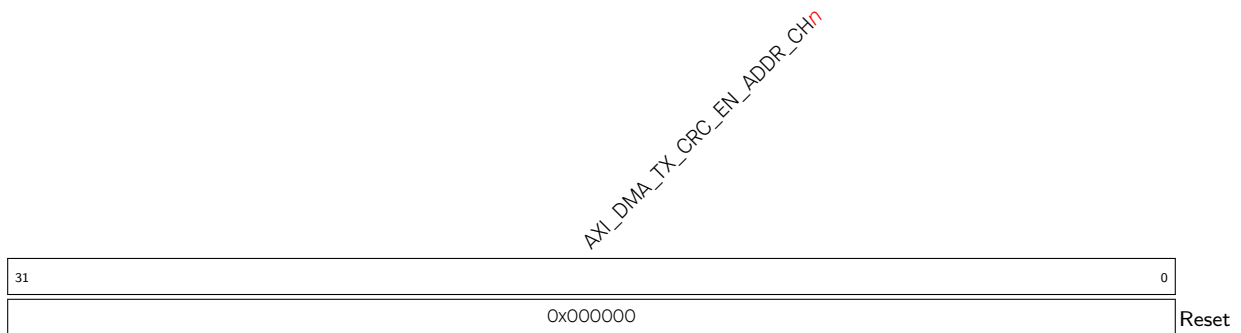
Register 4.98. AXI_DMA_TX_CRC_EN_WR_DATA_CH_{*n*}_REG (*n*: 0-2) (0x0190+0x68n*)**

AXI_DMA_TX_CRC_EN_WR_DATA_CH_{*n*} Configures whether to include each bit of the intermediate result in the CRC calculation matrix for TX channel *n*.

0: Not include

1: Include

(R/W)

Register 4.99. AXI_DMA_TX_CRC_EN_ADDR_CH_{*n*}_REG (*n*: 0-2) (0x0194+0x68n*)**

AXI_DMA_TX_CRC_EN_ADDR_CH_{*n*} Configures at which bit of the CRC result the AXI_DMA_TX_CRC_EN_WR_DATA_CH_{*n*}_REG register targets for TX channel *n*. (R/W)

(reserved)

AXI_DMA_TX_CRC_DATA_EN_WR_DATA_CHⁿ

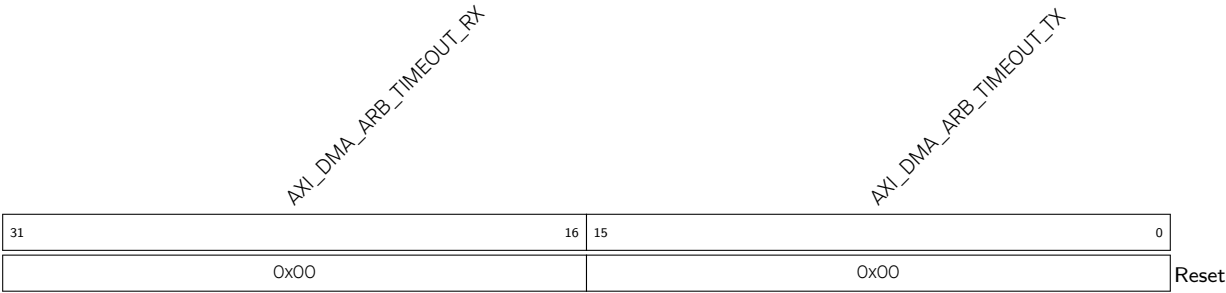
(R/W)

AXI_DMA_TX_ORC_DATA_EN_ADDR_CH10

0x000000

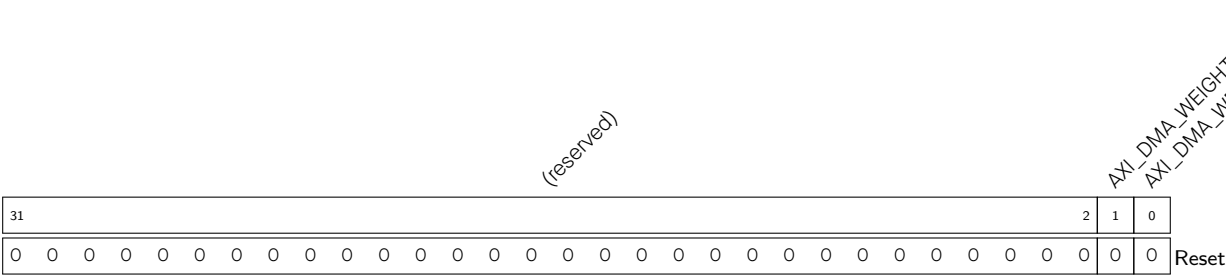
AAXI_DMA_TX_CRC_DATA_EN_WR_DATA_CH*n*_REG register targets for TX channel *n*.(R/W)

Register 4.102. AXI_DMA_ARB_TIMEOUT_REG (0x0270)



- AXI_DMA_ARB_TIMEOUT_TX** Configures the time slot for TX. Measurement unit: AXI bus clock cycle. (R/W)
- AXI_DMA_ARB_TIMEOUT_RX** Configures the time slot for RX. Measurement unit: AXI bus clock cycle. (R/W)

Register 4.103. AXI_DMA_WEIGHT_EN_REG (0x0274)



- AXI_DMA_WEIGHT_EN_TX** Configures whether to enable weight arbitration for TX.
- 0: Disable
 - 1: Enable
- (R/W)
- AXI_DMA_WEIGHT_EN_RX** Configures whether to enable weight arbitration for RX.
- 0: Disable
 - 1: Enable
- (R/W)

Register 4.104. AXI_DMA_IN_MEM_CONF_REG (0x0278)

(reserved)																										AXI_DMA_OUT_MEM_FORCE_PD AXI_DMA_OUT_MEM_FORCE_PU AXI_DMA_OUT_MEM_CLK_FORCE_EN AXI_DMA_IN_MEM_FORCE_PD AXI_DMA_IN_MEM_FORCE_PU AXI_DMA_IN_MEM_CLK_FORCE_EN																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																
31																										6	5	4	3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																										
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Reset

AXI_DMA_IN_MEM_CLK_FORCE_EN Configures memory clock gating for RX.

0: Support clock only when AXI DMA accesses memory

1: Always force the clock on

(R/W)

AXI_DMA_IN_MEM_FORCE_PU Configures whether to force power up memory for RX.

0: Not force power up

1: Force power up

(R/W)

AXI_DMA_IN_MEM_FORCE_PD Configures whether to force power down memory for RX.

0: Not force power down

1: Force power down

(R/W)

AXI_DMA_OUT_MEM_CLK_FORCE_EN Configures memory clock gating for TX.

0: Support clock only when AXI DMA accesses memory

1: Always force the clock on

(R/W)

AXI_DMA_OUT_MEM_FORCE_PU Configures whether to force power up memory for TX.

0: Not force power up

1: Force power up

(R/W)

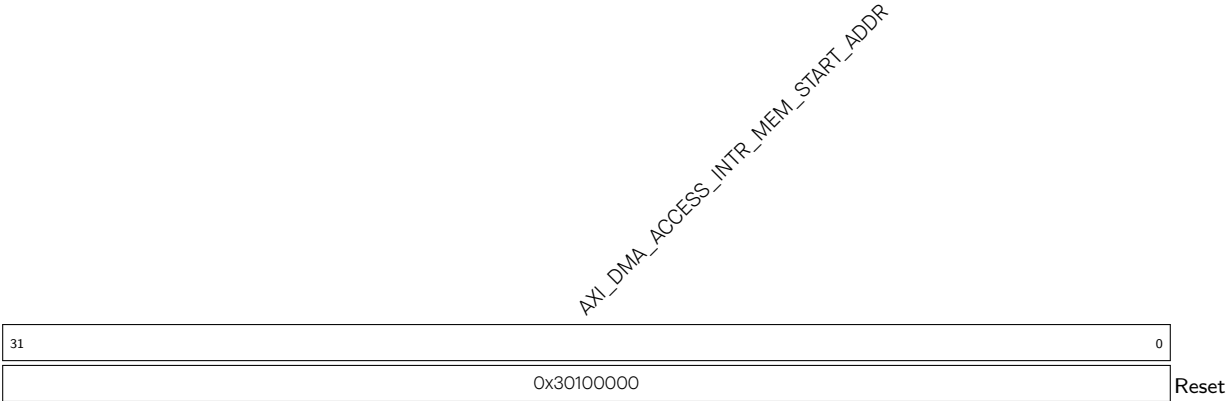
AXI_DMA_OUT_MEM_FORCE_PD Configures whether to force power down memory for TX.

0: Not force power down

1: Force power down

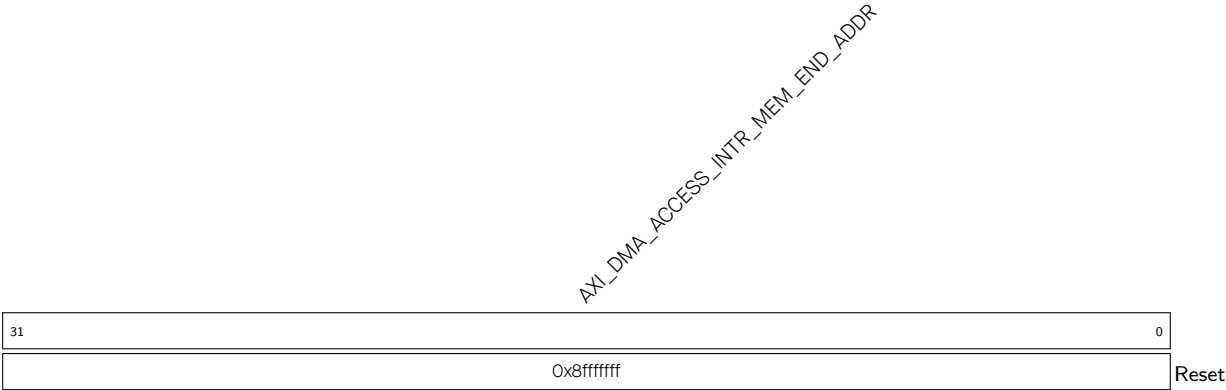
(R/W)

Register 4.105. AXI_DMA_INTR_MEM_START_ADDR_REG (0x027C)

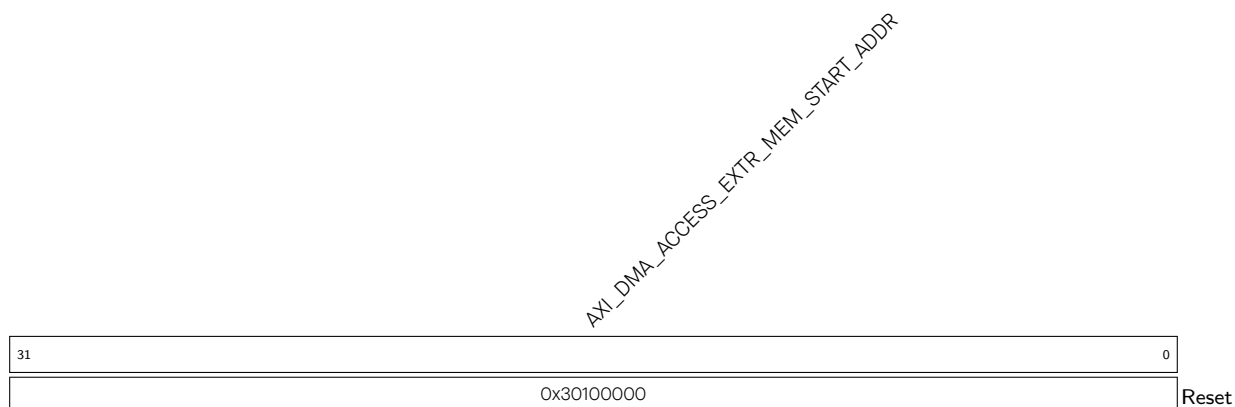


AXI_DMA_ACCESS_INTR_MEM_START_ADDR Configures the start address of the accessible internal memory address space. (R/W)

Register 4.106. AXI_DMA_INTR_MEM_END_ADDR_REG (0x0280)



AXI_DMA_ACCESS_INTR_MEM_END_ADDR Configures the end address of the accessible internal memory address space. (R/W)

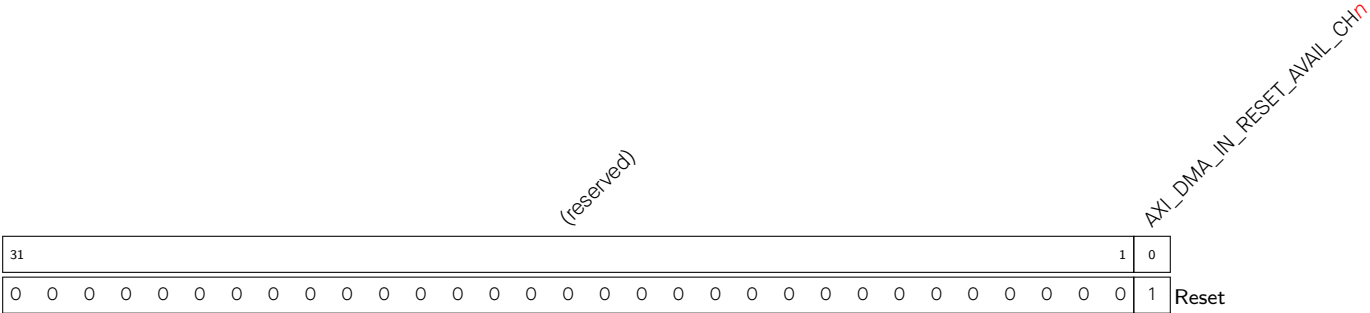
Register 4.107. AXI_DMA_EXTR_MEM_START_ADDR_REG (0x0284)

AXI_DMA_ACCESS_EXTR_MEM_START_ADDR Configures the start address of the accessible external memory address space. (R/W)

Register 4.108. AXI_DMA_EXTR_MEM_END_ADDR_REG (0x0288)

AXI_DMA_ACCESS_EXTR_MEM_END_ADDR Configures the end address of the accessible external memory address space. (R/W)

Register 4.109. AXI_DMA_IN_RESET_AVAIL_CH n _REG (n : 0-2) (0x028C+0x4* n)



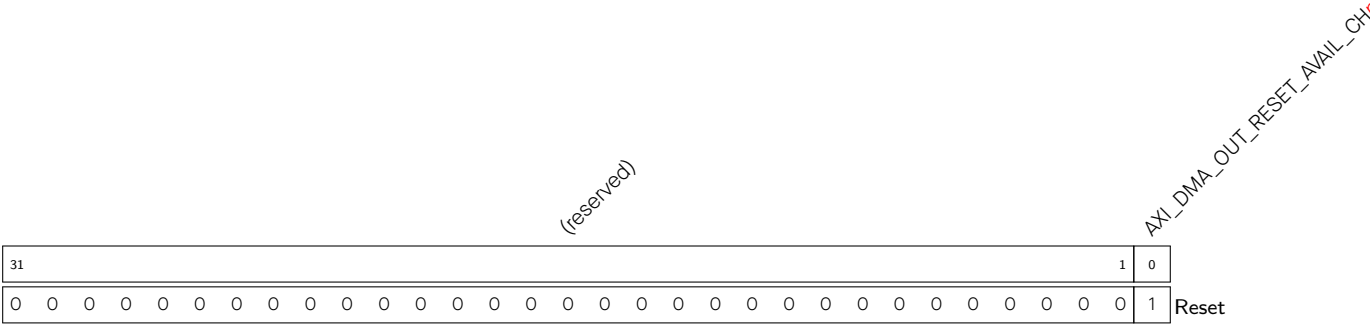
AXI_DMA_IN_RESET_AVAIL_CH n Represents whether RX channel n can be reset.

0: Cannot be reset by software

1: Can be reset

(RO)

Register 4.110. AXI_DMA_OUT_RESET_AVAIL_CH n _REG (n : 0-2) (0x0298+0x4* n)



AXI_DMA_OUT_RESET_AVAIL_CH n Represents whether TX channel n can be reset.

0: Cannot be reset by software

1: Can be reset

. (RO)

Register 4.111. AXI_DMA_MISC_CONF_REG (0x02A8)

(reserved)																										AXI_DMA_CLK_EN AXI_DMA_ARB_PRI_DIS (reserved) AXI_DMA_AXIM_RST_RD_INTER AXI_DMA_AXIM_RST_WR_INTER					
31																					5	4	3	2	1	0	Reset				
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

AXI_DMA_AXIM_RST_WR_INTER Write 1 and then 0 to reset the internal AXI write FSM. (R/W)

AXI_DMA_AXIM_RST_RD_INTER Write 1 and then 0 to reset the internal AXI read FSM. (R/W)

AXI_DMA_ARB_PRI_DIS Configures whether to disable the priority arbitration.

- 0: Enable
- 1: Disable
- (R/W)

AXI_DMA_CLK_EN Configures AXI DMA clock gating.

- 0: Support clock only when the application writes registers.
- 1: Always force the clock on for registers.
- (R/W)

Register 4.112. AXI_DMA_INFIFO_STATUS_CH_n_REG (*n*: 0-2) (0x0018+0x68n*)**

AXI_DMA_IN_BUF_HUNGRY_CH _n AXI_DMA_IN_REMAIN_UNDER_8B_CH _n AXI_DMA_IN_REMAIN_UNDER_7B_CH _n AXI_DMA_IN_REMAIN_UNDER_6B_CH _n AXI_DMA_IN_REMAIN_UNDER_5B_CH _n AXI_DMA_IN_REMAIN_UNDER_4B_CH _n AXI_DMA_IN_REMAIN_UNDER_3B_CH _n AXI_DMA_IN_REMAIN_UNDER_2B_CH _n AXI_DMA_IN_REMAIN_UNDER_1B_CH _n (reserved)																							AXI_DMA_INFIFO_L2_OVF_CH _n AXI_DMA_INFIFO_L2_UDF_CH _n AXI_DMA_INFIFO_L2_EMPTY_CH _n AXI_DMA_INFIFO_L2_FULL_CH _n AXI_DMA_INFIFO_L1_OVF_CH _n AXI_DMA_INFIFO_L1_UDF_CH _n AXI_DMA_INFIFO_L1_EMPTY_CH _n AXI_DMA_INFIFO_L1_FULL_CH _n AXI_DMA_INFIFO_L3_OVF_CH _n AXI_DMA_INFIFO_L3_UDF_CH _n AXI_DMA_INFIFO_L3_CNT_CH _n AXI_DMA_INFIFO_L3_EMPTY_CH _n AXI_DMA_INFIFO_L3_FULL_CH _n									
31	30	29	28	27	26	25	24	23	22				18	17	16	15	14	13	12	11	10	9	8	7				2	1	0		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0			1			1	Reset

AXI_DMA_INFIFO_L3_FULL_CH_n Represents whether L3 RX FIFO is full.

0: Not Full

1: Full

(RO)

AXI_DMA_INFIFO_L3_EMPTY_CH_n Represents whether L3 RX FIFO is empty.

0: Not empty

1: Empty

(RO)

AXI_DMA_INFIFO_L3_CNT_CH_n Represents the number of data bytes in L3 RX FIFO for RX channel *n*. (RO)

AXI_DMA_INFIFO_L3_UDF_CH_n Represents whether L3 RX FIFO underflows.

0: No underflow

1: Underflow

(RO)

AXI_DMA_INFIFO_L3_OVF_CH_n Represents whether L3 RX FIFO overflows.

0: No overflow

1: Overflow

(RO)

AXI_DMA_INFIFO_L1_FULL_CH_n Represents whether L1 RX FIFO is full.

0: Not Full

1: Full

(RO)

AXI_DMA_INFIFO_L1_EMPTY_CH_n Represents whether L1 RX FIFO is empty.

0: Not empty

1: Empty

(RO)

AXI_DMA_INFIFO_L1_UDF_CH_n Represents whether L1 RX FIFO underflows.

0: No underflow

1: Underflow

(RO)

Continued on the next page...

Register 4.112. AXI_DMA_INFIFO_STATUS_CH n _REG (n : 0-2) (0x0018+0x68* n)

Continued from the previous page...

AXI_DMA_INFIFO_L1_OVF_CH n Represents whether L1 RX FIFO overflows.

0: No overflow

1: Overflow

(RO)

AXI_DMA_INFIFO_L2_FULL_CH n Represents whether L2 RX FIFO is full.

0: Not Full

1: Full

(RO)

AXI_DMA_INFIFO_L2_EMPTY_CH n Represents whether L1 RX FIFO is empty.

0: Not empty

1: Empty

(RO)

AXI_DMA_INFIFO_L2_UDF_CH n Represents whether L2 RX FIFO underflows.

0: No underflow

1: Underflow

(RO)

AXI_DMA_INFIFO_L2_OVF_CH n Represents whether L2 RX FIFO overflows.

0: No overflow

1: Overflow

(RO)

AXI_DMA_IN_REMAIN_UNDER_1B_CH n Reserved.(RO)

AXI_DMA_IN_REMAIN_UNDER_2B_CH n Reserved.(RO)

AXI_DMA_IN_REMAIN_UNDER_3B_CH n Reserved.(RO)

AXI_DMA_IN_REMAIN_UNDER_4B_CH n Reserved.(RO)

AXI_DMA_IN_REMAIN_UNDER_5B_CH n Reserved.(RO)

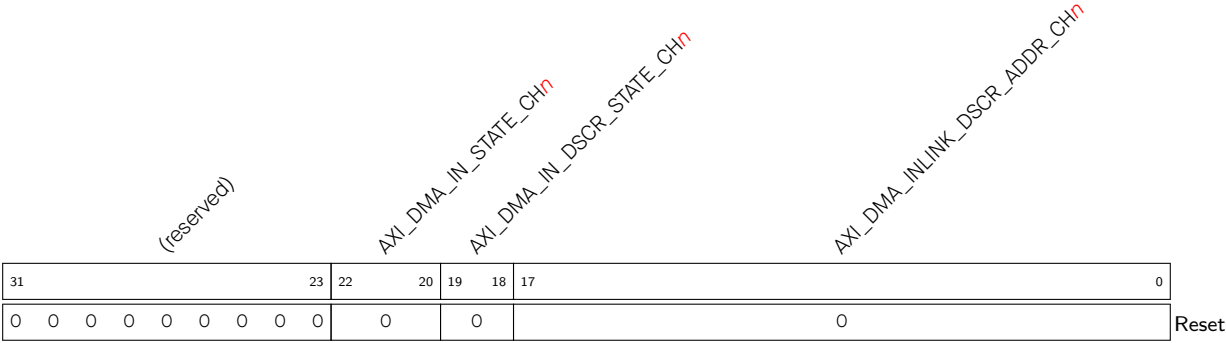
AXI_DMA_IN_REMAIN_UNDER_6B_CH n Reserved.(RO)

AXI_DMA_IN_REMAIN_UNDER_7B_CH n Reserved.(RO)

AXI_DMA_IN_REMAIN_UNDER_8B_CH n Reserved.(RO)

AXI_DMA_IN_BUF_HUNGRY_CH n Reserved.(RO)

Register 4.113. AXI_DMA_IN_STATE_CH n _REG (n : 0-2) (0x0028+0x68* n)

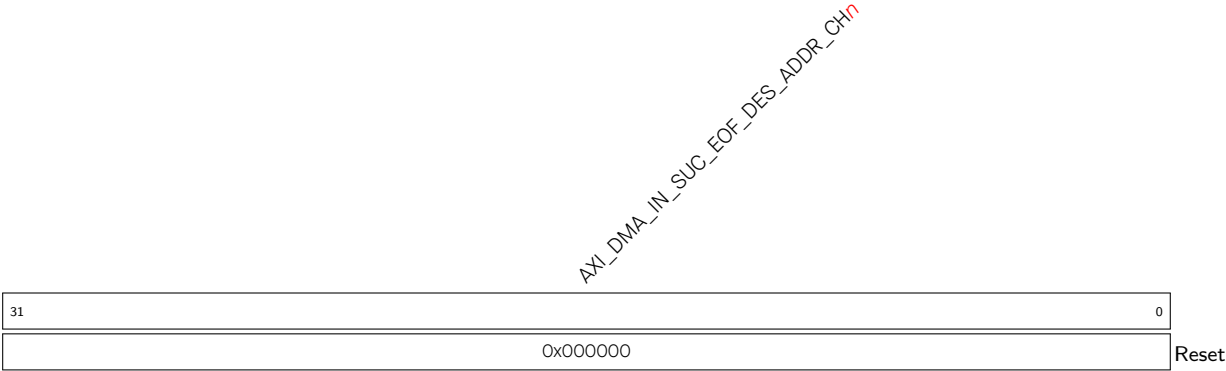


AXI_DMA_INLINK_DSCR_ADDR_CH n Represents the lower 18 bits of the next receive descriptor address that is pre-read (but not processed yet). If the current receive descriptor is the last descriptor, then this field represents the address of the current receive descriptor. (RO)

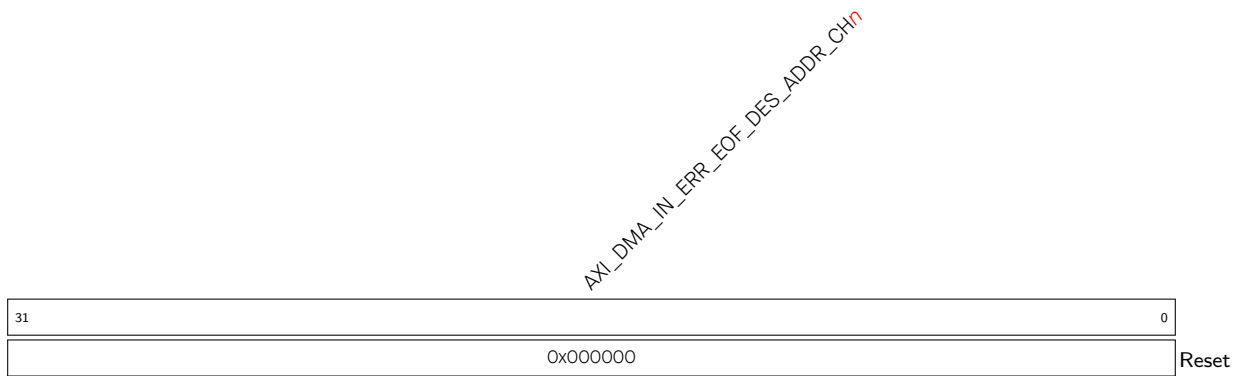
AXI_DMA_IN_DSCR_STATE_CH n Reserved. (RO)

AXI_DMA_IN_STATE_CH n Reserved. (RO)

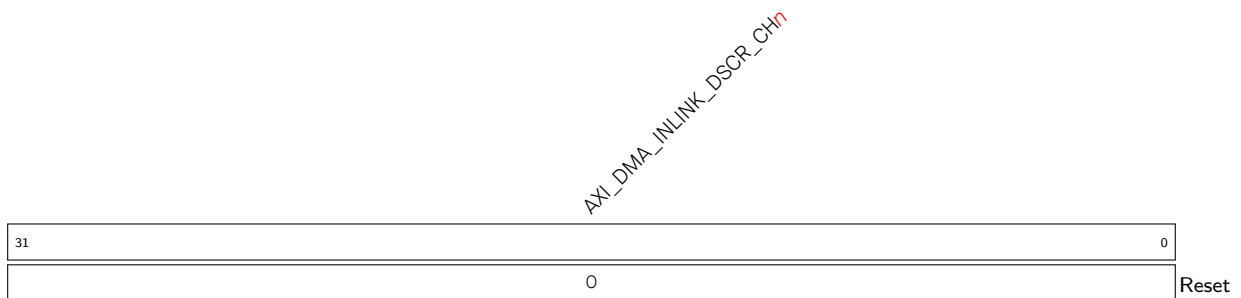
Register 4.114. AXI_DMA_IN_SUC_EOF_DES_ADDR_CH n _REG (n : 0-2) (0x002C+0x68* n)



AXI_DMA_IN_SUC_EOF_DES_ADDR_CH n Represents the address of the receive descriptor when the EOF bit in this descriptor is 1. (RO)

Register 4.115. AXI_DMA_IN_ERR_EOF_DES_ADDR_CH_{*n*}_REG (*n*: 0-2) (0x0030+0x68n*)**

AXI_DMA_IN_ERR_EOF_DES_ADDR_CH_{*n*} Represents the address of the receive descriptor when there are some errors in the currently received data. Valid only for UHCI. (RO)

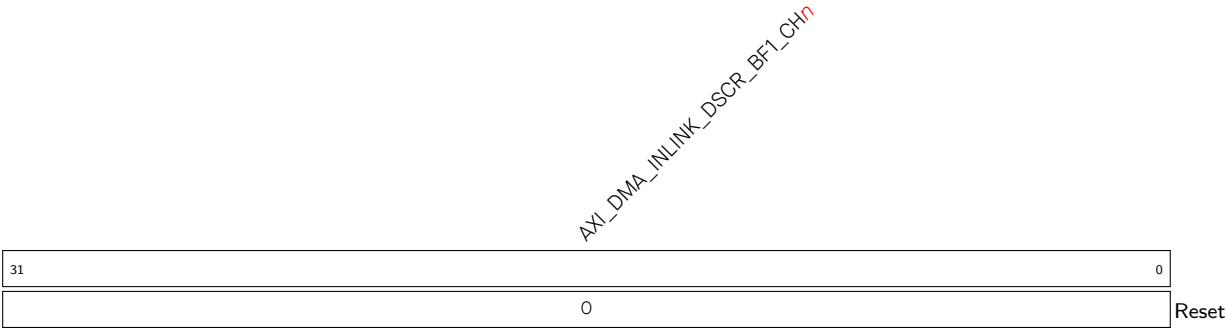
Register 4.116. AXI_DMA_IN_DSCR_CH_{*n*}_REG (*n*: 0-2) (0x0034+0x68n*)**

AXI_DMA_INLINK_DSCR_CH_{*n*} Represents the address of the next receive descriptor x+1 pointed by the current receive descriptor that is pre-read. (RO)

Register 4.117. AXI_DMA_IN_DSCR_BFO_CH_{*n*}_REG (*n*: 0-2) (0x0038+0x68n*)**

AXI_DMA_INLINK_DSCR_BFO_CH_{*n*} Represents the address of the current receive descriptor x that is pre-read. (RO)

Register 4.118. AXI_DMA_IN_DSCR_BF1_CHn_REG (n: 0-2) (0x003C+0x68*n)



AXI_DMA_INLINK_DSCR_BF1_CHn Represents the address of the previous receive descriptor x-1 that is pre-read. (RO)

Register 4.119. AXI_DMA_OUTFIFO_STATUS_CH_n_REG (n: 0-2) (0x0150+0x68*n)

(reserved)										(reserved)										AXI_DMA_OUT_FIFO_L2_OVF_CHn AXI_DMA_OUT_FIFO_L2_UDF_CHn AXI_DMA_OUT_FIFO_L2_EMPTY_CHn AXI_DMA_OUT_FIFO_L1_FULL_CHn AXI_DMA_OUT_FIFO_L1_OVF_CHn AXI_DMA_OUT_FIFO_L1_UDF_CHn AXI_DMA_OUT_FIFO_L1_EMPTY_CHn AXI_DMA_OUT_FIFO_L3_FULL_CHn AXI_DMA_OUT_FIFO_L3_OVF_CHn AXI_DMA_OUT_FIFO_L3_UDF_CHn										AXI_DMA_OUT_FIFO_L3_CNT_CHn AXI_DMA_OUT_FIFO_L3_EMPTY_CHn AXI_DMA_OUT_FIFO_L3_FULL_CHn									
31	30	29	28	27	26	25	24	23	22		18	17	16	15	14	13	12	11	10	9	8	7		2	1	0													
0	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	1	0	0	0	0		0		1	0	Reset												

AXI_DMA_OUTFIFO_L3_FULL_CH_n Represents whether L3 TX FIFO is full.

0: Not Full

1: Full

(RO)

AXI_DMA_OUTFIFO_L3_EMPTY_CH_n Represents whether L3 TX FIFO is empty.

0: Not empty

1: Empty

(RO)

AXI_DMA_OUTFIFO_L3_CNT_CH_n Represents the number of data bytes in L3 TX FIFO for TX channel _n. (RO)

AXI_DMA_OUTFIFO_L3_UDF_CH_n Represents whether L3 TX FIFO underflows.

0: No underflow

1: Underflow

(RO)

AXI_DMA_OUTFIFO_L3_OVF_CH_n Represents whether L3 TX FIFO overflows.

0: No overflow

1: Overflow

(RO)

AXI_DMA_OUTFIFO_L1_FULL_CH_n Represents whether L1 TX FIFO is full.

0: Not Full

1: Full

(RO)

AXI_DMA_OUTFIFO_L1_EMPTY_CH_n Represents whether L1 TX FIFO is empty.

0: Not empty

1: Empty

(RO)

AXI_DMA_OUTFIFO_L1_UDF_CH_n Represents whether L1 TX FIFO underflows.

0: No underflow

1: Underflow

(RO)

Continued on the next page...

Register 4.119. AXI_DMA_OUTFIFO_STATUS_CH n _REG (n : 0-2) (0x0150+0x68* n)

Continued from the previous page...

AXI_DMA_OUTFIFO_L1_OVF_CH n Represents whether L1 TX FIFO overflows.

0: No overflow

1: Overflow

(RO)

AXI_DMA_OUTFIFO_L2_FULL_CH n Represents whether L2 TX FIFO is full.

0: Not Full

1: Full

(RO)

AXI_DMA_OUTFIFO_L2_EMPTY_CH n Represents whether L2 TX FIFO is empty.

0: Not empty

1: Empty

(RO)

AXI_DMA_OUTFIFO_L2_UDF_CH n Represents whether L2 TX FIFO underflows.

0: No underflow

1: Underflow

(RO)

AXI_DMA_OUTFIFO_L2_OVF_CH n Represents whether L2 TX FIFO overflows.

0: No overflow

1: Overflow

(RO)

AXI_DMA_OUT_REMAIN_UNDER_1B_CH n Reserved.(RO)

AXI_DMA_OUT_REMAIN_UNDER_2B_CH n Reserved.(RO)

AXI_DMA_OUT_REMAIN_UNDER_3B_CH n Reserved.(RO)

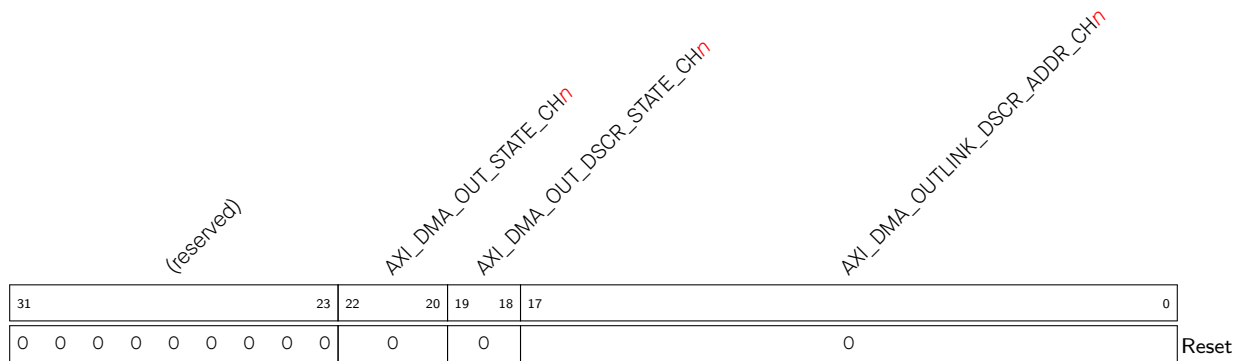
AXI_DMA_OUT_REMAIN_UNDER_4B_CH n Reserved.(RO)

AXI_DMA_OUT_REMAIN_UNDER_5B_CH n Reserved.(RO)

AXI_DMA_OUT_REMAIN_UNDER_6B_CH n Reserved.(RO)

AXI_DMA_OUT_REMAIN_UNDER_7B_CH n Reserved.(RO)

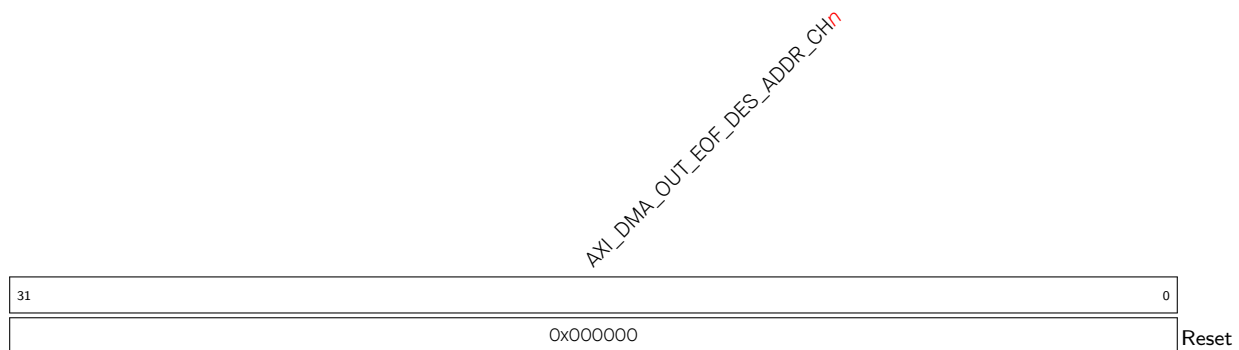
AXI_DMA_OUT_REMAIN_UNDER_8B_CH n Reserved.(RO)

Register 4.120. AXI_DMA_OUT_STATE_CH n _REG (n : 0-2) (0x0160+0x68* n)

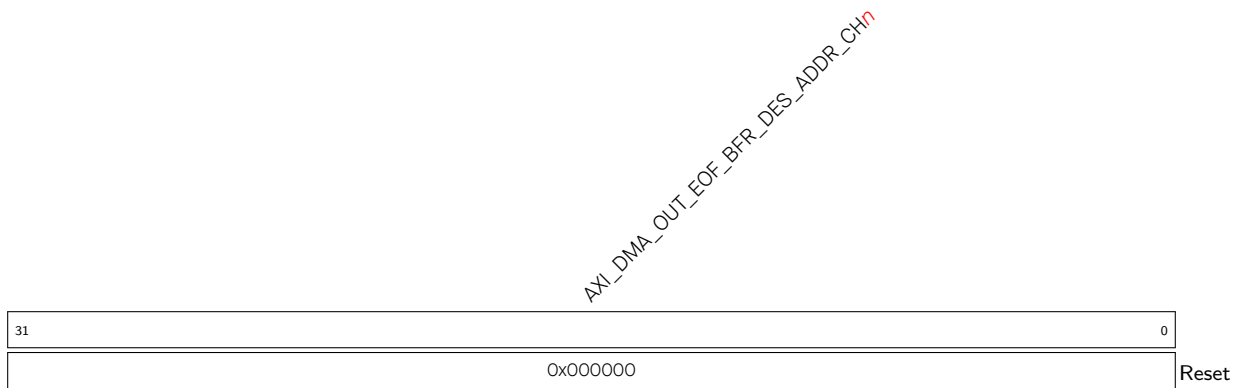
AXI_DMA_OUTLINK_DSCR_ADDR_CH n Represents the lower 18 bits of the next transmit descriptor address that is pre-read (but not processed yet). If the current transmit descriptor is the last descriptor, then this field represents the address of the current transmit descriptor. (RO)

AXI_DMA_OUT_DSCR_STATE_CH n Reserved. (RO)

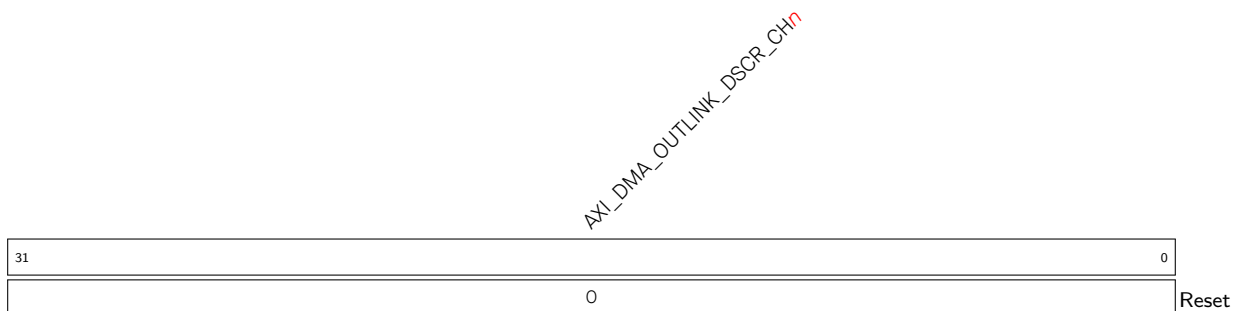
AXI_DMA_OUT_STATE_CH n Reserved. (RO)

Register 4.121. AXI_DMA_OUT_EOF_DES_ADDR_CH n _REG (n : 0-2) (0x0164+0x68* n)

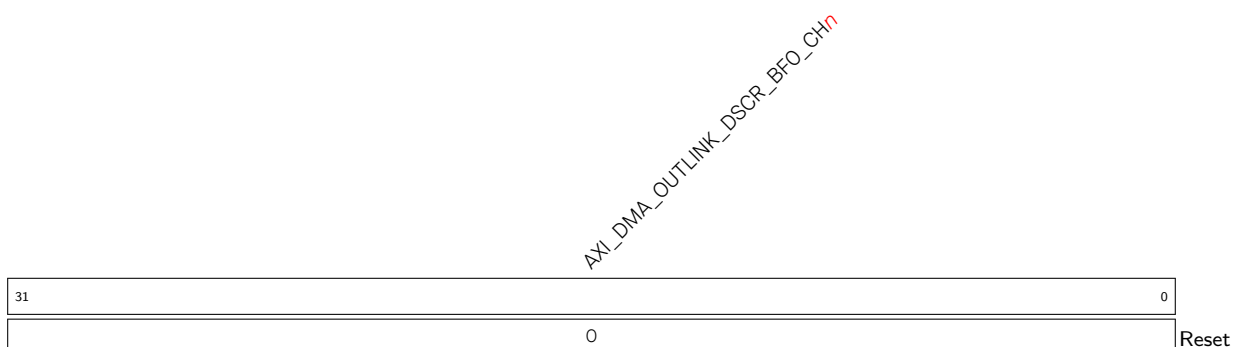
AXI_DMA_OUT_EOF_DES_ADDR_CH n Represents the address of the transmit descriptor when the EOF bit in this descriptor is 1. (RO)

Register 4.122. AXI_DMA_OUT_EOF_BFR_DES_ADDR_CH_{*n*}_REG (*n*: 0-2) (0x0168+0x68n*)**

AXI_DMA_OUT_EOF_BFR_DES_ADDR_CH_{*n*} Represents the address of the transmit descriptor before the last transmit descriptor. (RO)

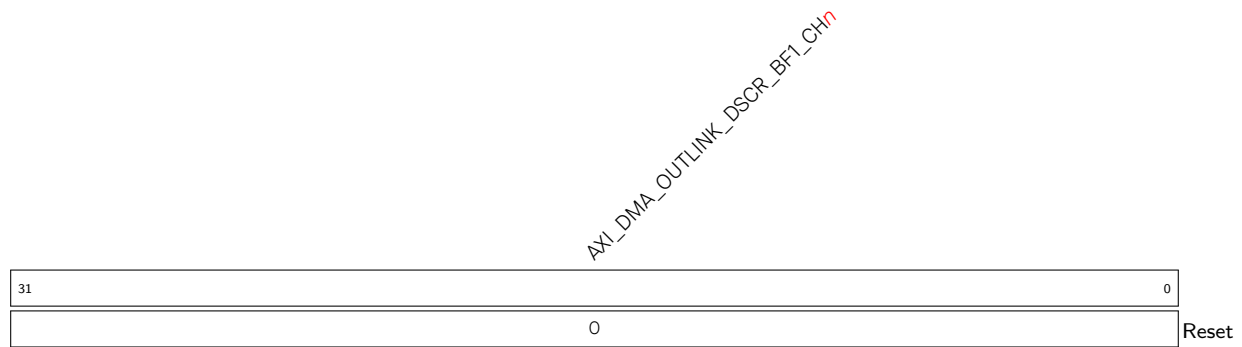
Register 4.123. AXI_DMA_OUT_DSCR_CH_{*n*}_REG (*n*: 0-2) (0x016C+0x68n*)**

AXI_DMA_OUTLINK_DSCR_CH_{*n*} Represents the address of the next transmit descriptor *y*+1 pointed by the current transmit descriptor that is pre-read. (RO)

Register 4.124. AXI_DMA_OUT_DSCR_BFO_CH_{*n*}_REG (*n*: 0-2) (0x0170+0x68n*)**

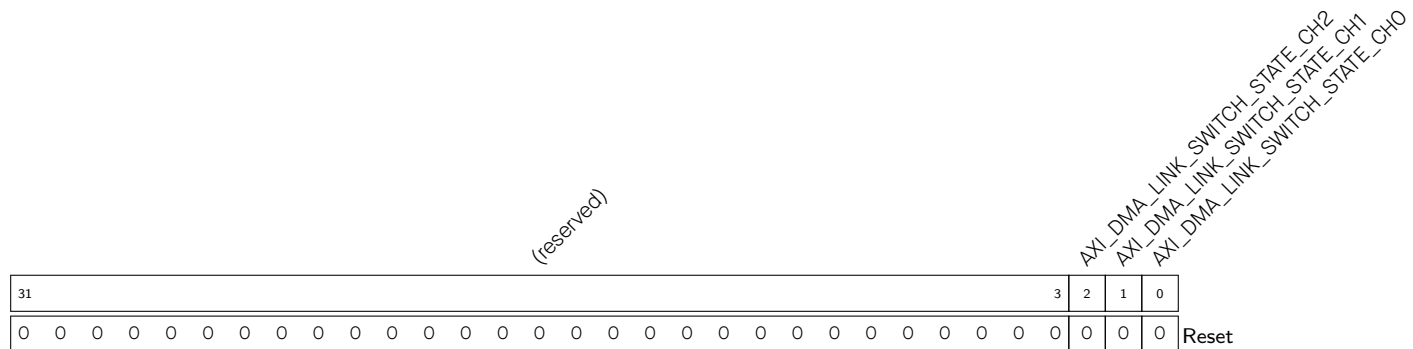
AXI_DMA_OUTLINK_DSCR_BFO_CH_{*n*} Represents the address of the current transmit descriptor *y* that is pre-read. (RO)

Register 4.125. AXI_DMA_OUT_DSCR_BF1_CH n _REG (n : 0-2) (0x0174+0x68* n)



AXI_DMA_OUTLINK_DSCR_BF1_CH_n Represents the address of the previous transmit descriptor y-1 that is pre-read. (RO)

Register 4.126. AXI_DMA_LINK_SWITCH_STATE_REG (0x02DC)



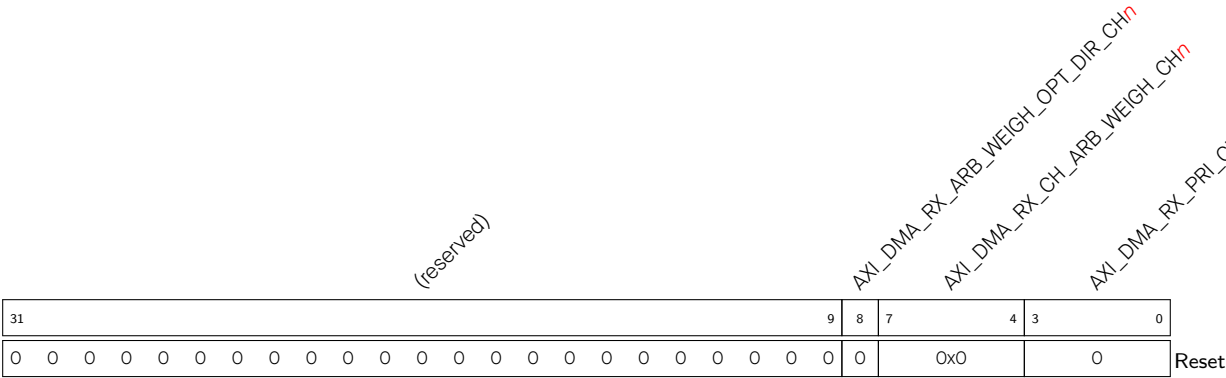
AXI_DMA_LINK_SWITCH_STATE_CH_n Configures whether to switch the linked lists read by AXI DMA.

0: Not switch

1: Switch

(R/W)

Register 4.127. AXI_DMA_IN_PRI_CHn_REG (n: 0-2) (0x0040+0x68*n)

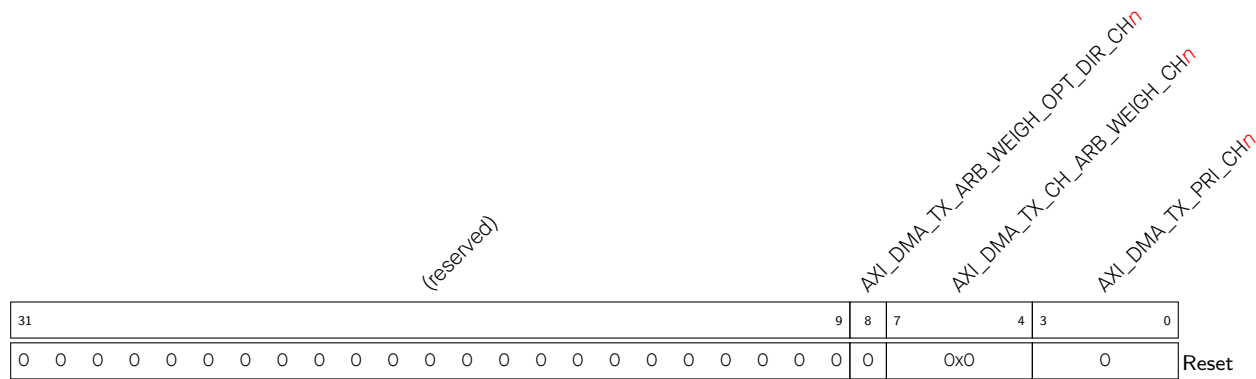


AXI_DMA_RX_PRI_CHn Configures the priority of RX channel *n*. The larger the value, the higher the priority.
Value range: 0 ~ 5. (R/W)

AXI_DMA_RX_CH_ARB_WEIGHT_CHn Configures the weight (i.e the number of tokens) of RX channel *n*.
Value range: 0 ~ 15. (R/W)

AXI_DMA_RX_ARB_WEIGHT_OPT_DIR_CHn Configures whether to enable weight optimization for RX channel *n*.
0: Enable
1: Disable
(R/W)

Register 4.128. AXI_DMA_OUT_PRI_CH*n*_REG (*n*: 0-2) (0x0178+0x68**n*)



AXI_DMA_TX_PRI_CH n Configures the priority of RX channel n . The larger the value, the higher the priority.

Value range: 0 ~ 5. (R/W)

AXI_DMA_TX_CH_ARB_WEIGHT_CH n Configures the weight (i.e the number of tokens) of TX channel n .

Value range: 0 ~ 15. (R/W)

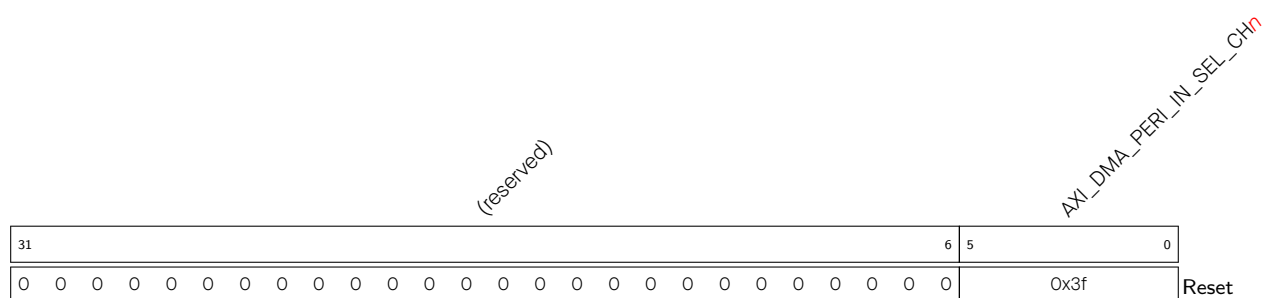
AXI_DMA_TX_ARB_WEIGHT_OPT_DIR_CH n Configures whether to enable weight optimization for RX channel n .

0: Enable

1: Disable

(R/W)

Register 4.129. AXI_DMA_IN_PERI_SEL_CH n _REG (n : 0-2) (0x0044+0x68* n)



AXI_DMA_PERI_IN_SEL_CH n Configures the peripheral connected to RX channel n .

0: LCD_CAM

1: SPI2

2: SPI3

3: PARLIO

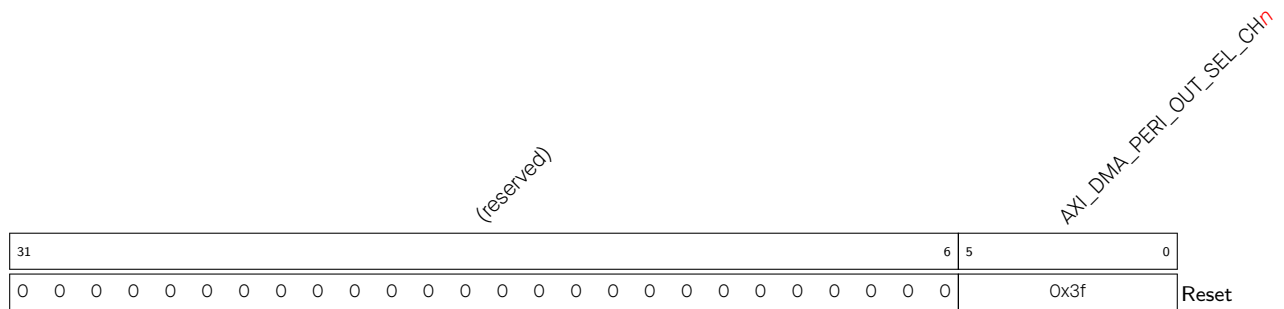
4: AES

5: SHA

6 ~ 15: Dummy-6 ~ Dummy-15

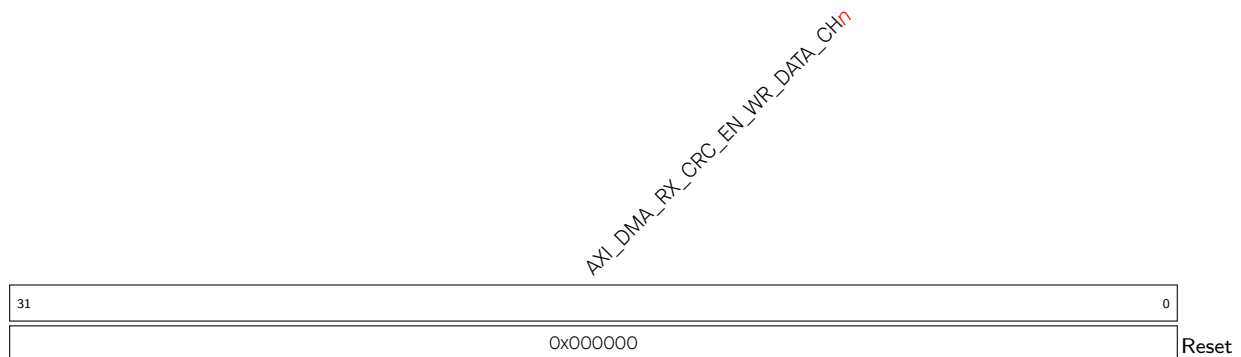
16 ~ 63: Invalid

(R/W)

Register 4.130. AXI_DMA_OUT_PERI_SEL_CH n _REG (n : 0-2) (0x017C+0x68* n)

AXI_DMA_OUT_PERI_SEL_CH n Configures the peripheral connected to TX channel n .

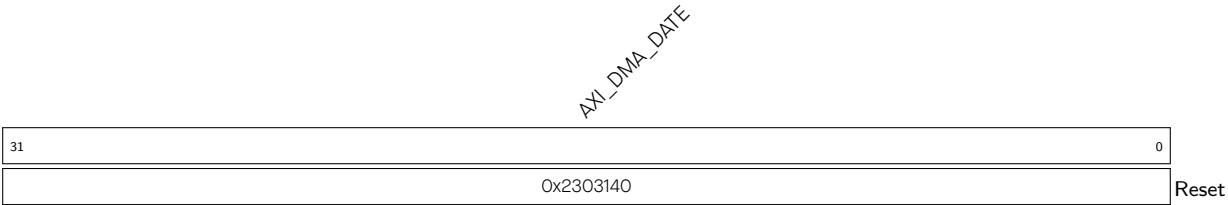
- 0: LCD_CAM
 - 1: SPI2
 - 2: SPI3
 - 3: PARLIO
 - 4: AES
 - 5: SHA
 - 6 ~ 15: Dummy-6 ~ Dummy-15
 - 16 ~ 63: Invalid
- (R/W)

Register 4.131. AXI_DMA_RX_CRC_EN_WR_DATA_CH n _REG (n : 0-2) (0x0058+0x68* n)

AXI_DMA_RX_CRC_EN_WR_DATA_CH n Configures whether to include each bit of the intermediate result in the CRC calculation matrix for RX channel n .

- 0: Not include
 - 1: Include
- (R/W)

Register 4.132. AXI_DMA_DATE_REG (0x02D8)



AXI_DMA_DATE Version control register. (R/W)